Sistemas distribuidos

Material de las unidades 1 a 4

Índice general

I Unidad 1 **1**

1. Arquitectura Cliente - Servidor **2**

2. Cliente.java **3**

2.1. La clase ByteBuffer 4

3. Codificación de texto **6**

3.1. Archivos binarios 6

3.2. Archivos con formato texto 6

3.3. Código ASCII 6

3.4. Código ANSI 6

3.5. Unicode 7

3.6. UTF-8 7

3.7. Archivos texto en Windows y en Unix (Linux) 7

3.8. Programas en Java 8

3.9. Editores de texto 8

3.10. Archivos CSV 8

4. Criptografía y sockets seguros **9**

4.1. Encriptado y desencriptado simétrico 9

4.2. Encriptado y desencriptado asimétrico 9

4.3. Firma digital 10

4.4. Certificado digital 10

4.5. Certificado autofirmado 11

4.6. Certificado firmado por una autoridad certificadora (CA) 11

5. Máquinas virtuales en Azure **13**

5.1. Crear una máquina virtual con Ubuntu 13

5.2. Abrir un puerto de entrada 14

5.3. Detener una máquina virtual 15

5.4. Encender una máquina virtual . . . 15
5.5. Eliminar una máquina virtual . . . 15
5.6. Restablecer la contraseña de una máquina virtual . . . 15
5.7. Crear una imagen de una máquina virtual con Ubuntu . . . 16
5.7.1. Notas importantes . . . 16
5.8. Crear una máquina virtual a partir de una imagen . . . 17

6. Preguntas **19**

7. Proxy **21**
7.1. Arquitectura de un proxy transparente . . . 21
7.2. Implementación de un proxy transparente en Java . . . 21

8. Servidor.java **25**
8.1. Un cliente con reintentos de conexión . . . 26

9. Servidor HTTP **28**
9.1. ¿Qué es un servidor HTTP? . . . 28
9.2. `ServidorHTTP.java` . . . 28
9.3. Respuesta `404 File Not Found` . . . 31
9.4. CORS — Cross-Origin Resource Sharing . . . 32
9.5. El encabezado `Last-Modified` . . . 32
9.6. El encabezado `Cache-Control` . . . 33
9.7. El encabezado `Content-Disposition` . . . 34
9.8. Los encabezados `Server` y `Date` . . . 34
9.9. ¿Cómo obtener la IP del cliente? . . . 34

10. Servidor multithread **35**
10.1. Ejecución paralela y ejecución concurrente . . . 35
10.2. Programación multithread en Java . . . 35
10.3. El método start . . . 36
10.4. El método join . . . 36
10.5. Servidor2.java . . . 37

11. Sincronización del acceso a memoria **39**
11.1. Operaciones de lectura y escritura . . . 39
11.2. Orden escritura-escritura . . . 39
11.3. Orden escritura-lectura . . . 39

11.4. La instrucción `synchronized` . . . 40
11.5. Sincronización de threads . . . 42
11.5.1. El método `wait()` . . . 42
11.5.2. Los métodos `notify()` y `notifyAll()` . . . 43

12. Cliente-servidor con sockets seguros **45**
12.1. `ClienteSSL.java` . . . 46
12.2. `ServidorSSL.java` . . . 47
12.3. Implementación de un servidor HTTPS . . . 49
12.3.1. `ServidorHTTPS.java` . . . 49

13. El tiempo en los sistemas distribuidos **51**
13.1. Sincronización de relojes . . . 52
13.2. Segundos solares . . . 52
13.3. Segundos atómicos . . . 52
13.4. Tiempo atómico internacional (TAI) . . . 53
13.5. Tiempo universal coordinado (UTC) . . . 53
13.6. Network Time Protocol (NTP) . . . 53
13.7. Instalación de NTP en Ubuntu . . . 54
13.8. Conclusiones . . . 54

14. Túnel SSH **55**
14.1. Implementación de un túnel SSH . . . 56
14.2. Autenticación mediante clave pública y clave privada . . . 57
14.2.1. Creación de la clave pública y la clave privada . . . 57

15. Código fuente: Proxy.java **59**

16. Código fuente: ServidorHTTP (variante).java **62**

17. Código fuente: ServidorHTTP.java **65**

18. Código fuente: ServidorHTTPS.java **68**

19. Código fuente: p.html **71**

II Unidad 2 **72**

20. Cómputo en la nube **73**

20.1. Aprovisionamiento de recursos informáticos On-premise . . . 73
20.2. Cómputo como servicio . . . 73
20.3. Fábricas de información . . . 73
20.4. Proveedores de servicios en la nube . . . 74
20.4.1. Amazon . . . 74
20.4.2. Google . . . 74
20.4.3. Microsoft . . . 74
20.5. Definición de cómputo en la nube . . . 75
20.6. Las cinco características esenciales del cómputo en la nube . . . 75
20.6.1. 1. Autoservicio bajo demanda . . . 75
20.6.2. 2. Acceso amplio por medio de la red . . . 75
20.6.3. 3. Recursos compartidos . . . 75
20.6.4. 4. Elasticidad . . . 75
20.6.5. 5. Medición del servicio . . . 76
20.7. Los tres modelos de servicio del cómputo en la nube . . . 76
20.7.1. 1. Infraestructura como servicio . . . 76
20.7.2. 2. Plataforma como servicio . . . 76
20.7.3. 3. Software como servicio . . . 76
20.8. Los cuatro modelos de despliegue del cómputo en la nube . . . 76
20.8.1. 1. Nube privada . . . 76
20.8.2. 2. Nube comunitaria . . . 77
20.8.3. 3. Nube pública . . . 77
20.8.4. 4. Nube híbrida . . . 77

21. Principios del cómputo en la nube **78**
21.1. Servicios virtuales a nivel masivo . . . 78
21.2. Escalabilidad en la nube . . . 78
21.3. Disponibilidad en la nube . . . 78
21.4. Confiabilidad en la nube . . . 79
21.5. Transparencia . . . 79
21.6. Rendimiento . . . 80
21.7. Desempeño . . . 80
21.8. La elasticidad en la nube . . . 80
21.8.1. Escenario On-premise . . . 81
21.8.2. Escenario nube . . . 81

III Unidad 3 **82**

22. Almacenamiento en la nube **83**

22.1. Azure Disk Storage . . . 83
22.2. Discos no administrados . . . 83
22.3. Discos administrados . . . 84
22.3.1. Discos HDD Standard . . . 84
22.3.2. Discos SSD Standard . . . 84
22.3.3. Discos SSD Premium . . . 84
22.3.4. Discos Ultra . . . 84
22.3.5. Crear un disco de datos administrado . . . 85
22.3.6. Adjuntar un disco de datos a una máquina virtual . . . 85
22.3.7. Formatear un disco de datos . . . 86
22.4. Montar un disco de datos . . . 86
22.4.1. Quitar un disco de datos de una máquina virtual . . . 86
22.4.2. Eliminar un disco de datos . . . 86
22.4.3. Encriptado de un disco de datos . . . 87
22.4.4. Cómo leer un disco a nivel RAW . . . 88

23. La clase de hoy vamos a ver el servicio de respaldos en la nube de Azure. **90**

23.1. Microsoft Azure Backup . . . 90
23.2. Punto de restauración . . . 90
23.3. Almacén de Recovery Services . . . 91
23.4. Geo-Redundant storage . . . 91
23.5. Políticas de respaldo . . . 91
23.6. Restauración de una máquina virtual . . . 91
23.7. Costo de Azure Backup . . . 92
23.8. Ventajas de Azure Backup . . . 92
23.9. Referencias . . . 92

24. Actividades individuales a realizar **93**

24.1. Habilitar el respaldo de una máquina virtual en Azure . . . 93
24.2. Iniciar un respaldo completo . . . 93
24.3. Restaurar una máquina virtual . . . 94
24.4. Eliminar un proceso de respaldo . . . 95
24.5. Azure Database for MySQL . . . 97
24.6. Referencias . . . 97

24.7. Crear un servidor en Azure Database for MySQL . . . 97
24.8. Crear una base de datos . . . 99
24.9. Crear un usuario remoto . . . 100
24.10Detener o eliminar la instancia de MySQL . . . 100

25.Balance de carga en la nube **101**
25.1. Azure Load Balancer . . . 101
25.1.1. Balanceador de carga público . . . 101
25.1.2. Balanceador de carga interno . . . 101
25.2. Escalamiento mediante balance de carga . . . 102
25.2.1. Sondeo de estado . . . 102
25.3. Región . . . 102
25.3.1. Datacenter . . . 102
25.3.2. Zona de disponibilidad . . . 103
25.3.3. Conjunto de disponibilidad . . . 103
25.4. Costo del balance de carga en Azure . . . 103

26.Azure Blob Storage **105**
26.1. Notas . . . 105
26.2. Reproducir Vídeo . . . 106
26.3. Referencias . . . 106
26.4. Crear una cuenta de almacenamiento . . . 106
26.5. Crear un contenedor en una cuenta de almacenamiento . . . 106
26.6. Cargar archivos a un contenedor . . . 107
26.7. Revocar una o varias SAS . . . 108
26.8. Notas . . . 108

27.Actividades individuales a realizar **110**
27.1. Descarga y carga de una imagen administrada . . . 110
27.1.1. Descargar la imagen . . . 110
27.1.2. Cargar la imagen . . . 111
27.2. Notas . . . 112
27.3. Agregar una subred a una red virtual . . . 115
27.4. Creación de una máquina virtual y su conexión a una red virtual existente . . . 116
27.5. Creación de una interfaz de red . . . 116
27.6. Añadir una interface de red a una máquina virtual . . . 117
27.7. Quitar una interface de red de una máquina virtual . . . 117

27.8. Agregar una configuración IP a una interface de red . . . 118
27.9. Agregar una IP pública a una máquina virtual . . . 118
27.10Quitar una IP pública a una máquina virtual . . . 119
27.11Creación de una Puerta de enlace de red virtual (VPN Gateway) . . . 119
27.12Creación de una VPN VNet-to-VNet . . . 120

28. OLAP - Online Analytical Processing **122**
28.1. Data warehouse . . . 122
28.1.1. Modelo estrella . . . 122
28.2. Modelo copo de nieve . . . 123
28.3. ROLAP . . . 123
28.4. Cubo OLAP . . . 124

29. Azure Files **125**

30. El sistema de archivos distribuidos NFS **128**
30.1. Instalación de NFS en el servidor . . . 128
30.1.1. Notas de seguridad. . . . 129
30.2. Instalación de NFS en el cliente . . . 130
30.3. Probar el acceso a archivos remotos . . . 131
30.4. Referencia . . . 131

31. NFS sobre un túnel SSH **132**
31.1. Instalación de NFS seguro en el servidor . . . 132
31.2. Instalación de NFS seguro en el cliente . . . 134

32. Clases de direcciones IP v4 **135**
32.1. Classless Inter-Domain Routing (CIDR) . . . 136
32.2. Espacios de direcciones IP . . . 137
32.3. Direcciones IP utilizables y no utilizables . . . 137

33. Redes virtuales en Azure **139**
33.1. Grupo de recursos . . . 139
33.2. Red virtual (VNet) . . . 139
33.3. Subred . . . 140
33.4. Grupo de seguridad de red . . . 140

34. Replicación de un sistema completo en la nube **142**
34.1. Arquitectura de un sistema replicado . . . 142

34.2. Replicación de una arquitectura de tres capas en la nube . . . 143
34.3. Administrador de tráfico SSL . . . 144

35. Tablas agregadas **146**
35.1. MapReduce . . . 148
35.1.1. Job MapReduce . . . 148
35.2. Cómo programar un proceso MapReduce en Java . . . 149
35.3. Azure HDInsight . . . 151

IV Unidad 4 **153**

36. La clase de hoy vamos a explicar cómo utilizar JSON para serializar y deserializar objetos. **154**
36.1. Sintaxis de JSON . . . 154
36.2. Ahora veremos un ejemplo, utilizando GSON (implementación de JSON desarrollada por Google). . . . 156

37. Servicios web **158**
37.1. Conceptos básicos de Servicios web . . . 158
37.1.1. Participación en un servicio web . . . 159
37.2. Servicios web SOAP . . . 159
37.2.1. Componentes de un mensaje SOAP . . . 160
37.2.2. Web Services Description Language (WSDL) . . . 160
37.2.3. A continuación, se explica las secciones del WSDL: . . . 162
37.3. Servicios web REST . . . 164
37.3.1. OpenAPI . . . 165

38. Actividades individuales a realizar **166**

Parte I

Unidad 1

Capítulo 1

Arquitectura Cliente - Servidor

La clase de hoy vamos a ver cómo programar un cliente y un servidor en Java.

Un cliente es un programa que se conecta a un programa servidor. Notar que el cliente inicia la conexión con el servidor.

Una vez que el cliente está conectado al servidor, el cliente puede enviar datos al servidor y el servidor puede mandar datos al cliente. A este tipo de comunicación se le conoce como bi-direccional, debido a que los datos pueden fluir en ambas direcciones.

En particular, los clientes y servidores que utilizaremos en el curso usan sockets TCP.

Para compilar y ejecutar los programas del curso vamos a utilizar JDK8 desde la línea de comandos.

Los que quieran utilizar ambientes de desarrollo como Netbeans o Eclipse pueden hacerlo, sin embargo, en la clase vamos a ejecutar los programas en la línea de comandos.

Capítulo 2

Cliente.java

El programa Cliente.java es un ejemplo de un cliente de sockets TCP que se conecta a un servidor y posteriormente envía y recibe datos.

Primeramente, vamos a crear un socket que se conectará al servidor. En este caso el servidor se llama “localhost“ (computadora local) y el puerto abierto en el servidor es el 50000.

En general el nombre del servidor puede ser un nombre de dominio (como midominio.com o una dirección IP). El número de puerto es un número entero entre 0 y 65535.

Socket conexion = new Socket(“localhost“,50000);

En este caso declaramos una variable de tipo Socket llamada “conexión“ la cual va a contener una instancia de la clase Socket.

Es importante aclarar que antes de crear el socket, el programa servidor debe estar en ejecución y esperando una conexión, de otra manera la instrucción anterior produce una excepción, la cual desde luego debería controlarse dentro de un bloque try.

Para enviar datos al servidor a través del socket, vamos a crear un stream de salida de la siguiente manera:

```
DataOutputStream salida = new DataOutputStream(conexion.getOutputStream());
```

De la misma forma, para leer los datos que envía el servidor a través del socket, creamos un stream de entrada:

```
DataInputStream entrada = new DataInputStream(conexion.getInputStream());
```

Ahora podemos enviar y recibir datos del servidor. Veamos algunos ejemplos.

Vamos a enviar un entero de 32 bits, en este caso el número 123, utilizando el método `writeInt`:

```
salida.writeInt(123);
```

Ahora vamos a enviar un número punto flotante de 64 bits utilizando el método `writeDouble`:

```
salida.writeDouble(1234567890.1234567890);
```

Vamos a enviar la cadena de caracteres “hola“:

```
salida.write(``hola``.getBytes());
```

Debido a que el método write envía un arreglo de bytes, para enviar la cadena de caracteres “hola“ es necesario convertirla a arreglo de bytes mediante el método getBytes.

Por omisión el método getBytes utiliza la codificación default (UTF-8), para usar otra codificación se puede pasar como parámetro el nombre de la codificación como string, por ejemplo “UTF-8“. Ver más adelante el tema: Codificación de texto.

Ahora supongamos que el servidor envía al cliente una cadena de caracteres.

Para que el cliente reciba la cadena de caracteres es necesario que conozca el número de bytes que envía el servidor, en este caso el servidor envía una cadena de caracteres de 4 bytes.

Para recibir los bytes se utiliza el método read de la clase `DataInputStream`. El método read tiene tres parámetros, el primer parámetro es un arreglo de bytes con una longitud suficiente para contener los bytes a recibir.

El segundo parámetro indica la posición, dentro del arreglo de bytes, donde se pondrán los bytes a recibir, y el tercer parámetro indica el número de bytes a recibir.

El siguiente código crea un arreglo de 4 bytes, invoca el método read de la clase DataInputStream, crea una instancia de la clase String utilizando los bytes recibidos.

Debido a que la variable buffer contiene los bytes correspondientes a la cadena de caracteres que envió el servidor, para obtener la cadena de caracteres utilizamos el constructor de la clase String para crear una cadena de caracteres a partir del arreglo de bytes indicando la codificación, en este caso UTF-8.

```
byte[] buffer = new byte[4];
entrada.read(buffer,0,4);
System.out.println(new String(buffer,``UTF-8``));
```

Sin embargo, el método read podría obtener solo una parte del mensaje enviado. Esta situación la explicaremos más adelante.

2.1. La clase ByteBuffer

Ahora veremos cómo enviar de manera eficiente un conjunto de números punto flotante de 64 bits.

Supongamos que vamos a enviar cinco números punto flotante de 64 bits.

Primero “empacaremos“ los números utilizando un objeto ByteBuffer.

Cinco números punto flotante de 64 bits ocupan 5x8 bytes (64 bits=8 bytes). Entonces vamos a crear un objeto de tipo ByteBuffer con una capacidad de 40 bytes:

```
ByteBuffer b = ByteBuffer.allocate(5*8);
```

Utilizamos el método putDouble para agregar cinco números al objeto ByteBuffer:

```
b.putDouble(1.1);
b.putDouble(1.2);
b.putDouble(1.3);
b.putDouble(1.4);
b.putDouble(1.5);
```

Para enviar el "paquete" de números, convertimos el objeto `ByteBuffer` a un arreglo de bytes utilizando el método array de la clase `ByteBuffer`:

```
byte[] a = b.array();
```

Entonces enviamos el arreglo de bytes utilizando el método write:

```
salida.write(a);
```

Para terminar el programa cerramos la conexión con el servidor (al cerrar el socket se cierran también los streams asociados), en este caso vamos a poner un retardo de un segundo, antes de cerrar la conexión, para permitir que el servidor tenga tiempo de recibir los datos:

```
Thread.sleep(1000);
conexion.close();
```

Capítulo 3

Codificación de texto

Cuando estudiamos el tema de archivos en la clase de Sistemas Operativos, vemos que hay dos tipos de archivos: archivos binarios y archivos con formato texto.

3.1. Archivos binarios

Los bytes en un archivo binario pueden tomar cualquier valor entre 0 y 255.

Ejemplos de archivos binarios son: imágenes (fotografías), videos, archivos de Office (Word, Excel, etc.), archivos de Libre Office, programas ejecutables, etc.

3.2. Archivos con formato texto

Los archivos con formato texto son aquellos producidos por editores de texto como el Bloc de notas, Visual Studio Code, vi, nano, entre otros.

Debido a que las computadoras solo procesan y almacenan números (no texto), para procesar y guardar un archivo con formato texto es necesario traducir cada carácter a un número. Para ello, se utiliza una tabla de codificación.

3.3. Código ASCII

El código ASCII fue una de las primeras codificaciones utilizadas. En ASCII, cada carácter se traduce a un número entre 0 y 127, lo que permite representar 128 caracteres.

Esta codificación fue diseñada para computadoras de 7 bits y permite representar caracteres del inglés, pero no incluye acentos, ñ, o caracteres de otros idiomas.

3.4. Código ANSI

Debido a las limitantes del código ASCII, surgió la necesidad de representar más caracteres, especialmente para idiomas con caracteres latinos (como el español).

El código ANSI extendió el código ASCII utilizando 8 bits, lo que permitió representar hasta 256 caracteres.

Sin embargo, esta codificación no incluía caracteres de idiomas como el chino, japonés, árabe, ruso, griego, entre otros.

3.5. Unicode

Para resolver las limitaciones de ASCII y ANSI, se desarrolló Unicode, un estándar que asigna un número único a cada carácter de todos los idiomas del mundo.

Unicode utiliza 16 bits, lo que permite representar hasta 65,536 caracteres. Esto incluye caracteres de prácticamente todos los idiomas, símbolos matemáticos, emojis, y más.

Sin embargo, una desventaja de Unicode es que duplica el tamaño de los archivos de texto en comparación con ANSI, ya que utiliza dos bytes para cada carácter en lugar de uno.

3.6. UTF-8

Para optimizar el uso de espacio, se creó UTF-8, una codificación de longitud variable que utiliza 1, 2, 3 o 4 bytes para codificar cada carácter.

UTF-8 es compatible con ASCII y ANSI, ya que los caracteres ASCII (0-127) se representan con un solo byte, igual que en ANSI.

Los caracteres adicionales utilizan 2, 3 o 4 bytes según sea necesario. Esto permite que UTF-8 sea eficiente en términos de espacio y compatible con muchos idiomas.

Actualmente, UTF-8 es la codificación más utilizada en las aplicaciones web, lenguajes de programación y sistemas operativos.

3.7. Archivos texto en Windows y en Unix (Linux)

Los archivos con formato texto en Windows, terminan cada línea con los caracteres carriage return (código ASCII 13) y line feed (código ASCII 10). Para representar estos caracteres en un programa se utilizan las secuencias de escape "\r\n".

Los archivos con formato texto en Unix o Linux, terminan cada línea con el carácter line feed (código ASCII 10). Para representar este carácter en un programa se utiliza la secuencia de escape "\n".

Para convertir un archivo texto creado en Unix o Linux para ser utilizado en Windows, podemos utilizar el siguiente comando en Unix (Linux):

```
unix2dos nombre-del-archivo
```

Para convertir un archivo texto creado en Windows para ser utilizado en Unix o Linux, podemos utilizar el siguiente comando en Unix (Linux):

```
dos2unix nombre-del-archivo
```

3.8. Programas en Java

Los programas en Java pueden ser escritos en diferentes codificaciones, lo que permite que los nombres de variables, funciones y cadenas de caracteres puedan escribirse en cualquier idioma.

Por ejemplo, el siguiente programa calcula el factorial, los nombres de las variables y funciones están en chino. El código fuente se deberá guardar con codificación UTF-8 y se deberá compilar con la opción `-encoding UTF-8`: `javac -encoding UTF-8 Factorial.java`

```
public class Factorial
{
    // 方法: 计算阶乘 (factorial)
    public static long 阶乘(int 数字)
    {
        if (数字 <= 1)
            return 1;
        else
            return 数字 * 阶乘(数字 - 1);
    }
    public static void main(String[] 参数)
    {
        int 数字 = 5;
        long 结果 = 阶乘(数字);
        System.out.println(``数字 `` + 数字 + `` 的阶乘是: `` + 结果);
    }
}
```

3.9. Editores de texto

Cuando se crea un archivo de texto con un editor como el Bloc de notas, Visual Studio Code, o cualquier otro, se debe seleccionar la codificación que se utilizará para guardar el archivo.

3.10. Archivos CSV

Cuando se exporta una tabla de Excel a un archivo CSV (valores separados por comas), se debe seleccionar la codificación que se utilizará para guardar el archivo.

De igual manera, al abrir un archivo CSV, es necesario especificar la codificación que se utilizó para guardar el archivo, de modo que el texto se muestre adecuadamente.

Es importante destacar que “formato texto“ no es lo mismo que “extensión .txt“.

Archivos con extensiones como .java, .html, .py, entre otros, también son archivos con formato texto, aunque no tengan la extensión .txt

Capítulo 4

Criptografía y sockets seguros

En esta clase veremos cómo implementar un cliente y un servidor mediante sockets seguros.

Primeramente, vamos a explicar los conceptos básicos de PKI (Public Key Infrastructure).

4.1. Encriptado y desencriptado simétrico

La criptografía simétrica es un conjunto de algoritmos que permiten encriptar y desencriptar utilizando la misma clave, conocida como clave secreta o clave privada.

Ejemplos de algoritmos simétricos:

- AES-128
- AES-256
- RC4
- RC5
- DES
- 3DES

Los algoritmos simétricos son muy rápidos, sin embargo, resulta complicado intercambiar las claves.

Por ejemplo, supongamos que un amigo se va a estudiar a otro país y en un momento dado te pide le envíes un documento electrónico utilizando email. Desde luego habría que encriptar el documento para evitar que alguna otra persona pudiera hacer mal uso de él.

El problema es que ambos deberían tener la clave para encriptar y desencriptar. A este problema se le conoce como problema de distribución de clave (key distribution problem).

La única solución es que ambos hayan compartido la clave para encriptar y desencriptar antes del viaje, ya que tampoco es seguro enviar la clave por email.

4.2. Encriptado y desencriptado asimétrico

Una mejor solución al problema de distribución de claves es el uso de la criptografía asimétrica.

En la criptografía asimétrica el destinatario del mensaje tiene dos claves, una llamada clave privada y otra llamada clave pública.

La clave privada es una clave que mantiene en secreto el destinatario de los datos, mientras que la clave pública es una clave que puede conocer cualquiera.

En la criptografía asimétrica (también llamada criptografía de llave pública o PKI: Public Key Infrastructure), el originario utiliza la clave pública del destinatario para encriptar los datos. Entonces el destinatario utiliza su clave privada para desencriptar los datos encriptados y obtener los datos en claro.

4.3. Firma digital

El par de claves pública y privada pueden utilizarse para autenticar un documento electrónico. A esta autenticación se le llama firma digital.

El efecto de una firma digital es el mismo que tiene una firma autógrafa, esto es, el no repudio de un documento: la posibilidad de probar que el documento fue, en efecto, firmado por una persona (la persona no puede repudiar el documento).

Supongamos que vamos a enviar un documento (datos) a un destinatario. Para garantizar que el documento procede de un origen determinado, el originario genera un hash de los datos y encripta el hash con su clave privada (la cual solo es conocida por el originario). Al hash encriptado le llamamos la firma digital del documento.

Entonces el originario envía al destinatario el documento y la firma digital respectiva.

Para verificar la firma, el destinatario desencripta la firma digital utilizando la clave pública del originario, entonces obtiene el hash del documento. Así mismo, el destinatario genera el hash del documento recibido.

Si los dos hashes son iguales, entonces podemos estar seguros que el documento recibido procede del propietario de la clave pública y que el documento no ha sido modificado, debido a que cualquier modificación en el documento cambiaría el hash.

4.4. Certificado digital

Ahora bien, ¿Cómo podemos garantizar que una clave privada pertenece a una persona u organismo o dominio de Internet? (es decir, que en efecto la persona firmó un documento).

Considerando que la clave privada es uno a uno con la clave pública, ¿Cómo sabemos que la clave pública pertenece a la persona u organismo o dominio de Internet?

Si el originario nos envió su clave pública por email, y sabemos con certeza que la dirección de correo electrónico pertenece a esta persona, entonces concluimos que la clave pública pertenece a esta persona, de tal manera que podemos verificar la firma digital de cualquier documento que nos envíe.

Sin embargo, no siempre podemos conocer a ciencia cierta que una dirección de correo electrónico pertenece a una persona determinada.

Para resolver el problema de la identidad de una clave pública (y con ello la identidad de la clave privada), se utiliza un documento electrónico llamado certificado digital.

Un certificado digital es un documento electrónico que contiene, entre otros datos:

- La identidad —nombre— de una persona, organización o dominio de Internet.
- Las fechas de validez del certificado.

- La clave pública de la persona, organización o dominio de Internet.
- La firma digital de los datos que contiene el certificado.

La clave pública que contiene el certificado está asociada a una clave privada que solo conoce la persona u organización o dominio de Internet propietario del certificado digital.

Para mayor seguridad, la clave privada generalmente se encripta con una clave simétrica.

El certificado digital es firmado —avalado— por una autoridad certificadora (CA).

Cuando se adquiere un certificado firmado por una CA, además del certificado se obtiene un archivo conocido como *bundle*. Los certificados incluidos en el *bundle* se llaman certificados intermedios. El archivo contiene los certificados de CA que forman una ruta de certificación desde el certificado emitido hasta un certificado raíz preinstalado en la computadora.

El sistema operativo de nuestra computadora cuenta con un repositorio de certificados de confianza; en este repositorio se instalan los certificados raíz (autofirmados) de las autoridades certificadoras en las que confiamos.

Por defecto, el sistema operativo incluye en el repositorio de certificados de confianza los certificados de las autoridades certificadoras reconocidas internacionalmente. Estos certificados se conocen como certificados raíz (*root*).

Nuestra computadora reconoce como válido un certificado (por ejemplo, el certificado de un dominio al que se conecta el navegador), si hay una ruta de certificación entre el certificado y un certificado en el repositorio de certificados de confianza.

El estándar más utilizado para certificados digitales es el X.509 (el SAT utiliza el mismo estándar para la firma digital).

Existen dos tipos de certificados:

- Certificados autofirmados.
- Certificados firmados por una CA.

4.5. Certificado autofirmado

Un certificado autofirmado es aquel que se firma con la misma clave privada correspondiente a la clave pública que contiene el certificado.

Un certificado autofirmado se puede crear mediante el comando `keytool` del JDK.

Al crear un certificado autofirmado, se crea un par de claves pública y privada, la clave pública se incluye en el certificado y este se firma utilizando la clave privada.

Los datos de identidad en el certificado autofirmado los captura el usuario al crear el certificado, por tanto, un certificado autofirmado no tiene validez como medio de identificación (identidad de la clave pública), no obstante, puede utilizarse para implementar comunicación segura.

4.6. Certificado firmado por una autoridad certificadora (CA)

Un certificado firmado por una autoridad certificadora (CA) es un certificado el cual ha sido firmado por un organismo que actúa como autoridad, dicha autoridad garantiza que el certificado pertenece a una determinada persona u organización.

Un certificado de CA es similar a una credencial, la cual contiene el nombre, la fotografía y demás información de una persona; para garantizar que los datos de la credencial corresponden a la persona, la credencial deberá ser firmada por una autoridad.

Los certificados de autoridad certificadora se pueden adquirir con un proveedor de certificados digitales, por ejemplo, cheapsslsecurity.com.

Existen dos tipos de certificados firmados por una CA:

- Certificados que verifican el dominio.
- Certificados que verifican el dominio y, adicionalmente, la empresa.

Para poder tener un certificado con verificación de dominio, es necesario ser propietario de un dominio.

Por otra parte, para poder tener un certificado con verificación de empresa, es necesario tener una empresa y un dominio.

Capítulo 5

Máquinas virtuales en Azure

5.1. Crear una máquina virtual con Ubuntu

Ingresar al portal de Azure.

1. Dar clic al botón “Iniciar sesión“.
2. En el portal de Azure seleccionar “Máquinas virtuales“.
3. Seleccionar la opción “+Crear“.
4. Seleccionar la opción “Máquina virtual en Azure“
5. Seleccionar el grupo de recursos o crear uno nuevo. Un grupo de recursos es similar a una carpeta donde se pueden colocar los diferentes recursos de nube que se crean en Azure.
6. Ingresar el nombre de la máquina virtual.
7. Seleccionar la región donde se creará la máquina virtual. Notar que el costo de la máquina virtual depende de la región. Es recomendable crear la máquina virtual en una región cercana de manera que la comunicación con la máquina virtual sea más rápida.

7.1 En el campo “Opciones de disponibilidad“ seleccionar “No se requiere redundancia de la infraestructura“.

7.2 En el campo “Tipo de seguridad“ seleccionar “Estándar“.

1. Seleccionar la imagen, en este caso vamos a seleccionar Ubuntu Server 24.

8.1 En el campo “Arquitectura de VM” seleccionar: `x64`.

1. En el campo “Tamaño“ dar clic en “Ver todos los tamaños“, en este caso vamos a seleccionar una máquina virtual con 1 GB de memoria RAM (Tamaño de VM B1s). Dar clic en el botón “Seleccionar“.
2. En el campo “Tipo de autenticación“ seleccionar “Contraseña“.
3. Ingresamos el nombre del usuario, por ejemplo: ubuntu
4. Ingresamos la contraseña y confirmamos la contraseña. La contraseña debe tener al menos 12 caracteres, debe al menos una letra minúscula, una letra mayúscula, un dígito y un carácter especial.
5. En las “Reglas de puerto de entrada“ se deberá dejar abierto el puerto 22 para utilizar SSH (la terminal de secure shell).

6. Dar clic en el botón “Siguiente: Discos>“
7. Seleccionar el tipo de disco de sistema operativo, en este caso vamos a seleccionar HDD estándar.
8. Dar clic en el botón “Siguiente: Redes>“

Por default se creará una red virtual y la máquina virtual se conectará a esta red virtual. También se creará una IP pública.

1. Dar clic en el botón “Siguiente: Administración>“
2. Dar clic en el botón “Siguiente: Supervisión>“
3. En el campo “Diagnóstico de arranque“ seleccionar “Deshabilitar“.
4. Dar clic en el botón “Revisar y crear“.
5. Dar clic en el botón “Crear“.
6. Dar clic a la campana de notificaciones (barra superior de la pantalla) para verificar que la maquina virtual se haya creado.
7. Dar clic en el botón “Ir al recurso“. En la página se puede ver la dirección IP pública de la máquina virtual. Esta dirección puede cambiar cada vez que se apague y se encienda la máquina virtual.
8. Para conectarnos a la máquina virtual vamos a utilizar el programa ssh disponible en Windows, Linux y MacOS.
9. En una ventana de comandos de Windows o una terminal de Linux o macOS, ejecutar `ssh` así:

```
ssh usuario@ip-máquina-virtual
```

Donde usuario es el usuario que ingresamos en el paso 11, ip-máquina-virtual es la ip pública de la máquina virtual.

1. Para enviar o recibir archivos de la máquina virtual, utilizar `sftp`, disponible en Windows, Linux y macOS:

```
sftp usuario@ip-máquina-virtual
```

Para enviar archivos se utiliza el comando put y para recibir archivos se utiliza el comando get.

Para mayor información sobre `sftp`, consultar How To Use SFTP to Securely Transfer Files.

5.2. Abrir un puerto de entrada

Para que los programas que ejecutan en la máquina virtual pueda recibir conexiones a través de un determinado puerto, es necesario crear una regla de entrada para el puerto.

Por ejemplo, vamos a abrir el puerto 50000 en la máquina virtual que acabamos de crear:

1. Entrar al portal de Azure.

2. Seleccionar “Máquinas virtuales“.
3. Seleccionar la máquina virtual.
4. Dar clic en “Configuración de red“.
5. Dar clic en el botón “+Crear ACL del puerto“.
6. Seleccionar “Regla de puerto de entrada“.
7. En “Intervalos de puertos de destino“, ingresar `50000`.
8. Seleccionar el protocolo `TCP`.
9. En “Nombre“, ingresar un nombre para la regla, por ejemplo: `Puerto_50000`.
10. Dar clic en el botón “Aceptar“.

5.3. Detener una máquina virtual

Cuando una máquina virtual no se utiliza es conveniente detenerla con el fin de reducir el costo. Para detener una máquina virtual:

1. Dar clic en la opción “Detener“ en el portal de Azure.
2. Cuando se pregunta ¿Desea detener ...? dar clic en el botón “Sí“.

Esperar a que el estado de la máquina virtual sea “Desasignada“.

5.4. Encender una máquina virtual

Para encender una máquina virtual

1. Seleccionar la opción “Iniciar“ en la página de la máquina virtual dentro del portal de Azure.

Esperar a que el estado de la máquina virtual sea “En ejecución“.

5.5. Eliminar una máquina virtual

Para eliminar una máquina virtual:

1. Seleccionar la opción “Eliminar“ en la página de la máquina virtual dentro del portal de Azure.
2. Seleccionar los recursos a eliminar: disco de SO, interfaces de red y direcciones IP públicas.
3. Aceptar la advertencia que dice que se eliminarán los recursos seleccionados.
4. Dar clic en el botón “Eliminar“.

5.6. Restablecer la contraseña de una máquina virtual

En caso de no recordar la contraseña de un usuario de una máquina virtual, se puede realizar el siguiente procedimiento:

1. Ejecutar el portal de Azure.
2. Seleccionar “Máquinas virtuales“. La máquina virtual debe estar encendida.
3. Seleccionar la máquina virtual que se desea modificar.
4. En el menú izquierdo, seleccionar “Restablecer contraseña“.
5. En el menú “Modo“, seleccionar “Restablecer contraseña“.
6. En “Nombre de usuario“, ingresar el usuario.
7. En “Contraseña“, ingresar la nueva contraseña.
8. En “Confirmar contraseña“, repetir la nueva contraseña.
9. Dar clic en el ícono “Actualizar“.

5.7. Crear una imagen de una máquina virtual con Ubuntu

Vamos a ver cómo crear la imagen de una máquina virtual con Ubuntu en Azure y cómo crear máquinas virtuales a partir de la imagen.

Una máquina virtual generalizada es una máquina virtual cuyo sistema operativo se ha despojado de la configuración específica y la configuración de usuarios.

Una imagen generalizada es la captura de un sistema operativo de una máquina virtual generalizada..

5.7.1. Notas importantes

1. La captura de la imagen de una máquina virtual inutiliza la máquina virtual ya que una máquina virtual generalizada no se puede iniciar o modificar.
2. La generalización de una máquina virtual no implica que se borre toda la información confidencial que pudiera existir en la máquina virtual. Es muy importante considerar lo anterior si se va a redistribuir la imagen de la máquina virtual.
3. La generalización de una máquina virtual no elimina el archivo /etc/resolv.conf (ver: resolvconf)
4. La generalización de una máquina virtual deshabilita la contraseña de root.
5. La opción +user del comando waagent elimina la última cuenta creada en la máquina virtual incluyendo el directorio del usuario. Si se desea conservar el usuario y el directorio, no se deberá utilizar la opción +user al generalizar la máquina virtual mediante el comando waagent.
6. Para generalizar una máquina virtual con Windows se utiliza el programa sysprep.exe, ver: https://docs.microsoft.com/en-us/azure/virtual-machines/generalize.
7. Una imagen se cobra de acuerdo al espacio en disco que ocupa, ver: Precios de Managed Disks.
8. El tamaño del disco predeterminado en la imagen es el tamaño del disco de sistema operativo de la máquina virtual. Al crear la máquina virtual a partir de la imagen, el tamaño del disco de sistema operativo podrá ser mayor al que tenía la máquina virtual original, pero no podrá ser de menor tamaño.

Para generalizar la máquina virtual utilizaremos el agente `waagent`, que elimina los datos específicos de la máquina virtual.

1. Crear una máquina virtual con Ubuntu.
2. Abrir una ventana cmd de Windows o una terminal de Linux o MacOs.
3. Ejecutar el programa ssh en la ventana, pasando como parámetros el usuario (por ejemplo ubuntu) y la ip pública de la máquina virtual:

```
ssh usuario@ip
```

1. Para generalizar la máquina virtual y eliminar la última cuenta de usuario creada incluyendo el directorio del usuario, ejecutar el comando:

```
sudo waagent -deprovision+user
```

Si se quiere conservar en la imagen la última cuenta de usuario creada, ejecutar el comando:

```
sudo waagent -deprovision
```

1. En el portal de Azure seleccionar la máquina virtual que se quiera capturar como imagen.
2. Seleccionar la opción “Captura“.

6.1 En la opción “Compartir imagen con Shared Image Gallery“ seleccionar “No, capturar solo una imagen administrada“.

1. Marcar la casilla “Eliminar automáticamente esta máquina virtual después de crear la imagen“, ya que una máquina virtual generalizada no se puede iniciar o modificar.
2. Ingresar el nombre de la imagen a crear.
3. Dar clic en el botón “Crear“.
4. Dar clic en la campana de notificaciones para verificar que se haya creado la imagen de la máquina virtual.

5.8. Crear una máquina virtual a partir de una imagen

1. En la sección “Todos los recursos“ en el portal de Azure seleccionar la imagen de la máquina virtual.
2. Seleccionar la opción “+Crear máquina virtual“.
3. Seleccionar el grupo de recursos donde se creará la máquina virtual.
4. Ingresar el nombre de la máquina virtual.
5. Seleccionar el tamaño de la máquina virtual.
6. Seleccionar el tipo de autenticación (Clave pública SSH o Contraseña). En su caso, ingresar el usuario y contraseña.
7. Dar clic en el botón “Siguiente: Discos >“
8. Seleccionar el tipo de disco del sistema operativo (p.e. HDD estándar). Notar que el tamaño predeterminado del disco es el tamaño del disco de sistema operativo de la máquina virtual

original. Se puede seleccionar un tamaño mayor pero no un tamaño menor al tamaño del disco original.

9. Si no hay otra configuración que se quiera realizar, dar clic en el botón “Revisar y crear“.
10. Dar clic en el botón “Crear“.

Capítulo 6

Preguntas

```
¿Qué método de la clase ByteBuffer me permite obtener una instancia de ByteBuffer a partir de un
↪  arreglo de bytes?
```

.wrap()

```
¿Qué método de la clase ByteBuffer me permite obtener un arreglo de bytes a partir de una instancia de
↪  ByteBuffer?
```

.array()

```
¿Qué es una barrera en hilos?
```

Es cuando un thread espera a uno o más threads.

```
¿Qué es la sección crítica de un programa?
```

Las intrucciones que ejecuta un thread.

```
¿Qué es starvation/inanición?
```

Es cuando un thread quiere entrar a un lock pero nunca puede lograrlo.

```
¿Cómo se instala ntp en ubuntu?
```

sudo apt install ntp

```
¿Cómo se sincronizan los segundos UTC con los segundos TAI?
```

Se adelanta el tiempo UTC para alcanzar el tiempo TAI, estos segundos que se adelantan se les conocen como segundos vacios

Capítulo 7

Proxy

Un proxy es un servidor cuya función es recibir el tráfico TCP de un cliente y reenviarlo a un servidor final, así mismo, recibir el tráfico TCP del servidor y reenviarlo al cliente.

Existen diferentes tipos de proxies, entre otros:

- **Proxy transparente:** redirige el tráfico TCP sin que el cliente se dé cuenta de que existe un proxy entre el cliente y el servidor. Reenvía el tráfico TCP sin considerar un protocolo específico sobre TCP, como HTTP, SMTP o FTP.
- **Proxy inverso:** recibe el tráfico de red de un cliente y lo distribuye entre diferentes servidores de acuerdo con una configuración específica, ocultando las direcciones IP de los servidores.

Un proxy inverso también puede:

- Almacenar contenido estático, como imágenes y archivos JS o CSS.
- Distribuir tráfico mediante reglas de balanceo de carga.
- Distribuir tráfico de acuerdo con la petición, como un API gateway.
- Centralizar la gestión de la comunicación SSL utilizando un solo certificado digital en el proxy.
- **Proxy HTTP/HTTPS:** permite monitorear y filtrar las peticiones que envía el cliente al servidor HTTP, así como las respuestas del servidor.

7.1. Arquitectura de un proxy transparente

Un proxy transparente consta de un servidor y un cliente.

Cuando el servidor del proxy recibe una conexión del cliente, el cliente del proxy se conecta al servidor.

El servidor del proxy recibe tráfico de red del cliente y lo envía al servidor, así mismo, el cliente del proxy recibe tráfico de red del servidor y lo envía al cliente.

7.2. Implementación de un proxy transparente en Java

Vamos a implementar un proxy transparente mediante un servidor TCP multithread.

Cuando el servidor del proxy recibe la conexión del cliente, crea un thread de tipo Worker_1, éste thread crea un segundo thread de tipo Worker_2.

El thread de tipo Worker_2 se conecta al servidor remoto y se encarga de reenviar el tráfico TCP del servidor remoto al cliente.

El thread de tipo Worker_1 se encarga de reenviar el tráfico TCP del cliente al servidor remoto.

Debido a que un proxy transparente no implementa un protocolo específico sobre TCP, es necesario que el reenvío del tráfico TCP del cliente al servidor se pueda realizar en paralelo al reenvío del tráfico TCP del servidor al cliente. Por esta razón, es necesario crear dos threads que reenvíen el tráfico TCP de manera independiente (full-duplex).

El código del proxy transparente es el siguiente:

```
/*
  Proxy.java
  Carlos Pineda G. 2024
*/
import java.io.InputStream;
import java.io.OutputStream;
import java.io.IOException;
import java.net.Socket;
import java.net.ServerSocket;
class Proxy
{
  static String host_remoto;
  static int puerto_remoto;
  static int puerto_local;
  static class Worker_1 extends Thread
  {
    Socket cliente_1,cliente_2;
    Worker_1(Socket cliente_1)
    {
      this.cliente_1 = cliente_1;
    }
    public void run()
    {
      try
      {
        // se conecta al host remoto
        cliente_2 = new Socket(host_remoto,puerto_remoto);
        // thread que dirige el tráfico del host remoto al cliente
        new Worker_2(cliente_1,cliente_2).start();
        InputStream entrada_1 = cliente_1.getInputStream();
        OutputStream salida_2 = cliente_2.getOutputStream();
        byte[] buffer = new byte[1024];
        int n;
        while((n = entrada_1.read(buffer)) != -1)
        {
          salida_2.write(buffer,0,n);
          salida_2.flush();
        }
      }
      catch (IOException e)
      {
        // captura silenciosamente las excepciones por desconexión
      }
      finally
```

```
    {
      try
      {
        if (cliente_1 != null) cliente_1.close();
        if (cliente_2 != null) cliente_2.close();
      }
      catch (IOException e2)
      {
        e2.printStackTrace();
      }
    }
  }
}
static class Worker_2 extends Thread
{
  Socket cliente_1,cliente_2;
  Worker_2(Socket cliente_1,Socket cliente_2)
  {
    this.cliente_1 = cliente_1;
    this.cliente_2 = cliente_2;
  }
  public void run()
  {
    try
    {
      InputStream entrada_2 = cliente_2.getInputStream();
      OutputStream salida_1 = cliente_1.getOutputStream();
      byte[] buffer = new byte[4096];
      int n;
      while((n = entrada_2.read(buffer)) != -1)
      {
        salida_1.write(buffer,0,n);
        salida_1.flush();
      }
    }
    catch (IOException e)
    {
      // captura silenciosamente las excepciones por desconexión
    }
    finally
    {
      try
      {
        if (cliente_1 != null) cliente_1.close();
        if (cliente_2 != null) cliente_2.close();
      }
      catch (IOException e2)
      {
        e2.printStackTrace();
      }
    }
  }
}
public static void main(String[] args) throws Exception
{
  if (args.length != 3)
  {
    System.err.println(``Uso:\njava Proxy <host-remoto> <puerto-remoto> <puerto-local>``);
    System.exit(1);
```

```
    }
    host_remoto = args[0];
    puerto_remoto = Integer.parseInt(args[1]);
    puerto_local = Integer.parseInt(args[2]);
    System.out.println(``host_remoto: `` + host_remoto + ``, puerto_remoto: `` + puerto_remoto + ``,
    ↪  puerto_local: `` + puerto_local);
    ServerSocket ss = new ServerSocket(puerto_local);
    for(;;)
    {
      // espera una conexión del cliente
      Socket cliente_1 = ss.accept();
      // thread que dirige el tráfico del cliente al host remoto
      new Worker_1(cliente_1).start();
    }
  }
}
```

Capítulo 8

Servidor.java

El programa Servidor.java va a esperar una conexión del cliente, entonces recibirá los datos que envía el cliente y a su vez, enviará datos al cliente.

Primeramente, vamos a crear un socket servidor que va a abrir, en este caso, el puerto 50000:

```
ServerSocket servidor = new ServerSocket(50000);
```

Notar que en Windows, por razones de seguridad el firewall solicita al usuario administrador permiso para abrir este puerto.

Ahora invocamos el método accept de la clase ServerSocket.

El método accept es bloqueante, lo que significa que el thread principal del programa quedará en estado de espera pasiva (una espera que no ocupa ciclos de CPU) hasta recibir una conexión del cliente. Cuando se recibe la conexión el método accept regresa un socket, en este caso vamos a declarar una variable de tipo Socket llamada “conexion“:

```
Socket conexion = servidor.accept();
```

Una vez establecida la conexión con el cliente, el servidor podrá enviar y recibir datos.

Creamos un stream de salida y un stream de entrada:

```
DataOutputStream salida = new DataOutputStream(conexion.getOutputStream());
DataInputStream entrada = new DataInputStream(conexion.getInputStream());
```

Recordemos que el cliente envía un entero de 32 bits, entonces el servidor deberá recibir este dato utilizando el método readInt:

```
int n = entrada.readInt();
System.out.println(n);
```

Ahora el servidor recibe un número punto flotante de 64 bits utilizando el método readDouble:

```
double x = entrada.readDouble();
System.out.println(x);
```

El servidor recibe una cadena de cuatro caracteres:

```
byte[] buffer = new byte[4];
read(entrada,buffer,0,4);
System.out.println(new String(buffer,``UTF-8``));
```

El servidor envía una cadena de cuatro caracteres:

```
salida.write(``HOLA``.getBytes());
```

Ahora vamos a recibir los cinco números punto flotante empacados en un arreglo de bytes. Recordemos que los cinco números punto flotante de 64 bits ocupan 40 bytes (5x8bytes).

```
byte[] a = new byte[5*8];
read(entrada,a,0,5*8);
```

Una vez recibido el arreglo de bytes, lo convertimos a un objeto `ByteBuffer` utilizando el método wrap de la clase `ByteBuffer`:

```
ByteBuffer b = ByteBuffer.wrap(a);
```

Para extraer los números punto flotante, utilizamos el método getDouble de la clase ByteBuffer:

```
for (int i = 0; i < 5; i++) System.out.println(b.getDouble());
```

Finalmente, cerramos la conexión con el cliente:

```
conexion.close();
```

8.1. Un cliente con reintentos de conexión

Como vimos anteriormente, para que el cliente se conecte al servidor es necesario que el servidor inicie su ejecución antes que el cliente, sin embargo, para algunas aplicaciones el cliente debe esperar a que el servidor inicie su ejecución.

Veamos ahora cómo implementar el reintento de conexión cuando el servidor no está ejecutando.

```
Socket conexion = null;
for(;;)
  try
  {
    conexion = new Socket(``localhost``,50000);
    break;
  }
  catch (Exception e)
  {
    Thread.sleep(100);
  }
```

Como podemos ver, cada vez que el cliente falla en establecer la conexión con el servidor, espera 100 milisegundos y vuelve a intentar la conexión. Cuando el cliente logra conectarse con el servidor entonces sale del ciclo for.

Capítulo 9

Servidor HTTP

9.1. ¿Qué es un servidor HTTP?

La clase de hoy vamos a construir un servidor HTTP utilizando como base el servidor multi-thread que vimos anteriormente.

Un servidor HTTP es un servidor que recibe peticiones HTTP a través de una conexión TCP.

En general, los servidores HTTP son multi-thread, de tal manera que el servidor creará un thread cada vez que un cliente se conecta, de esta forma cada thread se encarga de recibir la petición y enviar la respuesta al cliente.

Por default los servidores HTTP reciben conexiones a través del puerto 80, sin embargo un servidor HTTP puede utilizar otro puerto, por ejemplo el puerto 8080.

El protocolo HTTP define las operaciones que el servidor HTTP puede realizar cuando recibe una petición, a estas operaciones se les conoce como "métodos".

Los métodos HTTP más utilizados son: GET, POST, PUT, DELETE y HEAD.

El método GET es utilizado por los navegadores (browsers) para solicitar un recurso o contenido al servidor HTTP.

Si el recurso existe, el servidor regresa al navegador como mínimo lo siguiente: el código 200 (OK), el tipo del recurso, la longitud del recurso y el contenido.

Si el recurso no existe, el servidor regresa al navegador el código 404 (Not Found).

El método HEAD es idéntico al método GET, solo que el método HEAD no regresa el recurso.

Más adelante veremos cómo utilizar los métodos GET, POST, PUT y DELETE para implementar servicios web de tipo REST.

9.2. `ServidorHTTP.java`

Ahora vamos a implementar un servidor HTTP; tomaremos como base el código del servidor multithread que vimos la clase anterior.

El servidor HTTP escuchará el puerto 80 (el puerto default para HTTP).

El servidor HTTP va a responder dos end-points:

- `/suma?a=<valor>&b=<valor>&c=<valor>` va a regresar la suma de los

tres valores.

- / va a regresar una página HTML incluyendo un botón; cuando se presione el botón se hará la siguiente petición: `/suma?a=1&b=2&c=3`.

Tal como lo hace el servidor multithread que vimos la clase anterior, cada vez que un cliente (navegador) se conecte al servidor HTTP, se creará un thread el cual recibirá como parámetro el socket cliente.

Primeramente, escribimos las siguientes instrucciones en el método run():

```
BufferedReader entrada = new BufferedReader(new InputStreamReader(conexion.getInputStream()));
PrintWriter salida = new PrintWriter(conexion.getOutputStream());
```

En este caso no vamos a utilizar las clases DataInputStream y DataOutpuStream debido a que necesitamos los métodos readLine() y println() los cuales están disponibles en las clases BufferedReader y PrintWriter, respectivamente.

Notar que creamos la instancia de BufferedReader a partir del InputStream disponible en el socket, así mismo, creamos la instancia de PrintWriter a partir del OutputStream disponible en el socket.

Ahora vamos a recibir la petición mediante el método readLine().

Cada línea en el protocolo HTTP termina con un carriage return (código ASCII 13) y line feed (código ASCII 10), por tanto, para leer la petición utilizamos el método readline().

```
String req = entrada.readLine();
System.out.println(``Petición: `` + req);
```

El método readLine() reconoce cualquier terminador de línea (\r, \n o \r\n) y lo descarta; no obstante, la petición enviada por el cliente termina con \r\n, la string “req“ no contiene \r ni \n.

Además de la petición, el navegador envía encabezados; cada encabezado termina con carriage return y line feed).

Después de los encabezados, el navegador envía una línea en blanco (carriage return y line feed).

Podemos leer los encabezados mediante el siguiente código (notar que no estamos procesando los encabezados, solo los desplegamos):

```
for (;;)
{
  String encabezado= entrada.readLine();
  System.out.println(``Encabezado: `` + encabezado);
  if (encabezado.equals(````)) break;
}
```

Para extraer los valores que pasan como parámetros a la función “suma“, implementaremos la función valor().

Esta función obtiene el valor de cada parámetro en la URL; recibe como parámetros una string con los parámetros incluidos en la URL, y el nombre de uno de los parámetros (variable), entonces regresa el valor del parámetro.

```
int valor(String parametros, String variable) throws Exception
{
  String[] p = parametros.split(``&``);
  for (int i = 0; i < p.length; i++)
  {
    String[] s = p[i].split(``=``);
    if (s[0].equals(variable))
      return Integer.parseInt(s[1]);
  }
  throw new Exception(``Se espera la variable: `` + variable);
}
```

Para que el servidor HTTP procese la petición, vamos a utilizar el método startsWith() de la clase String.

```
if (req.startsWith(``GET / ``))
{
  String respuesta =
    ``<html>`` +
    ``<script>`` +
    ``function get(req,callback)`` +
    ``{`` +
    ``const xhr = new XMLHttpRequest();`` +
    ``xhr.open('GET', req, true);`` +
    ``xhr.onload=function()`` +
    ``{`` +
    ``if (callback != null) callback(xhr.status,xhr.response);`` +
    ``};`` +
    ``xhr.send();`` +
    ``}`` +
    ``</script>`` +
    ``<body>`` +
    ``<button onclick=\``get('/suma?a=1&b=2&c=3',function(status,response){alert('estatus:' + status +
    ↪  ' ' + 'respuesta: ' + response);})\``>Aceptar</button>`` +
    ``</body>`` +
    ``</html>``;
  salida.println(``HTTP/1.1 200 OK``);
  salida.println(``Content-type: text/html; charset=utf-8``);
  salida.println(``Content-length: `` + respuesta.length());
  salida.println(``Connection: close``);
  salida.println();
  salida.println(respuesta);
  salida.flush();
}
```

```
else
if (req.startsWith(``GET /suma?``))
{
  String parametros = req.split(`` ``)[1].split(``\\?``)[1];
```

```
  String respuesta = String.valueOf(valor(parametros,``a``) + valor(parametros,``b``) +
                                                          valor(parametros,``c``));
  salida.println(``HTTP/1.1 200 OK``);
  salida.println(``Access-Control-Allow-Origin: *``);
  salida.println(``Content-type: text/plain; charset=utf-8``);
  salida.println(``Content-length: ``+respuesta.length());
  salida.println(``Connection: close``);
  salida.println();
  salida.println(respuesta);
  salida.flush();
}
```

Notar que en la respuesta que envía el servidor HTTP al navegador, es necesario separar los encabezados del contenido, mediante una línea en blanco; en este caso el contenido es de tipo texto.

El servidor HTTP envía al navegador el código 200 indicando que no hubo error, además envía el tipo mime del contenido utilizando el encabezado Content-Type y la longitud del contenido mediante el encabezado Content-Length.

El navegador debe conocer el tipo mime del contenido para poder interpretarlo correctamente, por ejemplo, si el recurso es una página web, el tipo mime es "text/html", entonces el navegador interpretará el contenido como un archivo HTML (desplegará botones, campos de captura, etc.).

Si el tipo mime es text/plain, el navegador desplegará el texto en la pantalla.

La forma general del encabezado Content-Type es la siguiente:

```
Content-Type: tipo/subtipo
```

Donde tipo es el tipo del contenido (por ejemplo, text) y subtipo es una especificación del tipo (por ejemplo: html).

A la cadena tipo/subtipo se le llama tipo mime (Multipurpose Internet Mail Extensions).

Los servidores pueden obtener el tipo mime de un archivo a partir de la extensión del archivo. Ver: Lista completa de tipos MIME

Por default, el protocolo HTTP/1.1 deja abierta la conexión entre el cliente y el servidor, de manera que el cliente (navegador) pueda enviar varias peticiones utilizando la misma conexión.

En este caso, el servidor envió al navegador el encabezado "Connection: close", indicando que la conexión se cerrará después de enviar la respuesta.

En código anterior, el servidor HTTP regresa al navegador un texto plano: la suma de los valores que pasan como parámetros en la URL.

Más adelante se explicará el encabezado "Access-Control-Allow-Origin", el cual habilita CORS (Cross-Origin Resource Sharing).

9.3. Respuesta 404 File Not Found

Si la petición enviada por el navegador no inicia con "/" o "GET /suma¿' entonces el servidor regresará al navegador el código de error 404 indicando que no se encontró el recurso:

```
else
{
  String respuesta = ``<html><body><h1>404 File Not Found</h1></body></html>``;
  salida.println(``HTTP/1.1 404 Not Found``);
  salida.println(``Content-Type: text/html; charset=utf-8``);
  salida.println(``Content-Length: `` + respuesta.length());
  salida.println(``Connection: close``);
  salida.println();
  salida.println(respuesta);
  salida.flush();
}
```

Si queremos que el servidor envíe al navegador otro mensaje de error, podemos enviar un código entre 400 y 499, agregando un mensaje específico. Ver: Códigos de estado de respuesta HTTP.

9.4. CORS — Cross-Origin Resource Sharing

Supongamos que tenemos un servidor HTTP que actúa a la vez como servidor web y como servidor de aplicaciones (p.e. ServidorHTTP.java, Tomcat, Weblogic, GlassFish, IIS, etc.).

El servidor HTTP va a alojar la página web p.html. Esta página web invoca (utilizando Javascript), el método web suma(int a,int b,int c)

Cuando el navegador solicita la página web p.html al servidor, el servidor le envía la página.

Supongamos que el usuario selecciona una opción en la página web, la cual invoca el método "suma".

Entonces el servidor HTTP envía la respuesta al navegador, en este caso, el número 6 (la suma 1+2+3).

¿Qué pasa si la página web está alojada en un servidor que ejecuta en una computadora diferente a la computadora donde ejecuta el método web?

Para que el navegador no bloquee la respuesta del método web, la respuesta debe incluir uno o varios encabezados "Access-Control-Allow-Origin", cada encabezado deberá incluir el dominio desde el cual se permite el acceso al método web.

Un asterisco * indica que cualquier origen puede acceder el método web.

9.5. El encabezado `Last-Modified`

Un servidor HTTP puede regresar al navegador una variedad de contenidos, por ejemplo, una página web, una imagen, un archivo, etc.

En general, estos recursos no se modifican con frecuencia en el servidor, por esta razón es conveniente que el navegador guarde una copia del recurso y solo reciba una actualización del recurso cuando éste ha sido modificado en el servidor.

Los navegadores almacenan copias de los recursos en una cache.

Almacenar los recursos en la cache tiene muchas ventajas, entre otras: mejora la velocidad de carga, reduce el ancho de banda utilizado, ahorra energía en el cliente (esto es especialmente útil en dispositivos móviles), mejora la experiencia del usuario y reduce la carga en el servidor.

Para que el navegador guarde un recurso en la cache, el servidor envía al navegador la respuesta "HTTP/1.1 200 OK", el encabezado "Last-Modified" y el recurso solicitado. El encabezado, "Last-Modified" le dice al navegador la fecha (en formato RFC1123) de la última modificación del recurso.

Por ejemplo, la respuesta:

```
HTTP/1.1 200 OK
Last-Modified:  Fri, 25 Aug 2023 21:00:00 GMT
```

Le dice al navegador que guarde el recurso en la cache indicando así mismo la fecha de la última modificación, en este caso el viernes 25 de agosto de 2023 a las 21 horas tiempo GMT.

Cada vez que el navegador solicita el recurso al servidor, el navegador agregará el siguiente encabezado a la petición:

```
If-Modified-Since: Fri, 25 Aug 2023 21:00:00 GMT
```

Esto le dice al servidor que el navegador tiene una copia del recurso solicitado, indicando la fecha de última modificación (la cual le envío el servidor previamente utilizando el encabezado "Last-Modified").

Entonces el servidor deberá comparar la fecha enviada por el navegador y la fecha de la última modificación del recurso en el servidor, si la primera fecha es menor a la segunda fecha, entonces el servidor deberá enviar al navegador la respuesta "HTTP/1.1 200 OK", el encabezado "Last-Modified" incluyendo la nueva fecha de modificación y el recurso solicitado. De otra forma, el servidor solo enviará al navegador la siguiente respuesta:

```
HTTP/1.1 304 Not Modified
```

Como podemos ver, el navegador recibe la actualización del recurso solo si el recurso se modificó después de que el navegador lo recibió por última vez.

¿Es conveniente guardar cualquier respuesta en la cache?

No siempre, ya que en algunos casos las respuestas podrían ser distintas en cada petición, por ejemplo, la respuesta de un servicio web.

En tal caso, el servidor no regresará al navegador el encabezado "Last-Modified", por tanto, el navegador no guardará en la cache la respuesta y tampoco enviará el encabezado "If-Modified-Since" en cada petición ulterior.

9.6. El encabezado `Cache-Control`

El encabezado Cache-Control es enviado por el servidor para controlar la cache del navegador. Las opciones más comunes del encabezado Cache-Control son las siguientes:

- `public`: el contenido puede almacenarse en forma pública; cualquier usuario puede tener acceso al contenido.

- `private`: el contenido debe almacenarse en forma privada en el navegador del usuario; solo el usuario tiene acceso al contenido.
- `no-store`: el contenido no debe almacenarse.
- `no-cache`: el contenido puede almacenarse en la cache, pero el navegador validará si fue modificado.
- `max-age=tiempo`: indica el tiempo en segundos que se guardará el contenido en la cache.
- `immutable`: indica que el contenido no será modificado por el servidor.

Es posible incluir varias opciones separadas por comas, considerando que las opciones más restrictivas pueden anular a las más permisivas.

9.7. El encabezado `Content-Disposition`

El encabezado Content-Disposition es enviado por el servidor, para indicar al navegador que guarde en el disco local el contenido que regresa una petición GET, en lugar de mostrarlo en la pantalla.

Por ejemplo, el siguiente encabezado indica al navegador que guarde el archivo descargado con el nombre “documento.pdf“:

```
Content-Disposition: attachment; filename=``documento.pdf``
```

9.8. Los encabezados `Server` y `Date`

Opcionalmente el servidor HTTP podría enviar al navegador los siguientes encabezados, indicando el nombre del servidor y la fecha:

```
salida.println(``Server: ServidorHTTP.java``);
salida.println(``Date: `` + new Date());
```

9.9. ¿Cómo obtener la IP del cliente?

Para desplegar la dirección IP del cliente que se conectó al servidor HTTP, podemos incluir la siguiente instrucción al principio del método run():

```
System.out.println(conexion.getRemoteSocketAddress().toString());
```

En general, los servidores guardan en una bitácora la fecha y hora, la IP del cliente y la petición GET de manera que se pueda monitorear cualquier ataque que reciba el servidor HTTP.

Capítulo 10

Servidor multithread

Anteriormente vimos el programa Servidor.java el cual invoca el método accept para esperar una conexión del cliente, debido a que este método es bloqueante el programa queda en espera pasiva hasta que el cliente se conecta.

Cuando el servidor recibe una conexión, el método accept regresa un socket. Entonces el cliente y el servidor podrán intercambiar datos. Generalmente el servidor procesa los datos que recibe del cliente y al terminar vuelve a invocar el método accept para esperar otra conexión.

Sin embargo, mientras el servidor procesa los datos que recibe del cliente, no puede recibir otra conexión. Para resolver este problema los servidores se construyen utilizando threads.

Entonces, vamos a ver cómo construir un servidor multithread.

10.1. Ejecución paralela y ejecución concurrente

En las clases de Sistemas Operativos se explica que un thread (hilo) es la ejecución secuencial de las instrucciones de un programa.

Un proceso puede crear uno o más threads (hilos de ejecución), los cuales van a ejecutar simultáneamente.

Si la computadora tiene un CPU dual core, entonces el CPU podrá ejecutar en paralelo (al mismo tiempo) dos threads, si el CPU es quad core entonces podrá ejecutar en paralelo cuatro threads, y así sucesivamente.

Por otra parte, si un programa crea un número de threads mayor al número de procesadores físicos (cores) disponibles en la computadora, entonces los threads ejecutarán en forma concurrente (por turnos).

10.2. Programación multithread en Java

Supongamos que tenemos una clase llamada P.

Dentro de la clase P definimos una clase interior (nested class) llamada Worker la cual es subclase de la clase Thread:

```
class P
{
```

```
  static class Worker extends Thread
  {
    public void run()
    {
    }
  }
  public static void main(String[] args) throws Exception
  {
  }
}
```

Podemos ver que hemos incluido en la clase Worker un método público llamado run el cual no tiene parámetros ni regresa resultado.

10.3. El método start

Para iniciar la ejecución de un thread, debemos crear una instancia de la clase Worker e invocar el método start (este método se hereda de la clase Thread):

```
Worker w = new Worker();
w.start();
```

Entonces se crea un hilo que inicia invocando el método run que hemos definido en la clase Worker.

Un thread finaliza su ejecución cuando el método run termina. Cuando un thread finaliza, no puede volver a ejecutarse.

10.4. El método join

Supongamos que el thread principal (el thread que invocó el método start) requiere esperar que el thread w termine su ejecución, entonces el thread principal deberá invocar el método join:

```
Worker w = new Worker();
w.start();
w.join();
```

El método join queda en un estado de espera pasiva mientras el thread “w“ se encuentra ejecutando, cuando el thread “w“ termina, el método join regresa, entonces el thread principal continua su ejecución.

Ahora supongamos que el thread principal requiere crear dos threads y esperar a que terminen su ejecución. Entonces creamos dos instancias de la clase Worker e invocamos los métodos start y join para cada thread:

```
Worker w1 = new Worker();
```

```
Worker w2 = new Worker();
w1.start();
w2.start();
w1.join();
w2.join();
```

Cuando un thread (en este caso el thread principal) espera la terminación de uno o más threads para continuar su ejecución, se dice que se implementa una barrera. En este caso estamos implementando una barrera mediante dos métodos join.

10.5. Servidor2.java

Ahora vamos a implementar un servidor de sockets multithread. La idea es que el servidor multithread espere conexiones y para cada conexión cree un thread que procese los datos que envía el cliente.

Vamos a invocar el método accept dentro de un ciclo, y para cada conexión vamos a crear un thread.

```
class Servidor2
{
  static class Worker extends Thread
  {
    Socket conexion;
    Worker(Socket conexion)
    {
      this.conexion = conexion;
    }
    public void run()
    {
    }
  }
  public static void main(String[] args) throws Exception
  {
    ServerSocket servidor = new ServerSocket(50000);
    for (;;)
    {
      Socket conexion = servidor.accept();
      Worker w = new Worker(conexion);
      w.start();
    }
  }
}
```

Este código será la base para otros programas que desarrollaremos en el curso.

El constructor de la clase Worker pasa como parámetro el socket que crea el método accept, ya que el método run requiere el socket para recibir y enviar datos al cliente.

Ahora agregaremos el siguiente código al método run:

```
try
```

```
{
  DataOutputStream salida = new DataOutputStream(conexion.getOutputStream());
  DataInputStream entrada = new DataInputStream(conexion.getInputStream());
  int n = entrada.readInt();
  System.out.println(n);
  double x = entrada.readDouble();
  System.out.println(x);
  byte[] buffer = new byte[4];
  read(entrada,buffer,0,4);
  System.out.println(new String(buffer,``UTF-8``));
  salida.write(``HOLA``.getBytes());
  byte[] a = new byte[5*8];
  read(entrada,a,0,5*8);
  ByteBuffer b = ByteBuffer.wrap(a);
  for (int i = 0; i < 5; i++) System.out.println(b.getDouble());
}
catch(Exception e)
{
  System.err.println(e.getMessage());
}
finally
{
  try
  {
    conexion.close();
  }
  catch (Exception e2)
  {
    System.err.println(e2.getMessage());
  }
}
```

Podemos ver en el programa Servidor2.java que el método run crea los streams que se utilizarán para enviar y recibir datos del cliente. Notar que el programa Servidor2.java es completamente compatible con el programa Cliente.java

Capítulo 11

Sincronización del acceso a memoria

11.1. Operaciones de lectura y escritura

Los threads dentro de un proceso se comunican entre sí utilizando la memoria. Las operaciones que se realizan sobre la memoria son la lectura y escritura de variables (localidades de memoria).

Para que un thread pueda leer los datos que escribe otro thread en la memoria, es necesario ordenar las operaciones de lectura y escritura que realizan los threads.

Supongamos que tenemos dos threads, el thread T1 y el thread T2.

Si el thread T2 escribe a una variable y posteriormente el thread T1 lee la variable, el thread T1 tendrá el valor que escribió el thread T2.

11.2. Orden escritura-escritura

Ahora supongamos que los threads T1 y T2 escriben al mismo tiempo una variable y posteriormente los threads leen al mismo tiempo la misma variable. ¿Qué valores leyeron los threads?

Evidentemente no es posible garantizar que el thread T1 haya leído el valor que escribió el thread T2 o que haya leído el valor que escribió el mismo thread T1.

Igualmente, no es posible garantizar que el thread T2 haya leído el valor que escribió el thread T1 o que el thread T1 haya el valor que escribió el mismo thread T2.

Entonces, para garantizar que un thread lee el valor de una variable escrito por otro thread, es necesario ordenar las operaciones de escritura y de lectura que realizan los threads.

Ahora supongamos que el thread T1 espera a que el thread T2 escriba la variable, entonces después el thread T1 escribe a la variable.

En este caso, el valor que leen los threads T1 Y T2 es el valor que escribió el thread T1.

Notar que dos o más threads pueden leer simultáneamente una variable.

11.3. Orden escritura-lectura

Ahora supongamos que el thread T1 lee la variable y posteriormente el thread T2 escribe la variable ¿qué valor leyó el thread T1?

Para que el thread T1 pueda leer el valor que escribe el thread T2, es necesario ordenar las

operaciones de escritura y lectura.

Para garantizar que el thread T1 lea el valor escrito por el thread T2, el thread T1 debe esperar a que el thread T2 escriba la variable.

Si dos o más threads acceden a una misma variable, y al menos uno de los threads escribe la variable, entonces es necesario sincronizar el acceso de los threads a la variable.

Sincronizar el acceso de los threads significa ordenar las operaciones de escritura y de lectura que realizan los threads.

11.4. La instrucción `synchronized`

Para ordenar las lecturas y escrituras que realizan los threads en Java, se utiliza la instrucción synchronized.

```
synchronized (objeto)
{
  instrucciones
}
```

Debemos recordar que en Java todos los objetos tienen un lock asociado.

En la clase de Sistemas Operativos se explica que un lock (o cerrojo) es un mecanismo que permite limitar el acceso a un recurso compartido por varios procesos o threads, por ejemplo, una variable, un archivo, una impresora. etc.

A la capacidad que tiene un sistema de controlar el acceso a recursos compartidos se le conoce como exclusión mutua.

La instrucción `synchronized` funciona de la siguiente manera:

- Primero se verifica si el lock del objeto está bloqueado. Si el lock está bloqueado, el thread espera a que se desbloquee.
- Si el lock está desbloqueado, el thread lo bloquea —se dice que adquiere el lock— y ejecuta las instrucciones dentro del bloque.
- Al terminar las instrucciones, el thread desbloquea el lock —se dice que libera el lock—. Entonces, el sistema operativo notifica a uno de los threads que esperan el lock para que lo adquiera y ejecute las instrucciones dentro del bloque.
- Al terminar, el thread desbloquea nuevamente el lock y el sistema operativo notifica a alguno de los threads que esperan.

Como podemos ver, la instrucción `synchronized` evita que dos o más threads ejecuten simultáneamente un bloque de instrucciones. Al bloque de instrucciones que solo puede ser ejecutado por un thread se le llama sección crítica.

Veamos un ejemplo.

Supongamos que tenemos dos threads que incrementan una variable estática llamada “n“ dentro de un ciclo for.

La variable estática “n“ es “global“ a todas las instancias de la clase, por tanto los threads pueden leer y escribir esta variable.

```
class A extends Thread
{
  static long n;
  public void run()
  {
    for (int i = 0; i < 100000; i++)
        n++;
  }
  public static void main(String[] args) throws Exception
  {
    A t1 = new A();
    A t2 = new A();
    t1.start();
    t2.start();
    t1.join();
    t2.join();
    System.out.println( n );
  }
}
```

En este caso, la clase principal A es subclase de la clase Thread, por tanto hereda los métodos run, start y join (entre otros).

El programa debería desplegar 200000 ya que cada thread incrementa 100,000 veces la variable "n".

Preguntas:

- ¿Por qué despliega un número menor a 200,000?
- ¿Por qué cada vez que se ejecuta el programa despliega un número diferente?

La instrucción `n++` se compone de tres operaciones:

- Copiar el valor de la variable `n` a un registro del procesador.
- Incrementar el valor del registro.
- Escribir el valor del registro en la variable `n`.

Debido a que los threads t1 y t2 ejecutan en paralelo en una computadora con dos o más núcleos, el thread t1 puede leer y escribir la variable n al mismo tiempo que el thread t2:

Dado que la lectura que realiza t1 no está ordenada con respecto a la escritura que hace t2 y que la lectura que realiza t2 no está ordenada con respecto a la escritura que hace t1, es posible que no se escriba algún incremento en la variable n lo que produce un valor final menor a 200,000.

Entonces debemos impedir que ambos threads ejecuten al mismo tiempo la instrucción `n++`. Justamente esta instrucción es la sección crítica.

Para resolver este problema vamos a ejecutar la instrucción `n++` dentro de una instrucción `synchronized`; notar que utilizamos el objeto `obj` para sincronizar los threads. Entonces el código del programa queda de la siguiente manera:

```
class A extends Thread
{
```

```
  static long n;
  static Object obj = new Object();
  public void run()
  {
    for (int i = 0; i < 100000; i++)
      synchronized(obj)
      {
        n++;
      }
  }
  public static void main(String[] args) throws Exception
  {
    A t1 = new A();
    A t2 = new A();
    t1.start();
    t2.start();
    t1.join();
    t2.join();
    System.out.println( n );
  }
}
```

Al ejecutar varias veces el programa anterior podemos ver que el valor final de la variable n siempre es 200,000, debido a que ahora las operaciones de lectura y escritura se ejecutan en el orden correcto:

Si bien es cierto que es necesario sincronizar los threads para que el programa funcione correctamente, la sincronización hace más lento el programa, ya que obliga a que ciertas partes del programa se ejecuten en serie (una tras otra) y no en paralelo (al mismo tiempo).

11.5. Sincronización de threads

11.5.1. El método `wait()`

Cuando un thread ejecuta el método wait() de la clase Object, el thread entra en un estado de espera pasiva (el thread no consume ciclos de CPU mientras espera).

El método wait() debe utilizarse dentro de una instrucción synchronized.

En Java cada lock tiene una cola de espera, de manera que cada thread que ejecuta el método wait(), se forma en la cola de espera del lock asociado a la instrucción synchronized.

Una vez que el thread queda en estado de espera pasiva, se libera el lock. Cuando el thread "despierta" vuelve a adquirir el lock.

El método wait() puede producir la excepción InterruptedException, por tanto es necesario gestionar las interrupciones colocando el método wait() dentro de un bloque try.

Nota

Si un método no gestiona una interrupción, es necesario agregar al encabezado del método la claúsula throws seguida de una lista de las interrupciones que pueden producirse dentro del método, esto le dice al compilador que el método no gestiona esas interrupciones, por tanto, tendrá que gestionarlas el método que invoca al método actual.

Por ejemplo:

```
public static void main(String[] args) throws Exception
```

En este caso, el método “main” no va a gestionar ninguna excepción, por tanto, la cláusula “throws Exception” le dice al compilador que la gestión de la excepción Exception y todas sus subclases, la realizará el método que invoca al método “main”.

Cuando usamos “throws Exception”, la gestión de todas las excepciones se delega al método que invoca el método actual, debido a que “Exception“ es la superclase de todas las excepciones.

Debido a que el método “main” no es invocado por otro método Java, si se produce una excepción en el método “main”, el programa termina y se despliega en la pantalla la excepción que se produjo.

11.5.2. Los métodos `notify()` y `notifyAll()`

Para “despertar“ un thread que se encuentra en espera pasiva, otro thread deberá invocar el método notify() o el método notifyAll().

El método notify() “despierta“ el primer thread que se encuentra en la cola de espera del lock.

El método notifyAll() “despierta“ todos los threads que se encuentran en la cola de espera del lock.

Al igual que el método wait(), los métodos notify() y notifyAll() deben ejecutar dentro de una instrucción synchronized.

Debido a que los métodos wait(), notify() y notifyAll() son métodos de la clase Object, se heredan a todas las clases en Java.

El siguiente programa muestra un ejemplo del uso de wait() y notify().

En este caso se crea un thread el cual entra en modo de espera pasiva invocando el método wait(), entonces el thread principal “despierta“ el thread invocando el método notify().

```
class Notificacion
{
  static Object obj = new Object();
  static class Worker extends Thread
  {
    public void run()
    {
      try
      {
        synchronized(obj)
        {
          System.out.println(``antes de wait``);
          obj.wait();
          System.out.println(``después de wait``);
        }
      }
      catch(Exception e)
      {
        e.printStackTrace();
```

```
      }
    }
  }
  public static void main(String[] args) throws Exception
  {
    Worker w = new Worker();
    w.start();
    Thread.sleep(1000);
    synchronized(obj)
    {
      System.out.println(``antes de notify``);
      obj.notify();
      System.out.println(``después de notify``);
    }
  }
}
```

Capítulo 12

Cliente-servidor con sockets seguros

Para que un cliente y un servidor se comuniquen mediante sockets seguros, se requiere un certificado digital.

Para propósitos de prueba, vamos a crear un certificado autofirmado utilizando el programa keytool incluido en el JDK.

En Java los certificados y las llaves privadas correspondientes se almacenan en un archivo llamado "keystore" o repositorio de certificados.

Para crear el repositorio de certificados `keystore_servidor.jks`, ejecutar:

```
keytool -genkeypair -keyalg RSA -alias certificado_servidor -keystore keystore_servidor.jks -storepass
↪  1234567
```

La opción genkeypair genera un par de claves pública y privada.

La clave pública se pone en un certificado autofirmado con un solo elemento en la ruta de certificación. El alias, en este caso "certificado_servidor", define un nombre con el cual vamos a identificar el certificado. Keystore es un archivo (repositorio) donde se va a almacenar el certificado y la clave privada correspondiente. Keyalg es el algoritmo a utilizar para generar el par de claves, en este caso RSA. storepass es la contraseña para el keystore.

Entonces se deberán capturar los datos del certificado. Ejemplo:

```
¿Cuáles son su nombre y su apellido?
  [Unknown]:  nombre
¿Cuál es el nombre de su unidad de organización?
  [Unknown]:  unidad
¿Cuál es el nombre de su organización?
  [Unknown]:  organizacion
¿Cuál es el nombre de su ciudad o localidad?
  [Unknown]:  CDMX
¿Cuál es el nombre de su estado o provincia?
  [Unknown]:  CDMX
¿Cuál es el código de país de dos letras de la unidad?
  [Unknown]:  MX
¿Es correcto CN=nombre, OU=unidad, O=organizacion, L=CDMX, ST=CDMX, C=MX?
  [no]:  si
```

```
Introduzca la contraseña de clave para <certificado_servidor>

(INTRO si es la misma contraseña que la del almacén de claves):
```

>**Importante:** Para Java, la clave del certificado `<certificado_servidor>` y la clave (`storepass`) del almacén (`keystore`) deben ser iguales. En este caso: `1234567`.

>**Nota:** En un certificado firmado por una CA, el campo `CN` (nombre común) contiene el nombre del dominio que ampara el certificado digital. Cuando un navegador web se conecta a un servidor web, compara el campo `CN` con el dominio indicado en la URL. Si no coinciden, despliega una advertencia.

Debido a que el keystore "keystore_servidor.jks" contiene la llave privada del servidor, es conveniente crear un keystore para el cliente, el cual contenga solamente el certificado del servidor, para esto vamos a exportar el certificado "certificado_servidor" contenido en el keystore:

```
keytool -exportcert -keystore keystore_servidor.jks -alias certificado_servidor -rfc -file
↪  certificado_servidor.pem
```

La opción exportcert lee del keystore el certificado identificado por el alias y genera un archivo texto que contiene el certificado, en este caso se genera el archivo certificado_servidor.pem

Entonces vamos a crear un keystore que utilizará el cliente (repositorio de confianza), este keystore deberá contener el certificado del servidor:

```
keytool -import -alias certificado_servidor -file certificado_servidor.pem -keystore
↪  keystore_cliente.jks -storepass 123456
```

La opción import lee el archivo certificado_servidor.pem e inserta el certificado en el keytore keystore_cliente.jks, identificando el certificado mediante el alias. Storepass es la contraseña del keystore.

En la siguiente figura podemos ver que el servidor utilizará el keystore que contiene el certificado del servidor y la clave privada respectiva. El cliente utilizará el keystore que solo contiene el certificado del servidor.

Por otra parte, también es posible que los programas cliente y servidor utilicen un certificado firmado por una CA, en este caso será necesario que el keystore que utilizará el servidor contenga: 1) los certificados intermedios, 2) el certificado del servidor y 3) la clave privada correspondiente.

Debido a que el certificado del servidor es un certificado de confianza emitido por una CA, no es necesario crear el repositorio keystore_cliente.jks. Más adelante en el curso veremos el proceso de adquisición de un certificado digital para un dominio y cómo crear el keystore del servidor.

Ahora veremos cómo crear un cliente y un servidor los cuales se comunicarán mediante sockets seguros.

12.1. ClienteSSL.java

Primeramente asignamos a la propiedad "javax.net.ssl.trustStore" el nombre del keystore del cliente, y a la propiedad "javax.net.ssl.trustStorePassword" la contraseña del keystore del cliente:

```
System.setProperty(``javax.net.ssl.trustStore``, ``keystore_cliente.jks``);
System.setProperty(``javax.net.ssl.trustStorePassword``,``123456``);
```

Nota. El método System.setProperty(String nombre_propiedad,String valor_propiedad) permite asignar un valor a una propiedad (variable) del sistema (Java). Debido a que las propiedades son "globales", es posible recuperar el valor de una propiedad del sistema desde cualquier parte del código, utilizando el método System.getProperty(String nombre_propiedad).

Ahora el cliente debe crear una instancia de la clase SSLSocketFactory:

```
SSLSocketFactory cliente = (SSLSocketFactory) SSLSocketFactory.getDefault();
```

Entonces vamos a crear un socket que se conectará al servidor invocando el método createSocket de la clase SSLSocketFactory. En este caso el servidor se llama "localhost" (computadora local) y el puerto abierto en el servidor es el 50000.

```
Socket conexion = cliente.createSocket(``localhost``, 50000);
```

Abrimos los streams de salida y de entrada como lo hicimos anteriormente.

```
DataOutputStream salida = new DataOutputStream(conexion.getOutputStream());
DataInputStream entrada = new DataInputStream(conexion.getInputStream());
```

Ahora podemos enviar datos al servidor, por ejemplo vamos a enviar un double:

```
salida.writeDouble(123456789.123456789);
```

Para terminar el programa cerramos la conexión con el servidor (al cerrar el socket se cierran también los streams asociados), en este caso vamos a poner un retardo de un segundo antes de cerrar la conexión, para permitir que el servidor tenga tiempo de recibir los datos:

```
Thread.sleep(1000);
conexion.close();
```

12.2. ServidorSSL.java

Al igual que el cliente, el servidor requiere el nombre del keystore y la contraseña, entonces asignamos a la propiedad "javax.net.ssl.keystore" el nombre del keystore del servidor, y a la propiedad "javax.net.ssl.keystorePassword" la contraseña del keystore del servidor:

```
System.setProperty(``javax.net.ssl.keyStore``, ``keystore_servidor.jks``);
System.setProperty(``javax.net.ssl.keyStorePassword``,``1234567``);
```

El servidor debe crear una instancia de la clase SSLServerSocketFactory:

```
SSLServerSocketFactory socket_factory = (SSLServerSocketFactory) SSLServerSocketFactory.getDefault();
```

Vamos a crear un socket servidor que va a abrir, en este caso, el puerto 50000 utilizando el método createServerSocket de la clase SSLServerSocketFactory:

```
ServerSocket socket_servidor = socket_factory.createServerSocket(50000);
```

Ahora invocamos el método accept de la clase ServerSocket. Cuando se recibe la conexión el método accept regresa un socket:

```
Socket conexion = socket_servidor.accept();
```

Abrimos los streams de salida y de entrada:

```
DataOutputStream salida = new DataOutputStream(conexion.getOutputStream());
DataInputStream entrada = new DataInputStream(conexion.getInputStream());
```

Ahora podemos recibir datos del cliente, en este caso vamos a recibir un double:

```
double x = entrada.readDouble();
System.out.println(x);
```

Finalmente, cerramos la conexión:

```
conexion.close();
```

>**Nota:** Si el certificado del servidor fue emitido por una CA, no es necesario crear el repositorio `keystore_cliente.jks`. Por tanto, `ClienteSSL.java` no requiere inicializar las propiedades `javax.net.ssl.trustStore` y `javax.net.ssl.trustStorePassword`.

12.3. Implementación de un servidor HTTPS

Un servidor HTTPS es un servidor HTTP con sockets seguros, por lo tanto es muy sencillo convertir nuestro servidor HTTP en un servidor HTTPS.

Como vimos anteriormente, para establecer una comunicación segura entre un cliente y un servidor es necesario contar con un certificado digital, el cual puede ser un certificado autofirmado o bien, un certificado firmado por una autoridad certificadora (CA).

En el ejemplo que sigue utilizaremos el mismo repositorio que creamos anteriormente (keystore_servidor.jks), el cual contiene la llave privada y el certificado digital del servidor.

Debido a que el certificado autofirmado no es reconocido por el navegador como un certificado válido, al acceder a nuestro servidor HTTPS el navegador desplegará un mensaje de advertencia, indicando que el certificado no es de confianza porque está autofirmado.

Para propósito de pruebas, se puede dar clic al botón "Avanzado" y continuar, toda vez que el certificado lo hemos firmado nosotros. Cabe aclarar que en general no se debe acceder a una página web si el navegador despliega un mensaje de advertencia.

12.3.1. ServidorHTTPS.java

El único cambio que haremos en el código del servidor HTTP, es agregar las instrucciones necesarias para crear el socket servidor seguro:

```
public static void main(String[] args) throws Exception
{
    System.setProperty(``javax.net.ssl.keyStore``,``keystore_servidor.jks``);
    System.setProperty(``javax.net.ssl.keyStorePassword``,``1234567``);
    SSLServerSocketFactory socket_factory = (SSLServerSocketFactory)
                                            SSLServerSocketFactory.getDefault();
    ServerSocket servidor = socket_factory.createServerSocket(443);
    for(;;)
    {
      Socket conexion = servidor.accept();
      new Worker(conexion).start();
    }
}
```

El puerto que escucha el servidor es el 443 el cual es el estándar para HTTPS, sin embargo, es recomendable que el servidor utilice un puerto mayor a 1024 ya que los puertos menores o iguales a 1024 solo pueden ser abiertos por procesos que ejecutan con permisos de administración (sudo).

En general, es recomendable que un servidor (Apache, Tomcat, Weblogic, MySQL, Oracle, etc.) abra un puerto mayor que 1024 y se ejecute con permisos no administrativos, de lo contrario, se corre el riesgo de que un atacante tome el control del servidor y con ello pueda acceder a recursos del sistema.

Para probar el acceso a nuestro servidor HTTPS solo tenemos que ingresar la siguiente URL en un navegador:

```
https://localhost
```

Más adelante veremos cómo adquirir un certificado firmado por una autoridad certificadora y cómo crear un repositorio (keystore) con este certificado, de manera que cualquier navegador pueda conectarse a nuestro servidor HTTPS sin ningún problema de seguridad.

Capítulo 13

El tiempo en los sistemas distribuidos

El tiempo es una referencia que utilizan los sistemas distribuidos en varias situaciones.

Supongamos una plataforma de comercio electrónico que funciona a nivel global. En cada país se tiene un servidor con una base de datos donde se registran las compras, incluyendo la fecha y hora en la que se realiza cada compra.

Para consolidar las compras a nivel mundial cada servidor debe enviar los datos a un servidor central.

Sin embargo, no es posible ordenar las compras por fecha debido a dos situaciones:

- Cada compra se ha registrado con la fecha y hora local.
- No es posible garantizar que los relojes de los servidores funcionen a la misma velocidad.

Para ilustrar este problema supongamos que un cliente en México realiza una compra a las 8 PM, y un cliente en España realiza una compra a las 2 AM del día siguiente. ¿Quién compró primero?

Aparentemente el cliente en México realizó la compra antes que el cliente en España, debido a que la fecha de la compra del cliente en México es un día anterior a la fecha de la compra del cliente en España.

Sin embargo, en realidad el cliente en España realizó la compra una hora antes que el cliente en México, debido a que la diferencia horaria entre México y España es de 7 horas.

>*Mapa de los husos horarios oficiales vigentes (dominio público).*

La solución a este problema es registrar en las bases de datos la fecha y hora global en un solo campo de tipo Datetime o Timestamp, en lugar de la fecha y hora local. Además, los servidores deberán sincronizar sus relojes internos a una misma hora (mediante NTP).

Como se dijo anteriormente, el tiempo es una referencia para establecer un orden en una secuencia de eventos (como serían las compras en una plataforma de comercio electrónico).

Por otra parte, si los servidores no requieren consolidar las compras, tampoco será necesario que exista un acuerdo en los tiempos que marcan sus relojes.

El ejemplo anterior ilustra una regla muy importante de los sistemas distribuidos, la cual podemos enunciar de la siguiente manera: si dos computadoras no están conectadas, entonces no requieren sincronizar sus tiempos.

13.1. Sincronización de relojes

Sincronizar dos o más relojes significa que los servidores se ponen de acuerdo en una misma hora.

Notar que un grupo de servidores pueden ponerse de acuerdo en una hora y otro grupo de servidores puede ponerse de acuerdo en otra hora; solo si ambos grupos de servidores se conectan, entonces ambos grupos de servidores deberán acordar una hora.

Actualmente la mayoría de las computadoras y dispositivos móviles se encuentran conectados a Internet, por tanto, la mayoría de los relojes están sincronizados mediante un servidor de tiempo.

13.2. Segundos solares

El concepto de tiempo que utilizamos en la práctica se basa en la percepción que tenemos del día.

Un día es un período de luz y obscuridad debido a la rotación de la tierra sobre su eje.

Dividimos convencionalmente el día en 24 horas, cada hora en 60 minutos y cada minuto en 60 segundos.

Por tanto, la tierra tarda 86,400 segundos en dar una vuelta sobre su eje, en términos de velocidad angular estamos hablando de 360/86400=0.00416 grados/segundo.

A la fracción 1/86400 de día le llamamos segundo solar.

Sin embargo la velocidad angular de la tierra no es constante, debido a que la rotación de la tierra se está deteniendo lentamente.

13.3. Segundos atómicos

Una forma más precisa de medir el tiempo es utilizar un reloj atómico de Cesio 133.

En un reloj atómico se aplica microondas con diferentes frecuencias a átomos de Cesio 133, entonces los electrones del átomo de Cesio 133 absorben energía y cambian de estado; posteriormente los átomos regresan a su estado basal emitiendo fotones.

A la frecuencia que produce más cambios de estado en los electrones del átomo de Cesio 133 se le llama frecuencia natural de resonancia.

La frecuencia natural de resonancia del Cesio 133 es de 9,192,631,770 ciclos/segundo, es decir, el átomo de Cesio 133 muestra un máximo de absorción de energía cuando se le aplica microondas con una frecuencia de 9,192,631,770 Hertzios.

Entonces se define el segundo atómico como el recíproco de la frecuencia natural de resonancia del Cesio 133 (recordar que el periodo de una onda es el recíproco de su frecuencia).

Los relojes atómicos de Cesio 133 son extremadamente precisos, ya que independientemente de las condiciones ambientales (temperatura, presión, etc.), se adelantan o atrasan un segundo cada 300 millones de años.

Los relojes atómicos son tan precisos que se han utilizado para probar los postulados de la teoría general de la relatividad, la cual predice la dilatación del tiempo debidos a la distorsión que causa la gravedad al espacio-tiempo.

Utilizando relojes atómicos de Cesio 133 se ha demostrado que el tiempo no transcurre a la

misma velocidad a diferentes altitudes, ya que al nivel del mar, donde la gravedad es mayor, el tiempo se dilata (transcurre más lentamente) con respecto al tiempo medido en una montaña elevada donde la gravedad es menor.

Por ejemplo, un reloj atómico en un satélite GPS (a unos 20,000 Km de altura), se adelanta 7 microsegundos al día con respecto a un reloj atómico en la superficie de la tierra.

A este fenómeno se le conoce como dilatación gravitacional del tiempo.

13.4. Tiempo atómico internacional (TAI)

Se define el tiempo atómico internacional (TAI) como el promedio de los segundos atómicos transcurridos desde el 1 de enero de 1958, dicho promedio obtenido de casi 70 relojes de Cesio 133 al rededor del mundo.

13.5. Tiempo universal coordinado (UTC)

El tiempo universal coordinado UTC (Coordinated Universal Time) es el estándar de tiempo que regula actualmente el tiempo de los relojes a nivel internacional.

El tiempo UTC ha reemplazado el tiempo medio de Greenwich GMT.

El tiempo GMT toma como referencia la posición del sol a medio día. Tanto el tiempo GMT como el tiempo UTC consideran el día solar compuesto por 86,400 segundos solares.

Debido a que nuestro planeta disminuye su velocidad angular lentamente, el segundo solar dura más que el segundo atómico.

Para sincronizar los segundos UTC con los segundos TAI, el tiempo UTC se debe "adelantar" para alcanzar el tiempo TAI, para esto "se salta" un segundo UTC una vez al año; se dice entonces que se introducen segundos vacíos en el tiempo UTC.

Por ejemplo, en el siguiente diagrama se muestra cómo los segundos solares son más largos que los segundos atómicos, en esta caso para sincronizar los segundos solares (UTC) con los segundos atómicos (TAI), se salta del segundo 6 al 8, es decir, el segundo "vacío" es el segundo 7:

La autoridad responsable de introducir los segundos vacíos es el International Earth Rotation and Reference Systems Service (IERS).

Los proveedores de nube han adoptado el uso del tiempo UTC para los relojes en las máquinas virtuales, por ejemplo, cuando se ejecuta el comando date en una máquina virtual con Ubuntu en Azure, se obtiene la fecha y hora UTC.

El tiempo UTC es mantenido por el International Bureau of Weights and Measures (BIPM).

13.6. Network Time Protocol (NTP)

El protocolo de tiempo de red (Network Time Protocol - NTP) define un procedimiento centralizado para la sincronización de relojes.

En este procedimiento los clientes consultan un servidor de tiempo, el cual proporciona el tiempo UTC ajustado al tiempo TAI (mediante la introducción de segundos vacíos).

En el protocolo NTP el servidor A se conecta al servidor de tiempo B, entonces el servidor B responde con los tiempos T2 y T3.

T2 es el tiempo en que el servidor B recibe la conexión del servidor A y T3 es el tiempo en que el servidor B envía la respuesta al servidor A.

T1 es el tiempo en que el servidor A se conecta al servidor B y T4 es el tiempo en que el servidor A recibe la respuesta del servidor B.

Cuando el servidor A recibe la respuesta del servidor B, calcula el tiempo de respuesta Tres suponiendo que el tiempo que tarda el requerimiento Treq es igual al tiempo de respuesta Tres.

```
T4 - T1 = Treq + Tres + (T3 - T2) = 2 Tres + (T3 - T2)
```

Despejando Tres el servidor A puede establecer su tiempo en T3+Tres, es decir, el tiempo que envió el servidor B más el tiempo que tarda en llegar la respuesta.

Debido a que el servidor de tiempo puede saturarse al recibir muchas peticiones, se suele implementar el mismo procedimiento sobre una topología de árbol.

Los servidores de los estratos superiores del árbol son más exactos que los servidores de estratos inferiores, debido a que la diferencia entre treq y tres es menor.

13.7. Instalación de NTP en Ubuntu

Para asegurar que un servidor Ubuntu mantiene su reloj sincronizado con el tiempo UTC, podemos instalar NTP en el servidor ejecutando los siguientes comandos:

```
sudo apt-get update
sudo apt-get install ntp
```

13.8. Conclusiones

1. En una empresa, es muy importante que se configure el tiempo UTC en todas las computadoras de back-end, incluso aquellas que ejecutan sistemas locales.

Las máquinas virtuales que ejecutan en la nube utilizan el tiempo UTC, por esta razón, es recomendable que los servidores de la empresa también utilicen el tiempo UTC, de manera que no sea necesario modificar las aplicaciones cuando la empresa migre a la nube.

1. No se debe utilizar el tipo de dato Date (fecha) en la base de datos, es recomendable utilizar el tipo de dato Datetime (fecha-hora).
2. Un sistema no debe registrar la fecha-hora local en la base de datos; el sistema deberá registrar la fecha-hora UTC, de manera que el sistema esté preparado para migrar a la nube.
3. El front-end del sistema debe mostrar al usuario la fecha-hora local, mientras que el back-end debe guardar la fecha-hora UTC. El front-end deberá realizar la conversión hora-local a hora UTC y viceversa, es decir, el front-end enviará la fecha-hora UTC al back-end, así mismo, el back-end enviará al front-end la fecha-hora UTC.
4. Es necesario instalar NTP en los servidores para mantener sincronizados sus relojes.

Capítulo 14

Túnel SSH

El túnel SSH es un mecanismo que utiliza el comando SSH para implementar un canal encriptado, esto es, usando SSH es posible ejecutar un cliente el cual se comunicará con un servidor remoto (application server) utilizando el túnel SSH.

El sistema SSH consiste en un cliente y un servidor.

Para abrir una terminal con conexión segura ejecutamos el comando “ssh“ el cual se conecta al servidor SSH.

En general, las distribuciones físicas de Linux no tienen instalado el servidor SSH, por tanto, es necesario instalarlo.

Por otra parte, las máquinas virtuales en la nube tienen instalado el servidor SSH, lo que permite al usuario conectarse a la máquina virtual utilizando el comando “ssh“.

El túnel SSH opera en la capa 5 del modelo OSI (capa de sesión) y en la capa 6 (capa de presentación). El túnel SSH utiliza la capa de sesión para establecer, mantener y finalizar la sesión entre el cliente y el servidor, y la capa de presentación para encriptar el tráfico entre el cliente y el servidor.

Cuando habilitamos el túnel SSH para comunicar el cliente con el servidor, el cliente se conecta localmente al cliente SSH (el cliente SSH hace las veces de servidor); entonces el cliente SSH envía los datos al servidor SSH, el cual a su vez redirige los datos al servidor (el servidor SSH hace las veces de cliente del servidor).

Debido a que el túnel SSH permite mapear los puertos, el cliente se puede conectar al cliente SSH a través de un puerto diferente, al puerto que utiliza el servidor SSH para conectarse al servidor.

El mapeo de puertos es de utilidad cuando el puerto que utiliza el servidor está ocupado.

En el siguiente diagrama podemos ver que el cliente se conecta a un servidor proxy en el cliente SSH, a través del puerto 3049, mientras que un cliente proxy en el servidor SSH, se conecta al servidor a través del puerto 2049:

Podemos notar que los programas cliente SSH y servidor SSH funcionan como proxys transparentes, cuando se utilizan para implementar un túnel SSH.

14.1. Implementación de un túnel SSH

Ahora vamos a crear una VPN cliente-servidor entre los programas Cliente.java y Servidor2.java utilizando un túnel SSH en modo local forwarding.

1. Crear una máquina virtual en Azure con Ubuntu 24.
2. Acceder a la máquina virtual utilizando SSH.
3. Instalar el JDK en la máquina virtual.
4. Enviar el programa ServidorHTTP.java a la máquina virtual.
5. Compilar y ejecutar en la máquina virtual el programa `ServidorHTTP.java`:

```
javac ServidorHTTP.java
java ServidorHTTP
```

1. En la computadora local (cliente) subir el túnel SSH:

```
ssh -N -f -L 80:localhost:8080 usuario@<ip-máquina-virtual>
```

Dónde:

- `-N`: no se ejecutará un comando remoto; solo se creará el túnel SSH.
- `-f`: `ssh` se ejecutará en segundo plano (*background*).
- `-L 80:localhost:8080`: el navegador se conecta al cliente SSH por el puerto 80. El destino `localhost:8080` indica que el servidor SSH se conecta al servidor HTTP por el puerto 8080. El túnel mapea el puerto 80 al puerto 8080.
- `usuario`: nombre del usuario en la computadora remota.
- `<ip-máquina-virtual>`: dirección IP de la computadora remota.

En Linux, ejecutando el siguiente comando, se puede ver que el túnel queda ejecutando en segundo plano:

```
ps -ef | grep ssh
```

1. En la computadora local (cliente), ingresar en el navegador:

```
http://localhost
```

En este caso suponemos que el navegador (cliente) se conecta al puerto local 80 de localhost, y el programa ServidorHTTP.java escucha el puerto 8080 (podemos ver que SSH mapea el puerto local 80 al puerto remoto 8080).

También se puede ejecutar el programa ServidorHTTP.java como comando remoto, desde la computadora local (cliente) de la siguiente manera:

```
ssh -L 80:localhost:8080 usuario@<ip-máquina-virtual> java ServidorHTTP
```

Notar que en este caso no se utilizaron las opciones -N y -f.

14.2. Autenticación mediante clave pública y clave privada

Cuando se ejecuta el comando SSH, la autenticación por omisión es mediante usuario y contraseña. Sin embargo, si queremos que el túnel SSH suba automáticamente en un script, será necesario implementar la autenticación mediante clave pública y clave privada.

14.2.1. Creación de la clave pública y la clave privada

1. Acceder a la computadora cliente utilizando SSH.
2. Para crear el par de claves, ejecutar el siguiente comando en la computadora local (cliente), la opción -t indica el algoritmo a utilizar y la opción -b el tamaño de la llave en bits:

```
ssh-keygen -t rsa -b 2048
```

Presionar la tecla Enter cuando se pregunta el nombre del archivo.

Cuando se pregunta la contraseña presionar la tecla “Enter“ si no se requiere la contraseña.

La contraseña permite encriptar la clave privada en la computadora local (cliente). Si se define una contraseña ésta se deberá ingresar cada vez que se ejecute SSH.

1. Para copiar la clave pública a la máquina virtual donde ejecuta el servidor:

```
ssh-copy-id usuario@<ip-máquina-virtual>
```

Dónde “usuario“ es el nombre del usuario en la máquina virtual donde ejecuta el servidor y <ip-máquina-virtual>es la dirección IP de la máquina virtual.

1. En la computadora local (cliente) probar el acceso a la máquina virtual, ahora la autenticación utilizará las claves pública y privada por tanto no se preguntará la contraseña:

```
ssh usuario@<ip-máquina-virtual>
```

>**Nota:**

Cuando se utiliza el par de llaves para la autenticación de SSH en Ubuntu, es recomendable incluir la opción -i (la cual indica la ruta de la llave privada) al crear el túnel.

Si el túnel escucha un puerto menor a 1024 del lado del cliente, se deberá crear el túnel utilizando “sudo“, lo que crea una sesión diferente a la sesión del usuario propietario del par de llaves, por tanto, “ssh“ no encuentra la llave privada y entonces pregunta la contraseña.

Por ejemplo, supongamos que el túnel escucha el puerto 80 del lado del cliente, y el usuario se llama “ubuntu“, entonces agregamos la opción -i al crear el túnel:

```
sudo ssh -i /home/ubuntu/.ssh/id_rsa -N -f -L 80:localhost:8080 ubuntu@<ip-máquina-virtual>
```

Por otra parte, cuando se utiliza el par de llaves para la autenticación de SSH en Windows, no es necesario utilizar la opción `-i`, ya que en Windows no es necesario ejecutar SSH con permisos de administrador.

Capítulo 15

Código fuente: Proxy.java

```
/*
  Proxy.java
  Carlos Pineda G. 2024
*/
import java.io.InputStream;
import java.io.OutputStream;
import java.io.IOException;
import java.net.Socket;
import java.net.ServerSocket;
class Proxy
{
  static String host_remoto;
  static int puerto_remoto;
  static int puerto_local;
  static class Worker_1 extends Thread
  {
    Socket cliente_1,cliente_2;
    Worker_1(Socket cliente_1)
    {
      this.cliente_1 = cliente_1;
    }
    public void run()
    {
      try
      {
        // se conecta al host remoto
        cliente_2 = new Socket(host_remoto,puerto_remoto);
        // thread que dirige el tráfico del host remoto al cliente
        new Worker_2(cliente_1,cliente_2).start();
        InputStream entrada_1 = cliente_1.getInputStream();
        OutputStream salida_2 = cliente_2.getOutputStream();
        byte[] buffer = new byte[1024];
        int n;
        while((n = entrada_1.read(buffer)) != -1)
        {
          salida_2.write(buffer,0,n);
          salida_2.flush();
        }
      }
      catch (IOException e)
      {
        // captura silenciosamente las excepciones por desconexión
```

```
    }
    finally
    {
      try
      {
        if (cliente_1 != null) cliente_1.close();
        if (cliente_2 != null) cliente_2.close();
      }
      catch (IOException e2)
      {
        e2.printStackTrace();
      }
    }
  }
}
static class Worker_2 extends Thread
{
  Socket cliente_1,cliente_2;
  Worker_2(Socket cliente_1,Socket cliente_2)
  {
    this.cliente_1 = cliente_1;
    this.cliente_2 = cliente_2;
  }
  public void run()
  {
    try
    {
      InputStream entrada_2 = cliente_2.getInputStream();
      OutputStream salida_1 = cliente_1.getOutputStream();
      byte[] buffer = new byte[4096];
      int n;
      while((n = entrada_2.read(buffer)) != -1)
      {
        salida_1.write(buffer,0,n);
        salida_1.flush();
      }
    }
    catch (IOException e)
    {
      // captura silenciosamente las excepciones por desconexión
    }
    finally
    {
      try
      {
        if (cliente_1 != null) cliente_1.close();
        if (cliente_2 != null) cliente_2.close();
      }
      catch (IOException e2)
      {
        e2.printStackTrace();
      }
    }
  }
}
public static void main(String[] args) throws Exception
{
  if (args.length != 3)
  {
```

```
      System.err.println(``Uso:\njava Proxy <host-remoto> <puerto-remoto> <puerto-local>``);
      System.exit(1);
    }
    host_remoto = args[0];
    puerto_remoto = Integer.parseInt(args[1]);
    puerto_local = Integer.parseInt(args[2]);
    System.out.println(``host_remoto: `` + host_remoto + ``, puerto_remoto: `` + puerto_remoto + ``,
    ↪  puerto_local: `` + puerto_local);
    ServerSocket ss = new ServerSocket(puerto_local);
    for(;;)
    {
      // espera una conexión del cliente
      Socket cliente_1 = ss.accept();
      // thread que dirige el tráfico del cliente al host remoto
      new Worker_1(cliente_1).start();
    }
  }
}
```

Capítulo 16

Código fuente: ServidorHTTP (variante).java

```
/*
  ServidorHTTP.java
  Carlos Pine3da G. 2026
*/

import java.net.ServerSocket;
import java.net.Socket;
import java.util.Date;
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.PrintWriter;
import java.nio.file.Files;
import java.nio.file.Paths;
import java.time.Instant;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.time.ZoneId;

class ServidorHTTP
{
  static class Worker extends Thread
  {
    Socket conexion;

    Worker(Socket conexion)
    {
      this.conexion = conexion;
    }

    int valor(String parametros, String variable) throws Exception
    {
      String[] p = parametros.split(``&``);
      for (int i = 0; i < p.length; i++)
      {
        String[] s = p[i].split(``=``);
        if (s[0].equals(variable))
          return Integer.parseInt(s[1]);
      }
      throw new Exception(``Se espera la variable: `` + variable);
```

```
  }

  public void run()
  {
    try
    {
      BufferedReader entrada = new BufferedReader(new InputStreamReader(conexion.getInputStream()));
      PrintWriter salida = new PrintWriter(conexion.getOutputStream());
      String fecha_if_modified_since = null;

      String req = entrada.readLine();
      System.out.println(``Petición: `` + req);

      for (;;)
      {
        String encabezado = entrada.readLine();
        System.out.println(``Encabezado: `` + encabezado);

        // obtiene la fecha enviada en el encabezado If-Modified-Since
        if (encabezado.startsWith(``If-Modified-Since: ``))
            fecha_if_modified_since = encabezado.substring(19);

        if (encabezado.equals(````)) break;
      }

      if (req.startsWith(``GET /suma?``))
      {
        String parametros = req.split(`` ``)[1].split(``\\?``)[1];

        String respuesta = String.valueOf(valor(parametros, ``a``) + valor(parametros, ``b``) +
        ↪  valor(parametros, ``c``));

        salida.println(``HTTP/1.1 200 OK``);
        salida.println(``Access-Control-Allow-Origin: *``);
        salida.println(``Content-type: text/plain; charset=utf-8``);
        salida.println(``Content-length: `` + respuesta.length());
        salida.println(``Connection: close``);
        salida.println();
        salida.println(respuesta);
        salida.flush();
      }
      else
      if (req.startsWith(``GET /``))
      {
        String path = req.split(`` ``)[1].substring(1);

        if (!path.equals(````) && Files.exists(Paths.get(path)))
        {
          byte[] buffer = Files.readAllBytes(Paths.get(path));

          // obtiene la fecha de modificación del archivo
          Instant instant = Files.getLastModifiedTime(Paths.get(path)).toInstant();
          String fecha_last_modified = LocalDateTime.ofInstant(instant,ZoneId.systemDefault()).form⌋
          ↪  at(DateTimeFormatter.ofPattern(``yyyy-MM-dd HH:mm:ss``));

          if (!fecha_last_modified.equals(fecha_if_modified_since))
          {
            salida.println(``HTTP/1.1 200 OK``);
            salida.println(``Content-type: text/html; charset=utf-8``);
```

```
            salida.println(``Content-length: `` + buffer.length);
            salida.println(``Last-Modified: `` + fecha_last_modified);
            salida.println(``Connection: close``);
            salida.println();
            salida.flush();
              conexion.getOutputStream().write(buffer);
          }
          else
          {
            salida.println(``HTTP/1.1 304 Not Modified``);
            salida.flush();
          }
        }
        else
        {
          String respuesta = ``<html><body><h1>404 File Not Found</h1></body></html>``;
          salida.println(``HTTP/1.1 404 Not Found``);
          salida.println(``Content-Type: text/html; charset=utf-8``);
          salida.println(``Content-Length: `` + respuesta.length());
          salida.println(``Connection: close``);
          salida.println();
          salida.println(respuesta);
          salida.flush();
        }
      }
    }
    catch (Exception e)
    {
      System.err.println(``Error en la conexión: `` + e.getMessage());
    }
    finally
    {
      try
      {
        conexion.close();
      }
      catch (Exception e)
      {
        System.err.println(``Error en close: `` + e.getMessage());
      }
    }
  }
}

public static void main(String[] args) throws Exception
{
  ServerSocket socket_servidor = new ServerSocket(80);

  for (;;)
  {
    Socket conexion = socket_servidor.accept();
    new Worker(conexion).start();
  }
}
```

Capítulo 17

Código fuente: ServidorHTTP.java

```
import java.net.ServerSocket;
import java.net.Socket;
import java.util.Date;
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.PrintWriter;
import java.nio.file.Files;;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.text.SimpleDateFormat;

class ServidorHTTP
{
  static class Worker extends Thread
  {
    Socket conexion;

    Worker(Socket conexion)
    {
      this.conexion = conexion;
    }

    int valor(String parametros, String variable) throws Exception
    {
      String[] p = parametros.split(``&``);
      for (int i = 0; i < p.length; i++)
      {
        String[] s = p[i].split(``=``);
        if (s[0].equals(variable))
          return Integer.parseInt(s[1]);
      }
      throw new Exception(``Se espera la variable: `` + variable);
    }

    public void run()
    {
      try
      {
        BufferedReader entrada = new BufferedReader(new InputStreamReader(conexion.getInputStream()));
        PrintWriter salida = new PrintWriter(conexion.getOutputStream());
          String fecha_if_modified_since = null;
```

```
        String req = entrada.readLine();
        System.out.println(``Petición: `` + req);

        for (;;)
        {
          String encabezado = entrada.readLine();
          System.out.println(``Encabezado: `` + encabezado);

          if (encabezado.startsWith(``If-Modified-Since: ``))
              fecha_if_modified_since = encabezado.substring(19);

          if (encabezado.equals(````)) break;
        }

        if (req.startsWith(``GET /suma?``))
        {
          String parametros = req.split(`` ``)[1].split(``\\?``)[1];

          String respuesta = String.valueOf(
            valor(parametros, ``a``) +
            valor(parametros, ``b``) +
            valor(parametros, ``c``)
          );

          salida.println(``HTTP/1.1 200 OK``);
          salida.println(``Access-Control-Allow-Origin: *``);
          salida.println(``Content-type: text/plain; charset=utf-8``);
          salida.println(``Content-length: `` + respuesta.length());
          salida.println(``Connection: close``);
          salida.println();
          salida.println(respuesta);
          salida.flush();
        }
          else if (req.startsWith(``GET /``))
        {
            Path path = Paths.get(req.substring(5,req.length()-9));

            if (Files.exists(path))
            {
              byte[] buffer = Files.readAllBytes(path);

              Date fecha_modificacion = new Date(Files.getLastModifiedTime(path).toMillis());
              String fecha_last_modified = String.format(``%04d-%02d-%02d %02d:%02d:%02d``,
                          fecha_modificacion.getYear()+1900,
                          fecha_modificacion.getMonth()+1,
                          fecha_modificacion.getDate(),
                          fecha_modificacion.getHours(),
                          fecha_modificacion.getMinutes(),
                          fecha_modificacion.getSeconds());

              if (!fecha_last_modified.equals(fecha_if_modified_since))
              {
              salida.println(``HTTP/1.1 200 OK``);
              salida.println(``Content-type: text/html; charset=utf-8``);
              salida.println(``Content-length: `` + buffer.length);
              salida.println(``Last-Modified: `` + fecha_last_modified);
              salida.println(``Connection: close``);
              salida.println();
              salida.flush();
```

```
            conexion.getOutputStream().write(buffer);
          }
          else
          {
          salida.println(``HTTP/1.1 304 Not Modified``);
          salida.flush();
          }
        }
        else
        {
          String respuesta = ``<html><body><h1>404 File Not Found</h1></body></html>``;
          salida.println(``HTTP/1.1 404 Not Found``);
          salida.println(``Content-Type: text/html; charset=utf-8``);
          salida.println(``Content-Length: `` + respuesta.length());
          salida.println(``Connection: close``);
          salida.println();
          salida.println(respuesta);
          salida.flush();
        }
      }
    }
    catch (Exception e)
    {
      System.err.println(``Error en la conexión: `` + e.getMessage());
    }
    finally
    {
      try
      {
        conexion.close();
      }
      catch (Exception e)
      {
        System.err.println(``Error en close: `` + e.getMessage());
      }
    }
  }

  public static void main(String[] args) throws Exception
  {
    ServerSocket socket_servidor = new ServerSocket(80);

    for (;;)
    {
      Socket conexion = socket_servidor.accept();
      new Worker(conexion).start();
    }
  }
}
```

Capítulo 18

Código fuente: ServidorHTTPS.java

```
/*
  ServidorHTTPS.java
  Carlos Pine3da G. 2026
*/

import java.net.ServerSocket;
import java.net.Socket;
import java.util.Date;
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.PrintWriter;
import java.nio.file.Files;
import java.nio.file.Paths;
import java.time.Instant;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.time.ZoneId;
import javax.net.ssl.SSLServerSocketFactory;

class ServidorHTTPS
{
  static class Worker extends Thread
  {
    Socket conexion;

    Worker(Socket conexion)
    {
      this.conexion = conexion;
    }

    int valor(String parametros, String variable) throws Exception
    {
      String[] p = parametros.split(``&``);
      for (int i = 0; i < p.length; i++)
      {
        String[] s = p[i].split(``=``);
        if (s[0].equals(variable))
          return Integer.parseInt(s[1]);
      }
      throw new Exception(``Se espera la variable: `` + variable);
    }
```

```
public void run()
{
  try
  {
    BufferedReader entrada = new BufferedReader(new InputStreamReader(conexion.getInputStream()));
    PrintWriter salida = new PrintWriter(conexion.getOutputStream());
    String fecha_if_modified_since = null;

    String req = entrada.readLine();
    System.out.println(``Petición: `` + req);

    for (;;)
    {
      String encabezado = entrada.readLine();
      System.out.println(``Encabezado: `` + encabezado);

      // obtiene la fecha enviada en el encabezado If-Modified-Since
      if (encabezado.startsWith(``If-Modified-Since: ``))
          fecha_if_modified_since = encabezado.substring(19);

      if (encabezado.equals(````)) break;
    }

    if (req.startsWith(``GET /suma?``))
    {
      String parametros = req.split(`` ``)[1].split(``\\?``)[1];

      String respuesta = String.valueOf(valor(parametros, ``a``) + valor(parametros, ``b``) +
      ↪  valor(parametros, ``c``));

      salida.println(``HTTP/1.1 200 OK``);
      salida.println(``Access-Control-Allow-Origin: *``);
      salida.println(``Content-type: text/plain; charset=utf-8``);
      salida.println(``Content-length: `` + respuesta.length());
      salida.println(``Connection: close``);
      salida.println();
      salida.println(respuesta);
      salida.flush();
    }
    else
    if (req.startsWith(``GET /``))
    {
      String path = req.split(`` ``)[1].substring(1);

      if (!path.equals(````) && Files.exists(Paths.get(path)))
      {
        byte[] buffer = Files.readAllBytes(Paths.get(path));

        // obtiene la fecha de modificación del archivo
        Instant instant = Files.getLastModifiedTime(Paths.get(path)).toInstant();
        String fecha_last_modified = LocalDateTime.ofInstant(instant,ZoneId.systemDefault()).form
        ↪  at(DateTimeFormatter.ofPattern(``yyyy-MM-dd HH:mm:ss``));

        if (!fecha_last_modified.equals(fecha_if_modified_since))
        {
          salida.println(``HTTP/1.1 200 OK``);
          salida.println(``Content-type: text/html; charset=utf-8``);
          salida.println(``Content-length: `` + buffer.length);
          salida.println(``Last-Modified: `` + fecha_last_modified);
```

```
            salida.println(``Connection: close``);
            salida.println();
            salida.flush();
              conexion.getOutputStream().write(buffer);
          }
          else
          {
            salida.println(``HTTP/1.1 304 Not Modified``);
            salida.flush();
          }
        }
        else
        {
          String respuesta = ``<html><body><h1>404 File Not Found</h1></body></html>``;
          salida.println(``HTTP/1.1 404 Not Found``);
          salida.println(``Content-Type: text/html; charset=utf-8``);
          salida.println(``Content-Length: `` + respuesta.length());
          salida.println(``Connection: close``);
          salida.println();
          salida.println(respuesta);
          salida.flush();
        }
      }
    }
    catch (Exception e)
    {
      System.err.println(``Error en la conexión: `` + e.getMessage());
    }
    finally
    {
      try
      {
        conexion.close();
      }
      catch (Exception e)
      {
        System.err.println(``Error en close: `` + e.getMessage());
      }
    }
  }
}

public static void main(String[] args) throws Exception
{
  System.setProperty(``javax.net.ssl.keyStore``,``keystore_servidor.jks``);
  System.setProperty(``javax.net.ssl.keyStorePassword``,``1234567``);
  SSLServerSocketFactory socket_factory = (SSLServerSocketFactory)
  ↪  SSLServerSocketFactory.getDefault();

  ServerSocket socket_servidor = socket_factory.createServerSocket(443);

  for (;;)
  {
    Socket conexion = socket_servidor.accept();
    new Worker(conexion).start();
  }
}
}
```

Capítulo 19

Código fuente: p.html

```
<html>
  <script>
    function get(req,callback)
    {
      const xhr = new XMLHttpRequest();
      xhr.open('GET', req, true);
      xhr.onload=function()
      {
        if (callback != null) callback(xhr.status,xhr.response);
      };
      xhr.send();
    }
  </script>
  <body>
    <button onclick=``get('/suma?a=1&b=2&c=3',function(status,response){alert(status + ' ' +
    ↪  response);})``>Aceptar</button>
  </body>
</html>
```

Parte II

Unidad 2

Capítulo 20

Cómputo en la nube

20.1. Aprovisionamiento de recursos informáticos On-premise

En el pasado, el aprovisionamiento de recursos informáticos On-premise (en las instalaciones de la empresa) representaba el concurso de diferentes proveedores de bienes y servicios, como eran los representantes de ventas, ingenieros de preventa, fabricantes de los equipos, fabricante del sistema operativo, fabricante de la base de datos, agentes aduanales, transportistas, instaladores del site, proveedor de energía, proveedor de comunicaciones, instaladores del hardware, instaladores del software, entre otros.

Entonces, el aprovisionamiento de recursos informáticos era un proceso complejo y tardado, el cual culminaba con el sistema en producción.

Después había que reaprovisionar cuando crecían las necesidades de la empresa.

20.2. Cómputo como servicio

El concepto de cómputo en la nube (cloud computing) está basado en las ideas del profesor John McCarthy de la Universidad de Stanford.

En un discurso en 1961, predijo que la tecnología de tiempo compartido (la posibilidad de que múltiples usuarios y procesos utilicen una misma computadora) podría llevar en el futuro, a ofrecer las capacidades de cómputo y las aplicaciones como servicios públicos.

Esta idea se concretaría en lo que actualmente conocemos como cómputo en la nube, al ofrecer servicios virtualizados a nivel masivo en Internet.

Por otra parte, en 1984 John Burdette Gage acuñó el slogan The network is the computer para Sun Microsystems, el cual significa que la computadora de escritorio es una "ventana" a la red, y mediante el software apropiado, es posible transferir tareas de cómputo a otras computadoras.

A la posibilidad de trasferir tareas de cómputo a otras computadoras se le conoce como computation offloading.

20.3. Fábricas de información

En 2006 aparece el artículo "The Information Factories" en la prestigiosa revista WIRED [1].

En este artículo se presenta la idea de "fábricas de información" (Information Factories).

El autor, George Gilder, argumenta que la tecnología de la información evoluciona hacia un modelo de procesamiento y almacenamiento de datos centralizados en grandes centros de datos.

Estos centros de datos estarían equipados con grandes servidores y redes de alta velocidad.

Entonces, en lugar de depender de computadoras personales, los usuarios accederán a servicios informáticos a través de redes de comunicación utilizando dispositivos más simples, como serían las computadoras portátiles y los dispositivos móviles como tabletas y teléfonos celulares.

>"The desktop is dead. Welcome to the Internet cloud, where massive facilities across the globe will store all the data you'll ever use", The Information Factories, George Gilder.

Este modelo de informática centralizada permitiría contar con capacidades "ilimitadas" de procesamiento y almacenamiento, lo cual proporcionará niveles de escalabilidad sin precedentes.

Si bien George Gilder no introduce el concepto de cómputo en la nube como tal, su artículo explora las implicaciones sociales y económicas de este modelo de computación basado en la idea de "fábrica de información".

20.4. Proveedores de servicios en la nube

20.4.1. Amazon

En 2006 Amazon lanzó la plataforma Amazon Web Services (AWS) incluyendo los servicios Elastic Computer (EC2) y Storage Cloud (S3).

AWS popularizó el modelo de Infraestructura como Servicio (IaaS, Infrastructure as a Service), el cual permite a las empresas alquilar recursos de cómputo y almacenamiento en lugar de invertir en hardware propio.

20.4.2. Google

En 2008 Google lanzó Google App Engine, inicialmente como una plataforma para desarrollar y alojar aplicaciones web.

Posteriormente, Google lanza el servicio Google Cloud Storage en 2010 y en 2013 el servicio Google Compute Engine (GCE). Con estos servicios Google entra en el mercado de infraestructura como Servicio (IaaS).

La marca Google Cloud Platform (GCP) se lanzó oficialmente en 2011.

20.4.3. Microsoft

En 2010 Microsoft lanzó la plataforma Windows Azure (ahora conocida como Microsoft Azure), con servicios para desarrolladores al nivel de plataforma (PaaS), entre otros:

- Windows Azure Compute para la ejecución de aplicaciones en la nube utilizando máquinas virtuales administradas por Microsoft.
- Windows Azure Storage para el almacenamiento de datos estructurados y no estructurados.
- Windows Azure SQL Database el cual es SQL Server en la nube.

En 2012 Microsoft lanzó los servicios:

1. Azure Virtual Machines, con el cual Microsoft entra en el mercado de IaaS.

2. Azure Virtual Network el cual permite crear redes virtuales en la nube.
3. Azure Storage IaaS el cual ofrece almacenamiento para las máquinas virtuales.

En 2015 Microsoft lanza el servicio Azure Container Service el cual permite ejecutar contenedores Docker en Azure.

20.5. Definición de cómputo en la nube

El Instituto Nacional de Estándares y Tecnología (NIST) de Estados Unidos, define el cómputo en la nube de la siguiente manera [2]:

>"Cloud computing is a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction."

20.6. Las cinco características esenciales del cómputo en la nube

20.6.1. 1. Autoservicio bajo demanda

Los usuarios pueden unilateralmente aprovisionar automáticamente capacidades en la infraestructura del proveedor del servicio de nube, tales como tiempo de servidor, almacenamiento y red, de acuerdo a sus necesidades sin intervención humana.

20.6.2. 2. Acceso amplio por medio de la red

Las capacidades que ofrece el proveedor de nube están disponibles a través de la red, y son accedidas mediante mecanismos estándares utilizando clientes heterogéneos como teléfonos celulares, tabletas, computadoras de escritorio, etc.

20.6.3. 3. Recursos compartidos

Los recursos de cómputo del proveedor de nube son compartidos por múltiples usuarios usando un modelo "multiinquilino" (multitenant). Los recursos físicos y virtuales (procesamiento, almacenamiento, memoria, ancho de banda, etc.) son asignados y reasignados dinámicamente de acuerdo a las necesidades de los usuarios.

Los recursos asignados son transparentes en términos de la ubicación, se manera que el usuario no tiene que saber qué recurso físico está utilizando. No obstante, el usuario puede determinar en qué región o datacenter creará los recursos virtuales.

20.6.4. 4. Elasticidad

Las capacidades pueden ser aprovisionadas y des-aprovisionadas elásticamente (manualmente o automáticamente), permitiendo al usuario escalar hacia arriba o hacia abajo, horizontalmente o verticalmente, bajo demanda. La escalabilidad hacia arriba con frecuencia parece ser ilimitada.

20.6.5. 5. Medición del servicio

Los recursos de nube pueden ser monitoreados con la finalidad de controlar y optimizar automáticamente su utilización. Este monitoreo se realiza a diferentes niveles de abstracción de acuerdo al tipo de servicio (almacenamiento, procesamiento, ancho de banda, cuentas de usuario, etc.).

20.7. Los tres modelos de servicio del cómputo en la nube

20.7.1. 1. Infraestructura como servicio

En el nivel de Infraestructura como servicio o Infrastructure as a Service (IaaS), el usuario puede aprovisionar procesamiento (máquinas virtuales), almacenamiento (discos virtuales), redes y otros servicios que le permiten ejecutar cualquier software.

En este nivel el usuario tiene el control de sistema operativo, el almacenamiento, algunos componentes de la red, y las aplicaciones que ejecutan sobre el sistema operativo.

20.7.2. 2. Plataforma como servicio

En el nivel de plataforma como servicio o Platform as a Service (PaaS), el usuario puede aprovisionar servicios de instancias de bases de datos, respaldos, ejecución de servicio web, plataformas de desarrollo y pruebas, y otros servicios sin la necesidad de aprovisionar o controlar la infraestructura subyacente (servicios de IaaS como serían la máquina virtual, la red o el almacenamiento).

20.7.3. 3. Software como servicio

En el nivel de software como servicio o Software as a Service (SaaS), el usuario puede utilizar las aplicaciones publicadas por el proveedor de nube, las cuales ejecutan sobre la infraestructura de nube. El usuario no tiene que aprovisionar servicios de PaaS o de IaaS.

Los servicios al nivel de SaaS se acceden mediante una aplicación cliente, p.e. un navegador (web browser). Los usuarios acceden a las aplicaciones de SaaS mediante un usuario y contraseña o mediante un par de claves.

20.8. Los cuatro modelos de despliegue del cómputo en la nube

20.8.1. 1. Nube privada

Los servicios de nube son aprovisionados para el uso exclusivo de una sola organización.

La organización puede ser propietaria de la infraestructura, de un tercero o de una combinación de ambas.

De igual forma, la administración y operación puede ser realizada por la misma organización, por un tercero o una combinación de ambas.

En cuanto a la ubicación, la infraestructura puede estar on-premise (en las instalaciones de la organización), o bien, off-premise (fuera de las instalaciones de la organización).

20.8.2. 2. Nube comunitaria

Los servicios de nube son aprovisionados para el uso de una comunidad de usuarios u organizaciones con objetivos compartidos (p.e. empresas, instituciones de gobierno, etc.).

Al igual que la nube privada, la propiedad, administración y operación de la infraestructura puede ser de la comunidad, un tercero o una combinación de ambos.

La infraestructura puede estar ubicada en instalaciones de la comunidad, un tercero o una combinación de ambas.

20.8.3. 3. Nube pública

Los servicios de nube son aprovisionados para el público en general.

La infraestructura está ubicada en las instalaciones del proveedor de los servicios de nube (Amazon, Microsoft, Google, etc.).

20.8.4. 4. Nube híbrida

La infraestructura de nube es una composición de dos o más modelos de despliegue (nube privada, nube comunitaria, nube pública).

La comunicación entre las redes de las diferentes nubes, se realiza mediante tecnologías propietarias o estándares que permitan implementar canales de comunicación seguros (como son las VPN).

Capítulo 21

Principios del cómputo en la nube

21.1. Servicios virtuales a nivel masivo

El cómputo en la nube es un modelo de arquitectura basado en una infraestructura de cómputo ofrecida como servicios virtuales a nivel masivo.

El concepto clave en el cómputo en la nube es el "servicio", así, se ofrece:

- IaaS (Infrastructure as a Service) Servicio de creación de infraestructura virtual y física.
- PaaS (Platform as a Service) Servicios de DBMS, plataformas de desarrollo y pruebas.
- SaaS (Software as a Service) Servicio de aplicaciones de software.
- Otros servicios con la terminación "as a Service":
- DaaS (Data as a Service) Servicio de acceso a datos.
- DRaaS (Disaster Recovery as a Service) Servicio de recuperación de máquinas virtuales ante desastres.

21.2. Escalabilidad en la nube

La escalabilidad es la capacidad de un sistema en la nube de aumentar o disminuir los recursos aprovisionados (CPU virtuales y memoria de una máquina virtual, disco de sistema operativo, disco de datos.

La escalabilidad vertical se refiere al crecimiento o descrecimiento de las capacidades de una computadora.

La escalabilidad horizontal se refiere a la creación o eliminación de máquinas virtuales.

21.3. Disponibilidad en la nube

La disponibilidad es la capacidad de un sistema en la nube para estar accesible (disponible) en todo momento, de manera que los usuarios puedan acceder el sistema.

La disponibilidad se mide en porcentaje que representa el tiempo durante el cual el sistema está disponible (tiempo de actividad o uptime).

La disponibilidad es: **100 x el tiempo de actividad / tiempo total**.

Por ejemplo, si el sistema está disponible 364 días de 365 días, entonces la disponibilidad es del 99.7260 %

Los proveedores de nube establecen la disponibilidad de los servicios en un SLA (Service Level Agreement) o Acuerdo de nivel de servicio.

21.4. Confiabilidad en la nube

La confiabilidad es la capacidad de un sistema en la nube para funcionar durante largos períodos de tiempo sin interrupciones.

Para garantizar la confiabilidad, los proveedores de nube implementan diversas estrategias, por ejemplo:

- **Geo-Redundant Storage.** La replicación de los respaldos en regiones a cientos de kilómetros de la región actual.
- **Zona de disponibilidad.** Uno o más datacenters equipados con su propia alimentación, refrigeración y red.
- **Dominios de error.** Las máquinas virtuales que pertenecen a un mismo dominio de error comparten la misma fuente de alimentación y red física.
- **Dominios de actualización.** Las máquinas virtuales dentro de un mismo dominio de actualización se reiniciarán simultáneamente durante el mantenimiento planeado. Solo se reiniciará un dominio de actualización a la vez.
- **Balanceo de carga.** Un balanceador de carga permite escalar un sistema, y al mismo tiempo evita puntos únicos de falla aumentando la disponibilidad.
- **Site Recovery.** Permite replicar y recuperar máquinas virtuales en caso de falla o desastre.
- **Respaldos.** Copia de seguridad de máquinas virtuales y archivos, permitiendo la recuperación de los datos en caso de ransomware, pérdida o corrupción de los datos
- **Alta disponibilidad en DBMS.** Crea una réplica de la base de datos en una zona de disponibilidad diferente.

21.5. Transparencia

La transparencia es la capacidad de los usuarios para acceder los recursos de nube sin la necesidad de conocer los detalles de la infraestructura de nube.

Podemos dividir la transparencia en siete categorías:

- **Transparencia en el acceso a los datos.** Los usuarios y aplicaciones acceden a los datos en la nube de manera estandarizada.
- **Transparencia de ubicación.** Los usuarios de nube acceden a los recursos independientemente de su localización física.
- **Transparencia de migración.** Los usuarios de nube no son afectados por la migración de los datos y de los procesos que pueda realizar la plataforma.
- **Transparencia de reubicación.** La transparencia de reubicación se refiere a la capacidad de la nube de cambiar la ubicación de un recurso mientras está en uso, sin que el usuario que accede el recurso se vea afectado.

- **Transparencia de replicación.** La transparencia de replicación es la capacidad de la nube de ocultar la existencia de recursos replicados.
- **Transparencia de concurrencia.** La transparencia de concurrencia se refiere a la capacidad de la nube de ocultar el hecho de que varios usuarios y procesos comparten los diferentes recursos de manera concurrente.
- **Transparencia ante fallas.** La transparencia ante fallas es la capacidad de la nube de ocultar una falla.

Por ejemplo, si un sistema que se encuentra totalmente replicado, cuando el sistema principal falla entonces el usuario accederá de manera transparente a la réplica del sistema.

Más adelante en el curso veremos cómo replicar un sistema completo en la nube utilizando un administrador de tráfico de red y cómo implementar sistemas tolerantes a fallas en la nube mediante balance de carga.

21.6. Rendimiento

El rendimiento es la eficiencia con la que un sistema en la nube utiliza los recursos de nube, incluyendo la capacidad de procesamiento, almacenamiento, ancho de banda, entre otros recursos.

21.7. Desempeño

El desempeño se refiere a la velocidad con la ejecuta un sistema en la nube.

El desempeño depende de las técnicas de programación y del rendimiento del sistema. El objetivo es mantener el sistema funcionando de manera consistente incluso bajo cargas de trabajo pesadas.

21.8. La elasticidad en la nube

Debido a que el cómputo en la nube está basado fundamentalmente en la virtualización de los recursos informáticos, este modelo de arquitectura ofrece una ventaja única, la posibilidad de hacer crecer y decrecer los recursos aprovisionados.

Supongamos un servicio de streaming bajo demanda, como es el caso de Netflix. En este tipo de servicio la demanda crece los fines de semana y decrece los días entre semana.

Si el proveedor del servicio no aprovisiona los recursos suficientes para atender la demanda del fin de semana, entonces muchos usuarios se quedarán sin servicio.

Por otra parte, si el proveedor del servicio aprovisiona los recursos necesarios para atender a sus usuarios el fin de semana, estos recursos estarán subutilizados los días entre semana, lo cual resulta en pérdidas económicas.

Sin lugar a dudas, el éxito que han alcanzado las empresas proveedoras de streaming bajo demanda, se debe a que su modelo de negocio está basado en la posibilidad que les ofrece la nube para crecer y decrecer los recursos aprovisionados, a esta característica de la nube se le llama elasticidad.

El cómputo elástico es la habilidad de hacer crecer y decrecer rápidamente la capacidad de cómputo (CPUs), la memoria y el almacenamiento para adaptarse a la demanda.

Para implementar el cómputo elástico se utilizan herramientas de monitoreo, las cuales aprovisionan y des-aprovisionan recursos conforme son necesarios, sin detener la operación.

Sin embargo, hay proveedores de nube que afirman que no existe tal cosa como "nube privada", ya que el tema de elasticidad se ve acotado en un equipo "privado" debido a la limitada escalabilidad.

En cambio, en la nube pública la escalabilidad es casi ilimitada, ya que si se agotan los recursos de un data center, sin ningún problema se puede escalar a otro data center, ya sea del mismo proveedor o de otros proveedores.

Una característica importante de la nube (pública) es "pagar por lo que se usa", esto significa que solo se paga por los recursos aprovisionados.

En el caso de un centro de cómputo tradicional (también es el caso de la "nube privada") la empresa paga por todo el equipo, lo use a toda su capacidad o no.

Consideremos la siguiente gráfica:

21.8.1. Escenario On-premise

Al inicio los requerimientos informáticos de la empresa son pocos, sin embargo ésta debe hacer una gran inversión inicial ya que debe adquirir el equipo que le permita operar durante un periodo de tiempo determinado.

Para saber cuál será la inversión inicial se deberá hacer una planeación de capacidad.

Suponiendo que la empresa crece, entonces en determinado momento deberá realizar la actualización de su equipo informático.

Entonces se deberá hacer una planeación de capacidad para establecer el tamaño de la inversión, estimando un tiempo determinado de operación.

No obstante las previsiones, la empresa podría decrecer, por tanto los requerimientos informáticos también decrecen. El costo de la actualización no se recupera, incluso, en poco tiempo el equipo se hará obsoleto.

21.8.2. Escenario nube

Ahora supongamos que la empresa utiliza servicios en la nube. Entonces la inversión inicial es mínima, ya que la empresa contratará los servicios indispensables para operar. Conforme la empresa crece, la elasticidad de la nube permite adecuar el tamaño de la infraestructura informática de acuerdo a las necesidades.

Si la empresa decrece, también decrecen sus necesidades y el costo de la infraestructura informática. En la gráfica, las barras corresponden al costo de los servicios de nube. Podemos ver que este costo se ajusta a las necesidades de la empresa.

Podemos concluir que al utilizar los servicios de nube se optimiza la relación costo/beneficio, ya que solo pagamos lo que realmente usamos.

Parte III

Unidad 3

Capítulo 22

Almacenamiento en la nube

Los servicios de almacenamiento en la nube que vamos a estudiar en el curso son los siguientes:

- Azure Disk Storage. Discos para máquinas virtuales.
- Azure Blob Storage. Grandes objetos binarios (Binary Large Objects) no estructurados.
- Azure Files. Archivos compartidos (File Systems) en la nube.

22.1. Azure Disk Storage

El servicio de Azure Disk Storage proporciona almacenamiento persistente para máquinas virtuales.

Cuando se crea una máquina virtual, se crea un disco de arranque para el sistema operativo.

También es posible crear discos de datos, independientes del disco de sistema operativo, los cuales se pueden adjuntar y quitar de la máquina virtual en cualquier momento.

Los discos pueden ser discos administrados (managed disks) o discos no administrados (unmanaged disks), así mismo, pueden ser HDD (Hard Disk Drive) o SSD (Solid State Disk).

Los discos de datos son utilizados como almacenamiento persistente para bases de datos, servidores de archivos (p.e. servidores web) y servidores de aplicaciones.

Azure Disk Storage ofrece:

- Disponibilidad de 99.999 %
- Encriptado.
- Durabilidad grado empresarial con tres réplicas.
- Respaldo nativo.
- Recuperación ante desastres.
- Monitoreo.

22.2. Discos no administrados

Los discos no administrados deben ser administrados por el usuario, esto es, el usuario es responsable de administrar la infraestructura de almacenamiento y la replicación para alta dispo-

nibilidad.

Los discos no administrados se almacenan en una cuenta de almacenamiento como blobs VHD (Virtual Hard Disk). El usuario debe crear y administrar la cuenta de almacenamiento y adjuntar los discos VHD a las máquinas virtuales.

La escalabilidad de los discos no administrados está limitada por la capacidad de la cuenta de almacenamiento.

22.3. Discos administrados

En los discos administrados, Azure se encarga de la administración del almacenamiento y la replicación para alta disponibilidad.

Los discos administrados se pueden adjuntar directamente con las máquinas virtuales.

La escalabilidad no se ve limitada por la capacidad de una cuenta de almacenamiento.

Los discos administrados pueden ser respaldados y restaurados directamente.

22.3.1. Discos HDD Standard

Los discos HDD estándares son recomendados para cargas de trabajo que no son críticas y accesos a los datos poco frecuentes.

Los discos HDD estándares ofrecen una capacidad máxima de 32 TB, con 2,000 IOPS (operaciones de entrada/salida por segundo) y un ancho de banda de 500 MB por segundo.

22.3.2. Discos SSD Standard

Los discos SSD estándares son recomendados para cargas de trabajo de producción, con requerimientos de latencia consistente a bajo costo.

Los discos SSD estándares ofrecen una capacidad máxima de 32 TB, con 6,000 IOPS (operaciones de entrada/salida por segundo) y un ancho de banda de 750 MB por segundo.

22.3.3. Discos SSD Premium

Los discos SSD premium son recomendados para cargas de trabajo de producción, con requerimientos de alto rendimiento.

Los discos SSD premium ofrecen una capacidad máxima de 32 TB, con 20,000 IOPS (operaciones de entrada/salida por segundo) y un ancho de banda de 900 MB por segundo.

22.3.4. Discos Ultra

Los discos ultra son recomendados para las aplicaciones más demandantes en entrada/salida intensiva y latencias por debajo de un milisegundo.

Los discos ultra ofrecen una capacidad máxima de 64 TB, con 160,000 IOPS (operaciones de entrada/salida por segundo) y un ancho de banda de 2 GB por segundo.

22.3.5. Crear un disco de datos administrado

1. Ingresar al portal de Azure.
2. En el campo de búsqueda escribir ”Discos”.
3. Seleccionar la pestaña Çomenzar”.
4. Dar clic en el botón Çrear”.
5. Seleccionar un grupo de recursos o crear uno nuevo.
6. Ingresar en nombre del disco.
7. Seleccionar la región (la región debe ser la misma que la región de la máquina virtual a la que se conectará el disco).
8. En el campo ”Zona de disponibilidad”seleccionar la zona de disponibilidad o bien, seleccionar ”No se requiere redundancia de la infraestructura”.

1. En el campo ”Tamaño”seleccionar Çambiar de tamaño”.
2. En el campo ”Tipo de almacenamiento”seleccionar el tipo de disco a crear, p.e. ”SSD estándar (almacenamiento con redundancia local)”.

Nota. El disco SSD Premium solo se puede adjuntar a máquinas virtuales creadas en una zona de disponibilidad.

1. Seleccionar el tamaño del disco, p.e. 4 GB.

1. Dar clic en el botón Revisar y crear”.
2. Dar clic en el botón Çrear”.

22.3.6. Adjuntar un disco de datos a una máquina virtual

1. Seleccionar la máquina virtual.
2. Seleccionar la opción ”Discos”.
3. Seleccionar la opción .Adjuntar discos existentes”.

1. En el campo ”Nombre del disco”seleccionar el disco a adjuntar.
2. Dar clic en el botón .Aplicar”.

Notar que un disco de datos puede ser adjuntado a la máquina virtual encendida.

En Azure los nombres de los discos se asignan en el orden en que son detectados por el sistema, p.e. si el primer disco detectado es /dev/sda (el disco de sistema operativo), /dev/sdb sería el segundo, y así sucesivamente. En este caso, el primer disco de datos adjuntado sería /dev/sdc ya que ya existe un disco /dev/sdb.

Para ver los nombres de los discos (asignados por Azure) y sus particiones, se puede ejecutar el siguiente comando:

```
lsblk
```

Cuando se crea un nuevo disco de datos, éste viene como disco RAW, lo cual significa que no tiene un sistema de archivos (filesystem) que permita almacenar datos de manera estructurada. Por tanto, es necesario formatearlo antes de que pueda ser montado y utilizado.

22.3.7. Formatear un disco de datos

1. Formatear con el sistema de archivos ext4 un disco de datos (p.e. /dev/sdc) recién creado:

```
sudo mkfs.ext4 /dev/sdc
```

22.4. Montar un disco de datos

1. Crear un punto de montaje (p.e. /mnt/datos):

```
sudo mkdir /mnt/datos
```

1. Montar el disco en el punto de montaje:

```
sudo mount /dev/sdc /mnt/datos
```

22.4.1. Quitar un disco de datos de una máquina virtual

1. Ingresar a la máquina virtual y desmontar el disco.
2. Seleccionar la máquina virtual.
3. Seleccionar la opción ”Discos”.
4. Dar clic en ícono ”Desasociar”que aparece al final del renglón correspondiente al disco que se va a quitar.
5. Dar clic en el botón .Aplicar”.

1. Ahora podemos ejecutar el comando lsblk para ver que el disco ya no está conectado a la máquina virtual.

Notas:

- Un disco de datos puede ser desasociado de la máquina virtual encendida.
- Antes de desasociar un disco de datos es necesario desmontarlo.

22.4.2. Eliminar un disco de datos

1. Ingresar al portal de Azure.
2. En el campo de búsqueda escribir ”Discos”.
3. Seleccionar la pestaña Recursos”.
4. Seleccionar el disco a eliminar.
5. Dar clic en el ícono .Eliminar”.

Reproducir Vídeo

22.4.3. Encriptado de un disco de datos

Como una medida se seguridad, adicional al encriptado que realiza Azure, vamos a encriptar el disco de datos utilizando criptografía simétrica.

1. Crear un punto de montaje, p.e. /mnt/securedata:

```
sudo mkdir /mnt/securedata
```

1. Instalar el módulo dm-crypt:

```
sudo apt-get update
sudo apt-get install dmsetup cryptsetup
```

1. Cargar el módulo dm-crypt:

```
sudo modprobe dm-crypt
```

1. Verificar que el device-mapper haya reconocido el modulo dm-crypt y agregado crypt como destino disponible:

```
sudo dmsetup targets
```

1. Debido a que el disco recien creado tiene un formato RAW, debemos formatear el disco para encripción, en este caso el disco es /dev/sdc:

```
sudo cryptsetup -y luksFormat /dev/sdc
```

Se debe ingresar la clave (passphrase).

1. Verificar que el disco ha sido inicializado para encripción:

```
sudo cryptsetup luksDump /dev/sdc
```

1. Inicializar un device mapper para ser montado (aqui "securedata.[es] el nombre del device mapeado, notar que el device mapper se coloca en el directorio /dev/mapper: /dev/mapper/securedata):

```
sudo cryptsetup luksOpen /dev/sdc securedata
```

1. Formatear el device mapper (en este caso con el tipo de filesystem ext4):

```
sudo mkfs.ext4 -m 0 /dev/mapper/securedata
```

1. Montar el device mapper:

```
sudo mount /dev/mapper/securedata /mnt/securedata
```

1. Para ver el filesystem montado, ejecutar el comando:

```
df
```

Notar que el directorio punto de montaje es propiedad del usuario root y del grupo root, si es necesario se puede cambiar el propietario y grupo.

1. Para desmontar el device mapper y cerrar el encriptado:

```
sudo umount /mnt/securedata
sudo cryptsetup luksClose securedata
```

1. Para montar nuevamente el disco (cada vez que se reinicie la máquina virtual):

```
sudo cryptsetup luksOpen /dev/sdc securedata
sudo mount /dev/mapper/securedata /mnt/securedata
```

22.4.4. Cómo leer un disco a nivel RAW

El siguiente programa permite leer un disco a nivel RAW (a un nivel por debajo del sistema de archivos), para verificar que los datos se almacenan encriptados, por otra parte, si el disco se formateó y se montó sin encriptar, entonces los datos aparecerán en claro:

```
// Carlos Pineda G. 2012
// Requiere instalar gcc:
// sudo apt-get update
// sudo apt-get install gcc
// Se compila:
// gcc -o p p.c
// Se ejecuta:
// sudo ./p
#include <stdio.h>
#include <errno.h>
#include <ctype.h>
#define LONG_BUFFER (4*1024)
char buffer[LONG_BUFFER];
int main ()
{
```

```
  FILE *f = fopen("aquí va el nombre del device, p.e. /dev/sdc","rb");
  int i;
  if (f == NULL)
  {
fprintf(stderr,"Error al abrir el device, errno=%i\n",errno);
return -1;
  }
  for(;;)
  {
fread(buffer,LONG_BUFFER,1,f);
if (feof(f))
break;
for (i = 0; i < LONG_BUFFER; i++)
if (isprint(buffer[i]))
printf("%c",buffer[i]);
  }
  return 0;
}
```

Capítulo 23

La clase de hoy vamos a ver el servicio de respaldos en la nube de Azure.

23.1. Microsoft Azure Backup

Azure Backup es un servicio de respaldos en la nube al nivel de PaaS, que no requiere que el usuario cuente con el personal para realizar los respaldos ni la instalación de una infraestructura de almacenamiento.

Azure Backup permite respaldar y restaurar archivos, directorios, máquinas virtuales completas, bases de datos de SQL Server, o archivos y directorios On-Premise (mediante un agente que ejecuta en el equipo local).

Azure Backup es un servicio al nivel de plataforma (PaaS: Platform as a Service) debido a que se trata de un servicio de respaldos totalmente administrado sin la necesidad de crear una infraestructura (máquina virtual y almacenamiento).

Para proteger al usuario de ataques de ransomware (secuestro de datos) Azure Backup implementa la autenticación multi-factor. Así mismo, genera alertas si se detecta actividad sospechosa de respaldo o restauración de los datos del usuario.

Reproducir Vídeo

23.2. Punto de restauración

Cada vez que se ejecuta el proceso de respaldo (backup job) para un recurso (p.e. una máquina virtual), se crea un punto de restauración, entonces es posible utilizar cualquier punto de restauración para recuperar los datos que se tenían en un momento dado.

Después del primer respaldo completo, los subsecuentes respaldos serán incrementales, es decir, solo van a incluir los cambios (archivos nuevos, archivos eliminados y archivos modificados) realizados desde el último respaldo.

23.3. Almacén de Recovery Services

Un almacén de Recovery Services (vault) es un contenedor lógico que almacena los puntos de restauración.

Cuando el proceso de respaldo es eliminado, Azure Backup retiene por dos semanas los datos en el almacén del recovery services, de manera que el usuario puede recuperar los respaldos en caso de borrado accidental.

Al eliminar el proceso de respaldo no se elimina el almacén de Recovery Services.

Al eliminar el proceso de respaldo, los puntos de restauración quedan en estado soft-deleted durante 14 días (o más si se configura Enhanced Soft Delete con un período mayor).

Mientras existan puntos de restauración en soft-deleted, no se puede eliminar el vault.

Pasados los 14 días, los puntos de restauración se borran definitivamente, sin embargo, el vault sigue existiendo. Por tanto, es necesario eliminar explícitamente el vault. Una característica importante de Azure Backup es que no limita la transferencia de datos de entrada o de salida.

La transferencia de datos de entrada se refiere a los datos transferidos al almacén de Recovery Services cuando se realiza el respaldo.

La transferencia de datos de salida se refiere a los datos que son transferidos desde el almacén de Recovery Services cuando se realiza una restauración.

23.4. Geo-Redundant storage

Por omisión, el almacén de Recovery Services se crea como Geo-Redundant storage, lo cual asegura que los datos respaldados se replicarán en una región a cientos de kilómetros de la región actual.

23.5. Políticas de respaldo

Es posible crear políticas de respaldo (también llamadas directivas de copia de seguridad), donde se define cuando se ejecuta el proceso de respaldo (backup job) y por cuánto tiempo se almacenarán los puntos de restauración.

Por omisión, la política de respaldo DailyPolicy ejecuta un respaldo diario y mantiene los puntos de restauración por 30 días.

23.6. Restauración de una máquina virtual

La restauración de una máquina virtual se realiza seleccionando un punto de restauración.

Para restaurar una máquina virtual completa no es necesario detener o eliminar la máquina virtual previamente respaldada.

Para conectarse a la máquina virtual restaurada de utiliza el usuario y contraseña de la máquina virtual respaldada.

23.7. Costo de Azure Backup

Azure Backup tiene costo (Azure Backup pricing) sin embargo es más barato que aprovisionar una máquina virtual con discos de almacenamiento para ejecutar los respaldos y mantener los datos.

El costo mensual del servicio de Azure Backup se calcula en base al tamaño del disco de la máquina virtual más el almacenamiento.

23.8. Ventajas de Azure Backup

Azure Backup tiene entre otras ventajas:

- La replicación automáticamente en otra región geográficamente alejada.
- Los mecanismos de seguridad adicionales como las notificaciones de actividad sospechosa y la autenticación multi-factor.
- La creación de puntos de restauración de la máquina virtual en diferentes tiempos sin tener que sacarla de producción.

23.9. Referencias

- Quick start - Back up a virtual machine in Azure
- Azure Backup pricing
- How to restore Azure VM data in Azure portal
- Redundancia de Azure Storage
- What is Azure Backup service?

Capítulo 24

Actividades individuales a realizar

En esta actividad vamos a realizar un respaldo de una máquina virtual en Azure y la recuperación de la máquina virtual a partir del respaldo.

24.1. Habilitar el respaldo de una máquina virtual en Azure

1. En el portal de Azure, seleccionar la máquina virtual que se va a respaldar o bien, crear una nueva máquina virtual.

1. Seleccionar la opción ”Backup”.

1. Crear un almacén de Recovery Services.

1. Seleccionar el grupo de recursos donde se colocará el almacén.

1. Seleccionar la política de respaldo, por omisión DailyPolicy, o dar clic en Çrear una nueva directiva”para crear una nueva política de respaldo.

Si se crea una nueva política se puede definir la frecuencia de respaldo (diario o semanal), la hora en la que se realizará el respaldo y el tiempo que se conservará los puntos de restauración.

1. Dar clic en el botón ”Habilitar Backup”

1. Dar clic en la campana de notificaciones para verificar que se haya realizado la implementación del proceso de respaldo.

24.2. Iniciar un respaldo completo

Anteriormente vimos cómo habilitar un proceso de respaldo (backup job) para realizar un respaldo diario de una máquina virtual completa a cierta hora del día.

Para iniciar al momento el respaldo completo de la máquina virtual:

1. Seleccionar la máquina virtual en el portal de Azure. Seleccionar ”Backup.[en] el menú de operaciones.

1. Seleccionar Realizar copia de seguridad ahora”para crear el primer respaldo completo de la máquina virtual. Los subsecuentes respaldos automáticos serán incrementales.

1. Indicar la fecha de retención de la copia de seguridad o aceptar la fecha establecida en la política de respaldo utilizada (por omisión, 30 días).

1. Dar clic en el botón .Aceptar"

1. Dar clic en la campana de notificaciones para verificar que se haya iniciado el respaldo de la máquina virtual.

1. Para ver el progreso del respaldo seleccionar la opción "Ver todos los trabajos.en la página "Backup"de la máquina virtual. Seleccionar la opción .Actualizar"para refrescar la pantalla que muestra el estado del proceso de respaldo. El respaldo ha terminado cuando se despliega Çompletada".

El tiempo que tarda el respaldo depende del número de procesadores virtuales, el tamaño de la memoria RAM y el tamaño del disco en la máquina virtual. El respaldo tardará más si la máquina virtual tiene poca memoria y pocos procesadores virtuales o el disco es grande.

Una vez terminado el respaldo en la página "Backup"de la máquina virtual aparecerá el punto de restauración creado.

24.3. Restaurar una máquina virtual

Para restaurar una máquina virtual completa:

1. Seleccionar la máquina virtual en el portal de Azure.

1. Seleccionar la opción "Backup.en el menú de .Operaciones".

1. Seleccionar la opción Restaurar VM".

1. En "Punto de restauración"dar clic en la opción "Seleccionar".

1. Seleccionar el punto de restauración y dar clic en el botón .Aceptar".

1. En "Tipo de restauración"seleccionar Çrear una nueva máquina virtual".

1. Ingresar el nombre de la nueva máquina virtual.

1. Seleccionar la red virtual.

1. Seleccionar la ubicación del almacenamiento provisional. Esta cuenta de almacenamiento se utilizará temporalmente durante la restauración.

Si no se encuentra una cuenta de almacenamiento será necesario crearla de la siguiente manera (cerrar la ventana actual Restauración de la máquina virtual"):

9.1 En la ventana de búsqueda de Azure escribir: cuentas de almacenamiento 9.2 Dar clic en la opción +Crear 9.3 Seleccionar el grupo de recursos de la máquina virtual. 9.4 Ingresar un nombre para la cuenta de almacenamiento (no debe existir en Azure). 9.5 Seleccionar la misma ubicación del vault (almacén de Recovery Services) en el procedimiento Habilitar el respaldo de una máquina virtual en Azure. 9.6 En Replicación"seleccionar .Almacenamiento con redundancia local (LRS)"9.7 Dar clic en el botón Revisar y crear". 9.8 Dar clic en el botón Çrear". 9.9 Ir al primer paso del procedimiento Restaurar una máquina virtual.

1. Dar clic en el botón Restaurar".

1. Dar clic en la campana de notificaciones para verificar que se haya iniciado la restauración de la máquina virtual.

1. Para ver el progreso de la restauración seleccionar la opción "Ver todos los trabajos.en la página "Backup"de la máquina virtual. Seleccionar la opción .Actualizar"para refrescar la pantalla que muestra el estado del proceso de restauración.

Una vez terminada la restauración de la máquina virtual, la nueva máquina virtual aparecerá en la lista de máquinas virtuales en el portal de Azure.

Para conectar con la nueva máquina virtual se utilizará las mismas credenciales (usuario y contraseña) definidas para la máquina virtual respaldada.

Podemos verificar que la configuración de la nueva máquina virtual es idéntica a la configuración de la máquina virtual respaldada.

24.4. Eliminar un proceso de respaldo

Para eliminar un proceso de respaldo y los puntos de respaldo asociados:

1. Seleccionar la máquina virtual en el portal de Azure.

1. Seleccionar la opción "Backup.en el menú de .Operaciones".

1. Seleccionar "Detener copia de seguridad". Si no se ve la opción presionar los tres puntos ...

1. Seleccionar la opción Retener datos de copia de seguridad.o bien .Eliminar datos de copia de seguridad".

1. Ingresar el nombre del elemento de copia de seguridad, es este caso el nombre de la máquina virtual respaldada.

1. Opcionalmente se puede indicar el motivo por el cual se va a eliminar el proceso de respaldo. También es posible escribir algún comentario en la ventana Çomentarios".

1. Dar clic en el botón "Detener copia de seguridad".

1. Dar clic en la campana de notificaciones para verificar que se haya detenido el proceso de copia de seguridad y en su caso, se haya eliminado los datos.

1. Para eliminar el almacén de Recovery Services (vault) hacer lo siguiente:

Nota importante: Para eliminar el almacén de Recovery Services es necesario que hayan pasado 14 días desde el último respaldo, ya que la retención de los datos es por dos semanas: Recovery Services vault cannot be deleted as there are backup items in soft deleted state in the vault. The soft deleted items are permanently deleted after 14 days of delete operation". Fuente: Portal de Microsoft Azure.

9.1 Ir al inicio del portal de Azure. 9.2 Seleccionar "Todos los recursos. 9.3 Seleccionar el vault (almacén de Recovery Services) a eliminar (tener la precaución de seleccionar el almacén correcto).

9.4 Seleccionar también la cuenta de almacenamiento, si se creó la cuenta de almacenamiento en el paso 9 del procedimiento Restaurar una máquina virtual. 9.5 Seleccionar la opción .Eliminar". 9.6 Confirmar la eliminación del almacén dando clic al botón "Sí"9.7 Dar clic en la campana de notificaciones para verificar que se haya eliminado el almacén.

En clases anteriores vimos servicios de la nube al nivel de infraestructura (IaaS: Infrastructure as a Service), como son la creación de máquinas virtuales con Windows y Linux, la creación de imágenes de máquinas virtuales, NSG, NIC, creación de redes y subredes virtuales, VPN Gateways y balance de carga.

La clase de hoy veremos el servicio administrado de base de datos MySQL, el cual ofrece Azure al nivel de plataforma (PaaS: Platform as a Service).

24.5. Azure Database for MySQL

Cuando una empresa instala sus sistemas en la nube requiere así mismo tener acceso a un DBMS.

Uno de los más populares manejadores de bases de datos en la actualidad es MySQL (usado por empresas de clase mundial como Google, YouTube, Facebook, Twitter, PayPal, etc.), el cual puede utilizarse en su versión gratuita (Community Server) o en su versión Enterprise Edition.

Como hemos visto en clases previas, es fácil instalar MySQL sobre una máquina virtual, crear una base de datos y acceder a ella utilizando el monitor de mysql o un programa escrito en algún lenguaje como Java, C#, Python, Node.js, entre otros.

Sin embargo, en un entorno empresarial será necesario llevar a cabo la administración de MySQL, lo cual incluye la instalación, monitoreo, respaldo de las bases de datos, recuperación ante desastres, espejo de discos, etc.

Entonces la empresa tiene dos opciones, la primera es contratar el personal que se encargue de la administración del DBMS, y la segunda contratar el servicio completo en la nube. Este servicio de DBMS completamente administrado es un ejemplo de plataforma como servicio (PaaS).

Azure ofrece MySQL Community Server completamente administrado como servicio.

Este servicio de nube, llamado Azure Database for MySQL, permite a la empresa delegar la administración de la base de datos y contar con alta disponibilidad (99.99 %) y escalabilidad dinámica, así como el acceso remoto a MySQL mediante conexión segura.

24.6. Referencias

- ¿Qué es PaaS?
- Azure Database for MySQL
- ¿Qué es Azure Database for MySQL (servidor flexible)?

24.7. Crear un servidor en Azure Database for MySQL

Ahora vamos a crear una instancia de MySQL al nivel de PaaS en la nube de Azure.

Para crear una instancia de MySQL:

1. Ingresar al portal de Azure.

1. En la barra de búsqueda escribir: Servidores de Azure Database for MySQL

1. Dar clic en -Crear”

1. Dar clic en el botón Çreación avanzada.[en] el cuadro "Servidor flexible".

1. Seleccionar un grupo de recursos existente o crear uno nuevo.

1. Ingresar el nombre del servidor, este nombre debe ser único en Azure ya que se le agregará un subdominio de Azure.

1. Seleccionar la región.

Nota. No todas las regiones soportan la creación de instancias de MySQL, si se produce un error al crear la instancia, cambiar la región.

1. Seleccionar la versión de MySQL, por ejemplo 8.0

1. En el campo "Tipo de carga de trabajo"seleccionar "Para proyectos de desarrollo o aficiones"

1. Seleccionar la opción Çonfigurar servidor.[en] la sección "Proceso y almacenamiento".

1. En el campo Çomputer tier"seleccionar "Flexible".

1. En el campo "Tamaño de proceso"seleccionar "Standard_B1ms".

1. En el campo "Storage size (in GiB)"seleccionar: 20

1. En el campo ÏOPS"seleccionar ÏOPS aprovisionados previamente". Mover el control deslizante a los IOPS (I/O per second) requerimos.

1. Habilitar Çrecimiento automático del almacenamiento".

1. Dar clic en el botón "Guardar".

1. En el campo "Método de autenticación"seleccionar .Autenticación de MySQL". Ingresar el "Nombre de usuario de administrador"(notar que no se permite ingresar rootτa que este usuario está reservado para la administración del DBMS por parte de Azure).

1. Ingresar la contraseña. Confirmar la contraseña ingresada.

1. Dar clic en el botón "Siguiente: Redes >".

1. En el campo "Método de conectividad"seleccionar .Acceso público".

Nota. Si se selecciona .Acceso privado"se deberá indicar la red y subred a la que se conectará la máquina virtual que ejecutar MySQL.

20.1 Habilitar el campo "Permitir el acceso público a este recurso a través de Internet mediante una dirección IP pública"

1. Ahora vamos a configurar el firewall del servidor:

21.1 Para que la computadora actual pueda conectarse al servidor (p.e. para ejecutar el monitor de mysql), seleccionar la opción - Agregar dirección IP del cliente actual".

21.2 Para que otra computadora se pueda conectar al servidor, ingresar en los campos ÏP inicialτ en ÏP final"la IP de la computadora.

21.3 Para que cualquier computadora se pueda conectar al servidor, ingresar 0.0.0.0 como IP inicial y 255.255.255.255 como IP final, pero esto no es recomendable por razones de seguridad.

21.4 Para que todos los recursos de Azure (máquinas virtuales, Azure Functions, etc.) tengan acceso al servidor (incluso los recursos que no están en la misma suscripción), seleccionar "Si.[en] la opción "Permitir el acceso público a este servidor desde cualquier servicio de Azure dentro de Azure"

1. Dar clic en el botón Revisar y crear".

1. Dar clic en el botón Çrear".

1. En el panel seleccionar la opción İnformación general.[en] el menú de la izquierda de la pantalla.

Podemos ver el nombre del dominio del servidor (en este caso el servidor se llama prueba-mysql):

prueba-mysql.mysql.database.azure.com

1. Dar clic al botón Revisar.[en] la sección "Ver cadena de conexión".

25.1 Abrir la pestaña Çonectarse desde el explorador o localmente"

```
Ahora podemos conectarnos al servidor de MySQL ejecutando el siguiente comando:

mysql -h prueba-mysql.mysql.database.azure.com -P 3306 -u administrador -p
```

En este caso el usuario configurado en el paso 17 es .[a]dministrador".

24.8. Crear una base de datos

```
create database servicio_web;

use servicio_web;

create table usuarios
(
id_usuario integer auto_increment primary key,
email varchar(100) not null,
nombre varchar(100) not null,
apellido_paterno varchar(100) not null,
apellido_materno varchar(100),
fecha_nacimiento datetime not null,
telefono varchar(20),
genero char(1)
);

create table fotos_usuarios
(
id_foto integer auto_increment primary key,
foto longblob,
id_usuario integer not null
);
```

```
alter table fotos_usuarios add foreign key (id_usuario) references usuarios(id_usuario);
create unique index usuarios_1 on usuarios(email);
```

24.9. Crear un usuario remoto

Para crear un usuario local en MySQL, se agrega al nombre del usuario "@localhost".

Para crear un usuario que pueda acceder en forma remota MySQL, en este caso el usuario se llama "hugo":

create user hugo identified by 'contraseña-del-usuario-hugo';

grant all on servicio_web.* to hugo;

24.10. Detener o eliminar la instancia de MySQL

Para evitar mayores gastos se puede detener o eliminar la instancia de MySQL.

Capítulo 25

Balance de carga en la nube

El balance de carga es la distribución equilibrada de carga entre un grupo de servidores (p.e. servidores web) o recursos en el back-end (p.e. unidades de almacenamiento).

La carga es el tráfico de red entrante a un recurso, por ejemplo, las peticiones a un servidor o las lecturas/escrituras a una unidad de almacenamiento.

25.1. Azure Load Balancer

El balanceador de carga en Azure opera en la capa de transporte (nivel 4) del modelo OSI, por tanto, soporta los protocolos TCP y UDP.

El balanceador de carga utiliza un algoritmo de distribución haciendo el hash de cinco elementos: la IP de origen, el puerto de origen, la IP de destino, el puerto de destino y el tipo de protocolo

En Azure se puede crear dos tipos de balanceadores de carga:

25.1.1. Balanceador de carga público

Un balanceador de carga público mapea la dirección IP pública (IP de front-end) y el puerto a una dirección IP privada y puerto de una máquina virtual (la IP pública y el puerto conforman un endpoint de Internet).

Utilizando reglas en el balanceador de carga, es posible distribuir la carga por tipo de tráfico (HTTP, HTTPS, FTP, SMTP, SSH, RDP, MySQL, SQLServer, etc).

25.1.2. Balanceador de carga interno

Un balanceador de carga interno distribuye el tráfico entre los recursos que se encuentran dentro de una red virtual.

Este tipo de balanceador de carga se utiliza para balancear carga (interna) que viene del firewall o para balancear la carga de las máquinas virtuales que ejecutan procesos de negocio que no son visibles al usuario externo.

La IP de un balanceador de carga interno nunca se expone como endpoint de Internet.

En un escenario de nube híbrida, es posible conectar servidores on-premise a un balanceador de carga interno.

25.2. Escalamiento mediante balance de carga

El balance de carga permite escalar las aplicaciones e implementar servicios con alta disponibilidad dentro de una zona de disponibilidad y a través de diferentes zonas de disponibilidad.

Agregando máquinas virtuales al balanceador de carga los sistemas incrementan su capacidad de atender peticiones, así mismo, se evita la interrupción del servicio cuándo uno de los servidores falla.

25.2.1. Sondeo de estado

Los sondeos de estado permiten verificar el estado de cada máquina virtual (instancia de back-end) conectada al balanceador de carga. Si el sondeo de estado no recibe respuesta de la máquina virtual entonces no se enviarán peticiones a la máquina virtual hasta que el sondeo de estado se restablezca.

El balanceador de carga estándar soporta sondeo de estado mediante los protocolos TCP, HTTP y HTTPS.

Antes de agregar reglas de equilibrio de carga a un balanceador de carga es necesario crear al menos un sondeo de estado. Ver: Sondeos de estado de Azure Load Balancer.

25.3. Región

Una región es un área geográfica con un conjunto de datacenters interconectados mediante una red de baja latencia.

25.3.1. Datacenter

Un datacenter es una instalación física con infraestructura de servidores, redes y almacenamiento.

Fuente: Microsoft

Cada datacenter cuenta con contenedores modulares (similares a los contenedores que utiliza el transporte marítimo).

Los contenedores se reciben completamente ensamblados y cuentan con tres entradas principales:

- Agua. Se utiliza para refrigerar los servidores.
- Energía. Suministra la energía eléctrica a los servidores.
- Red. Permite la conexión de los servidores con el exterior.

Cada contenedor puede tener cientos de servidores; los contenedores más grandes pueden tener entre 250 y 2,500 servidores.

Cada servidor tiene 2 o 4 CPU físicos, y cada CPU puede tener entre 24 y 64 núcleos (p.e. la serie Intel Xenon o AMD EPYC)

Cada servidor puede tener entre 64 GB y 1 TB de memoria RAM.

Cada servidor puede tener un almacenamiento local SSD entre 1 TB y 4 TB.

En los contenedores se usan redes de 10 Gbps a 100 Gbps para la conectividad interna y externa.

25.3.2. Zona de disponibilidad

Una zona de disponibilidad (Availability Zone) se compone por uno o más datacenters de una misma región, equipados con su propia alimentación, refrigeración y red.

Si una zona de disponibilidad falla, otras zonas estarán disponibles dentro de la misma región.

Las zonas de disponibilidad permiten implementar alta disponibilidad en caso de fallas de energía, red o refrigeración a nivel datacenter.

25.3.3. Conjunto de disponibilidad

Un conjunto de disponibilidad es una agrupación lógica de máquinas virtuales de una región, distribuidas en dominios de error y dominios de actualización.

Los conjuntos de disponibilidad reducen el riesgo de interrupciones en las máquinas virtuales por fallas de hardware o mantenimientos planificados.

El conjunto de disponibilidad se deberá asignar al momento de crear la máquina virtual, una vez creada la máquina virtual, no es posible asignar una máquina virtual a un conjunto de disponibilidad.

Un conjunto de disponibilidad consta de:

- 1, 2 o 3 dominios de error. Las máquinas virtuales que pertenecen a un mismo dominio de error comparten la misma fuente de alimentación, red física y refrigeración. En este caso, la alimentación, la red y la refrigeración son, cada uno, un único punto de fallo. En el siguiente ejemplo, se muestran cuatro máquinas virtuales distribuidas en dos dominios de error:

- De 1 a 20 dominios de actualización. Las máquinas virtuales dentro de un mismo dominio de actualización se reiniciarán simultáneamente durante el mantenimiento planeado. Solo se reiniciará un dominio de actualización a la vez. Si se creó un dominio de error, solo se puede crear un dominio de actualización. En el siguiente ejemplo, se muestran cuatro máquinas virtuales distribuidas en tres dominios de actualización; solo se apagará un dominio de actualización a la vez (en este caso, se está apagando el primer dominio de actualización) en caso de un mantenimiento planificado:

El conjunto de disponibilidad se deberá asignar al momento de crear la máquina virtual.

Azure no cobra por los conjuntos de disponibilidad; el pago es por cada máquina virtual dentro del conjunto de disponibilidad.

25.4. Costo del balance de carga en Azure

En Azure se puede configurar reglas de equilibrio de carga y reglas NAT.

Las reglas NAT son utilizadas para mapear el tráfico entre las direcciones IP públicas y privadas.

Las reglas de equilibrio de carga se cobran por número de reglas por hora.

Además, se cobra la cantidad de datos procesados de entrada y de salida, independientemente de las reglas.

En el Standard Load Balancer no se cobra por hora si no hay reglas configuradas.

Las reglas NAT son gratuitas. Ver: Precios de Load Balancer

El siguiente modelo muestra los componentes de un balanceador de carga en Azure:

Capítulo 26

Azure Blob Storage

Azure Blob Storage es un servicio que permite almacenar cantidades masivas de datos no estructurados, a un costo reducido.

Azure Blob Storage es escalable, durable, seguro y con la capacidad de manejar cargas de trabajo sobre demanda.

Azure Blob Storage puede almacenar objetos binarios, tales como datos binarios utilizados por las aplicaciones, videos, audios, textos, respaldos, etc. Los blobs se organizan en contenedores.

Azure Blob Storage permite optimizar los costos, ya que ofrece la posibilidad de colocar los datos en diferentes niveles de acceso (access tiers) de acuerdo a la frecuencia en la que son accedidos y/o modificados los datos:

- Frecuente (hot tier). Nivel online optimizado para almacenar datos que son accedidos y modificados frecuentemente. Este nivel es el más rápido y el más costoso.
- Esporádico (cool tier). Nivel online optimizado para almacenar datos que son accedidos y modificados con poca frecuencia. Este nivel es más lento y menos costoso que el nivel "frecuente". Los datos en el nivel .esporádico"deben almacenarse como mínimo 30 días.
- Inactivo (cold tier). Nivel online optimizado para almacenar datos que casi nunca son accedidos o modificados. Este nivel más lento y menos costoso que el nivel .esporádico". Los datos en el nivel ïnactivo"deben almacenarse como mínimo 90 días.
- Archivo (archive tier). Nivel offline optimizado para almacenar datos que casi nunca son accedidos y con la posibilidad de ser accedidos con altas latencias (del orden de horas). Los datos en el nivel .archivo"deben almacenarse como mínimo 180 días.

26.1. Notas

- El nivel de acceso predeterminado es "frecuente".
- Para cambiar al nivel de acceso .esporádico", se debe seleccionar la opción Çonfiguración.en la cuenta de almacenamiento.
- El nivel de acceso .archivo"solo se puede establecer para el blob no para la cuenta de almacenamiento.

Para acceder los blobs es posible utilizar APIs para .NET, Java, Python, JavaScript, C++, y Go, o bien, la API REST de Azure Blob Storage.

26.2. Reproducir Vídeo

26.3. Referencias

- Introduction to Azure Storage
- Niveles de acceso frecuente, esporádico y de archivo para los datos de blobs
- Delegación del acceso mediante una firma de acceso compartido
- Azure Blob Storage pricing

Para utilizar Azure Blob Storage, se debe crear una cuenta de almacenamiento, y dentro de ésta, se podrán crear contenedores; cada contenedor tiene una capacidad máxima de 100 TB. Una vez que hemos creado un contenedor, podemos cargar los archivos (blobs) al contenedor.

Ahora veremos cómo crear una cuenta de almacenamiento, un contenedor, y cómo cargar y acceder blobs.

26.4. Crear una cuenta de almacenamiento

1. Ingresar al portal de Azure.
2. En el campo de búsqueda escribir Çuentas de almacenamiento".
3. Seleccionar -Crear".
4. Seleccionar un grupo de recursos o crear uno nuevo.
5. Ingresar en nombre de la cuenta de almacenamiento. El nombre debe ser único en Azure, de 3 a 24 caracteres (letras minúsculas y dígitos).
6. Seleccionar la región.
7. En el campo "Servicio principal"seleccionar .Azure Blob Storage o Azure Data Lake Storage Gen2".
8. En el campo Rendimiento"seleccionar .Estándar". Esta selección no se puede cambiar después de crear la cuenta de almacenamiento.
9. En el campo Redundancia"seleccionar .Almacenamiento con redundancia local (LRS)".
10. Dar clic en el botón Revisar".
11. Dar clic en el botón Çrear".

26.5. Crear un contenedor en una cuenta de almacenamiento

1. Seleccionar la cuenta de almacenamiento.
2. Seleccionar Çontenedores.en la sección .Almacenamiento de datos".
3. Agregar un contenedor seleccionando en la opción -Contenedor".
4. Ingresar el nombre del contenedor. El nombre solo puede contener letras minúsculas, números o guiones, debe iniciar con una letra o número, no puede contener guiones seguidos.
5. Dar clic en el botón Çrear".

26.6. Cargar archivos a un contenedor

1. Seleccionar la cuenta de almacenamiento.
2. Seleccionar Çontenedores".
3. Seleccionar el contenedor.
4. Seleccionar la opción Çargar".
5. Seleccionar "Buscar archivos".
6. Seleccionar el archivo en la ventana.
7. Dar clic en el botón Çargar".

Crear un token de acceso compartido (SAS: Shared Access Signature) para un contenedor

1. Seleccionar la cuenta de almacenamiento.
2. Seleccionar Çontenedores".
3. Seleccionar el contenedor.
4. Seleccionar "Tokens de acceso compartido.[en] la sección Çonfiguración".
5. Seleccionar método de firma Çlave de cuenta".
6. En el campo Çlave de firma"seleccionar la clave que se va a asociar a la SAS (clave 1 o clave 2).
7. En el campo "Permisos"seleccionar los permisos que se otorgarán.
8. Si se requiere restringir desde donde se accederá el blob, ingresar la IP o intervalo de IPs.
9. Dar clic en el botón "Generar URL y token de SAS".
10. Copiar la URL que aparece en el campo ÜRL de SAS de Blob".

La forma de una URL de contenedor es la siguiente:

```
https://<cuenta-de-almacenamiento>.blob.core.windows.net/<nombre-del-contenedor>?sp=r&st=
<fecha-UTC-inicial>&se=<fecha-UTC-final>&spr=https&sv=2022-11-02&sr=c&sig=<SAS>
```

Crear un token de acceso compartido (SAS: Shared Access Signature) para un blob

1. Seleccionar la cuenta de almacenamiento.
2. Seleccionar Çontenedores".
3. Seleccionar el contenedor.
4. En el menú contextual del blob (los tres puntos al final) seleccionar la opción "Generar SAS".
5. Seleccionar método de firma Çlave de cuenta".
6. En el campo Çlave de firma"seleccionar la clave que se va a asociar a la SAS (clave 1 o clave 2).
7. En el campo "Permisos"seleccionar los permisos que se otorgarán.
8. Si se requiere restringir desde donde se accederá el blob, ingresar la IP o intervalo de IPs.
9. Dar clic en el botón "Generar URL y token de SAS".
10. Copiar la URL que aparece en el campo ÜRL de SAS de Blob".

Nota. A partir de una URL de contenedor, es posible obtener una URL para acceder un blob en el contenedor, solo se agrega el nombre del blob (nombre del archivo) a la URL del contenedor:

https://<cuenta-de-almacenamiento>.blob.core.windows.net/<nombre-del-contenedor>/<nombre-del-blob>?sp=<permisos>&st=<fecha-UTC-inicial>&se=<fecha-UTC-final>&spr=https&sv=2022-11-02&sr=<tipo de SAS: c: contenedor, b: blob>&sig=<SAS>

26.7. Revocar una o varias SAS

Para revocar (invalidar) una o varias SAS, asociadas a una clave de acceso de una cuenta de almacenamiento (Clave 1 o clave 2), se deberá cambiar (girar) la clave:

1. Seleccionar la cuenta de almacenamiento.
2. Seleccionar la opción Çlaves de acceso".
3. Para cambiar una clave, dar clic en "Girar clave.[en] la clave 1 o la clave 2 (key1 o key2).

26.8. Notas

- Se puede tener varias SAS simultáneamente.
- Una vez que se ha girado la clave, se deberá generar nuevas SAS asociadas a la clave modificada.

El siguiente programa Java obtiene un blob utilizando una URL de contenedor, a la que se ha agregado el nombre del blob. Una vez leído el blob, se despliega la longitud del blob:

```
// Blob.java
// Generado por ChatGPT. 2024
// Modificado por Carlos Pineda G.

import java.net.*;
import java.io.*;

public class Blob {
  public static void main(String[] args) {
// URL del blob con SAS
String sasBlobUrl =
"https://<cuenta-de-almacenamiento>.blob.core.windows.net/<nombre-del-contenedor>/<nombre-del-blob>?s⌋
↪  p=<permisos>&st=<fecha-UTC-inicial>&se=<fecha-UTC-final>&spr=https&sv=2022-11-02&sr=<tipo de
↪  SAS>&sig=<SAS>";
try {
    // Crear una conexión HTTP GET a la URL del blob con SAS
    URL url = new URL(sasBlobUrl);
    HttpURLConnection connection = (HttpURLConnection) url.openConnection();
    connection.setRequestMethod("GET");
    // Obtener la respuesta
    int responseCode = connection.getResponseCode();
    // Verificar si la solicitud fue exitosa (código 200)
    if (responseCode == HttpURLConnection.HTTP_OK) {
        // Leer el contenido del blob
        InputStream inputStream = connection.getInputStream();
        ByteArrayOutputStream outputStream = new ByteArrayOutputStream();
        byte[] buffer = new byte[4096];
        int bytesRead;
        while ((bytesRead = inputStream.read(buffer)) != -1) {
            outputStream.write(buffer, 0, bytesRead);
```

```
        }
        // Convertir el contenido a un arreglo de bytes
        byte[] blobBytes = outputStream.toByteArray();
        // Mostrar la longitud del blob en bytes
        System.out.println("Longitud del blob: " + blobBytes.length + " bytes");
        // Cerrar los flujos de entrada y salida
        outputStream.close();
        inputStream.close();
    } else {
        System.out.println("Error al obtener el blob. Código de respuesta: " + responseCode);
    }
    // Cerrar la conexión
    connection.disconnect();
} catch (Exception e) {
    System.err.println("Error al obtener el blob: " + e.getMessage());
}
}
```

}

Capítulo 27

Actividades individuales a realizar

27.1. Descarga y carga de una imagen administrada

En esta actividad, vamos a descargar una imagen administrada de Azure, cargarla nuevamente y crear una máquina virtual con la imagen cargada.

Pre-requisitos:

- 30 GB en el disco duro de la computadora local (Linux).
- Azure CLI instalado en la computadora local (Linux).

27.1.1. Descargar la imagen

```
1.Acceder a Azure desde la computadora local:

az login

2. Obtener el ID de la imagen:

GRUPO_RECURSOS=<grupo-de-recursos>
IMAGEN=<nombre-de-la-imagen>
ID_IMAGEN=$(az image show -g "$GRUPO_RECURSOS" -n "$IMAGEN" --query id -o tsv)
MAQUINA_VIRTUAL=<nombre-de-la-máquina-virtual-temporal>
VHD=<nombre-de-la-máquina-virtual-temporal>.vhd

3. Crear una máquina virtual temporal a partir de la imagen:

az vm create -g "$GRUPO_RECURSOS" -n "$MAQUINA_VIRTUAL" --image "$ID_IMAGEN" --size Standard_B1s
↪  --public-ip-address "" --os-disk-delete-option Detach --admin-username <nombre-del-usuario>
↪  --generate-ssh-keys

4. Obtener el ID del disco y el nombre del disco de sistema operativo:

ID_DISCO=$(az vm show -g "$GRUPO_RECURSOS" -n "$MAQUINA_VIRTUAL" --query
↪  "storageProfile.osDisk.managedDisk.id" -o tsv)
NOMBRE_DISCO=$(basename "$ID_DISCO")

5. Eliminar la máquina virtual temporal (el disco de sistema operativo queda sin asignar):

az vm delete -g "$GRUPO_RECURSOS" -n "$MAQUINA_VIRTUAL" --yes
```

```
6. Generar una SAS para descargar el VHD con AzCopy:

SAS=$(az disk grant-access -g "$GRUPO_RECURSOS" -n "$NOMBRE_DISCO" --access-level Read
↪  --duration-in-seconds 43200 -o tsv)

O bien:

SAS=$(az disk grant-access --resource-group "$GRUPO_RECURSOS" --name "$NOMBRE_DISCO" --access-level
↪  Read --duration-in-seconds 7200 --query accessSas -o tsv)

7. Podemos ver la SAS generada:

echo "$SAS"

8. Instalar AzCopy en Ubuntu:

curl -sL https://aka.ms/downloadazcopy-v10-linux | tar -xz
sudo cp ./azcopy_linux_amd64_*/azcopy /usr/local/bin/

9. Descargar la imagen del disco (notar que se ejecuta con sudo):

sudo azcopy copy "$SAS" "$VHD"

10. Revocar la SAS:

az disk revoke-access -g "$GRUPO_RECURSOS" -n "$NOMBRE_DISCO"
```

1. En el portal de Azure eliminar todos los recursos asociados a la máquina virtual temporal.

27.1.2. Cargar la imagen

1.Acceder a Azure desde la computadora local (Linux):

```
az login

2. Crear una cuenta de almacenamiento

GRUPO_RECURSOS=<grupo-de-recursos>
REGION="<región>" # p.e. "eastus2"
CUENTA_ALMACENAMIENTO=<nombre-de-la-cuenta-de-almacenamiento>
CONTENEDOR=<nombre-del-contenedor>
VHD=<nombre-de-la-máquina-virtual-temporal>.vhd
DISCO=<nombre-del-disco>
IMAGEN=<nombre-de-la-imagen-restaurada>

az storage account create -g "$GRUPO_RECURSOS" -n "$CUENTA_ALMACENAMIENTO" -l "$REGION" --sku
↪  Standard_LRS

3. Crear un contenedor dentro de la cuenta de almacenamiento:

az storage container create --account-name "$CUENTA_ALMACENAMIENTO" --account-key
↪  "$CLAVE_CUENTA_ALMACENAMIENTO" -n "$CONTENEDOR"

4. Obtener la clave (KEY) de la cuenta de almacenamiento:
```

```
CLAVE_CUENTA_ALMACENAMIENTO=$(az storage account keys list -g "$GRUPO_RECURSOS" -n
↪  "$CUENTA_ALMACENAMIENTO" --query [0].value -o tsv)

5. Obtener la SAS para acceder al contenedor:

SAS=$(az storage container generate-sas --account-name "$CUENTA_ALMACENAMIENTO" -n "$CONTENEDOR"
↪  --permissions acwl --expiry 2099-01-01T00:00Z --account-key "$CLAVE_CUENTA_ALMACENAMIENTO" -o tsv)

6. Subir el VHD como blob al contenedor (ejecutar en el directorio donde está la imagen descargada):

azcopy copy "$VHD" "https://$CUENTA_ALMACENAMIENTO.blob.core.windows.net/$CONTENEDOR/$VHD?$SAS"

7. Crear el disco a partir del blob:

az disk create -g "$GRUPO_RECURSOS" -n "$DISCO" --source
↪  "https://$CUENTA_ALMACENAMIENTO.blob.core.windows.net/$CONTENEDOR/$VHD" --hyper-v-generation V2
↪  --os-type Linux -l "$REGION"

8. Crear la imagen a partir del disco:

ID_DISCO=$(az disk show -g "$GRUPO_RECURSOS" -n "$DISCO" --query id -o tsv)
az image create -g "$GRUPO_RECURSOS" -n "$IMAGEN" --os-type Linux --source "$ID_DISCO"
↪  --hyper-v-generation V2 -l "$REGION"

9. En el portal de Azure eliminar la cuenta de almacenamiento y el disco.

10. Para verificar la imagen restaurada, crear una máquina virtual a partir de la imagen.

Notar que al crear la máquina virtual a partir de la imagen restaurada se deberá ingresar el grupo de
↪  recursos, nombre de la máquina virtual, el tamaño, el tipo de autenticación, el tamaño del disco
↪  del SO, Tipo del disco del sistema operativo,
las opciones de disponibilidad (p.e. "No se requiere redundancia de la infraestructura"), Red virtual,
↪  Subred, IP pública, Diagnóstico de arranque.
```

La región, el tipo de seguridad, la arquitectura de VM serán los mismos que la máquina virtual original.

27.2. Notas

- El usuario de la nueva máquina virtual deberá ser el mismo de la máquina virtual original. Si se crea otro usuario y la imagen se creó conservando el usuario, entonces la máquina virtual tendrá dos usuarios, el usurio de la máquina virtual original y el usuario de la nueva máquina virtual.
- Es necesario crear las reglas de puerto que requieren los procesos que ejecutan en la máuinaq virtual (p.e. servidores).
- Si se accederá la máquina virtual utilizando un dominio, es necesario asocial el dominio a la dirección IP pública de la máquina virtual.

Interfaz de red

Una interfaz de red (NIC, Network Interface Card) permite a una máquina virtual conectarse a la red virtual.

Una máquina virtual puede tener una o más interfaces de red.

Cada interfaz de red tiene al menos una configuración IP.

Una configuración IP contiene una IP privada y opcionalmente una IP pública.

A una configuración IP se le puede asociar una IP pública, la cual podrá ser utilizada por la máquina virtual para conectarse a Internet.

Existen dos tipos de configuraciones IP: la "Principalτ las "Secundarias". Solo la IP pública de la configuración "Principal.^{es} visible a Internet. La configuración "Principal"no se puede eliminar.

Una interfaz de red se puede crear y posteriormente se puede agregar a una máquina virtual. Una IP pública también se puede crear y posteriormente se puede asociar a una configuración IP.

Puerta de enlace de red virtual (VPN Gateway)

Una puerta de enlace de red virtual (Virtual Network Gateway o VPN Gateway) es un Gateway implementado en software el cual permite conectar, mediante un canal encriptado, redes virtuales (VNet) entre sí o bien redes virtuales con redes on-premise.

Para conectar dos redes virtuales se requiere crear dos puertas de enlace de red virtual (Virtual Network Gateway), una para cada red virtual a interconectar.

A la conexión de dos redes virtuales se le conoce como VPN VNet-to-VNet.

Fuente: Configure a VNet-to-VNet VPN gateway connection by using the Azure portal

Por otra parte, para conectar una red on-premise con una red virtual (VNet) se requiere crear una puerta de enlace virtual para la red virtual y configurar un router en la red on-premise.

A la conexión de una red on-premise a una red virtual se le conoce como VPN Site-to-Site o nube híbrida.

Subred de puerta de enlace

Antes de crear un VPN Gateway es necesario crear una subred de Gateway (Gateway subnet), esta subred contiene las direcciones IP que utilizarán las máquinas virtuales y servicios que implementarán el VPN Gateway.

La subred de gateway debe llamarse "GatewaySubnet", este nombre le indica a Azure que se trata de la subred donde desplegará el VPN Gateway.

Ver: About VPN Gateway configuration settings

Emparejamiento de redes virtuales

El emparejamiento (peering) de redes virtuales permite conectar redes virtuales utilizando la red interna de Microsoft, de manera que el tráfico de emparejamiento no pasa por Internet.

No se cobra por crear el emparejamiento, pero sí se cobra el tráfico entrante y el tráfico saliente.

En cuanto a los costos, el tráfico en la misma región (intra-región) es más barato que el tráfico entre diferentes regiones.

El emparejamiento es más barato, más rápido y con mayor ancho de banda que VPN vnet-to-vnet.

Al igual que una VPN vnet-to-vnet, los espacios de direcciones IP de las redes virtuales a

emparejar no deben superponerse.

1. Crear una red virtual en una región, llamarla: vnet-1
2. Crear una red virtual en otra región, llamarla: vnet-2
3. Seleccionar la red virtual vnet-1.
4. Seleccionar la opción .Emparejamientos.en la sección Çonfiguración".
5. Dar clic en la opción -Agregar".
6. En la sección Resumen de la red virtual remota":

6.1 Ingresar el nombre del vínculo de emparejamiento remoto, llamarlo: emparejamiento-vnet-2 6.2 En el campo "Tipo de emparejamiento"seleccionar Red virtual". 6.3 En el campo Red virtual"seleccionar la red virtual vnet-2. 6.4 Habilitar "Permitir 'vnet-2' para acceder a 'vnet-1'".

1. En la sección Resumen de la red virtual local":

7.1 Ingresar el nombre del vínculo de emparejamiento local, llamarlo: emparejamiento-vnet-1 7.2 Habilitar "Permitir 'vnet-1' para acceder a 'vnet-2'".

1. Dar clic en el botón .Agregar".

Modelo de red de Azure

El siguiente diagrama de clases muestra la relación entre los diferentes componentes de una red virtual en Azure:

Creación de una red virtual en Azure

1. Ingresar al portal de Azure.

1. Ingresar Redes virtuales.en el campo "Buscar recursos, servicios y documentos".

1. Dar clic en el botón -Crear".

1. Seleccionar el grupo de recursos o crear uno nuevo.

1. Ingresar el nombre de la red virtual en el campo "Nombre de red virtual".

1. Seleccionar la región donde se creará la red virtual (por ejemplo: East US)

1. Seleccionar la opción "Direcciones IP".

Por default aparece un intervalo de direcciones con una subred "default".

1. Para agregar más intervalos de direcciones escribir cada intervalo en notación CIDR en el campo que aparece abajo del último intervalo definido.

1. Para agregar una subred dar clic en -Agregar una subred":

9.1 Ingresar el nombre de la subred.

9.2 Ingresar el intervalo de direcciones de la subred en formato CIDR, este intervalo debe estar dentro del espacio de direcciones.

1. Dar clic en el botón Revisar y crear".

1. Dar clic en el botón Crear.

1. Dar clic en el botón Ïr al recurso"para ver la red virtual creada.

Creación de un grupo de seguridad de red

1. En la página de inicio de Azure buscar "Grupos de seguridad de red".

1. Seleccionar -Crear".

1. En el campo "Grupo de recursos"seleccionar el grupo de recursos donde se creará el grupo de seguridad de red o Çrear nuevo".

1. Ingresar el nombre del grupo de seguridad.

1. Seleccionar la región. La región del NSG debe ser la misma que las subredes o interfaces de red (NIC) que se asociarán al NSG.

1. Dar clic en el botón Revisar y crear".

1. Dar clic en el botón Çrear".

1. Dar clic en el botón Ïr al recurso".

1. Para agregar reglas de seguridad de entrada o de salida seleccionar las opciones Reglas de seguridad de entrada.º Reglas de seguridad de salida.en el menú Çonfiguración.en la parte izquierda de la pantalla.

27.3. Agregar una subred a una red virtual

1. En la página de inicio buscar Redes virtuales"

1. Seleccionar la red virtual donde se creará la subred.

1. Seleccionar "Subredes.en el menú Çonfiguración"que aparece a la izquierda de la pantalla.

1. Seleccionar la opción -Subred".

1. Ingresar el nombre de la subred.

1. Ingresar el intervalo de direcciones IP de subred en notación CIDR.

1. Si se quiere asociar un grupo de seguridad de red (NSG) a la subred, seleccionarlo.

1. Dar clic en el botón "Guardar".

Entonces se puede ejecutar ifconfig en la máquina virtual para ver las interfaces y las direcciones IP privadas correspondientes.

27.4. Creación de una máquina virtual y su conexión a una red virtual existente

Cuando se crea una máquina virtual se puede conectar a una subred de una red virtual existente.

1. Seleccionar el grupo de recursos o crear uno nuevo.
1. Ingresar el nombre de la máquina virtual.
1. Seleccionar la misma región donde se creó la red virtual.
1. Seleccionar la imagen, por ejemplo Ubuntu Server 20.04 LTS - x64 gen.2
1. Seleccionar el tamaño, por ejemplo: Standard_B1s
1. En el campo Tipo de autenticación seleccionar Çontraseña".
1. Ingresar el nombre del usuario y la contraseña.
1. Seleccionar la opción "Discos"
1. Seleccionar el tipo de disco de sistema operativo:, por ejemplo: HDD estándar
1. Seleccionar la opción Redes".
1. Seleccionar la red virtual a la que se conectará la máquina virtual. Notar que sólo se puede seleccionar las redes virtuales que se encuentran en la misma región.
1. Seleccionar la subred a la que se conectará la máquina virtual.
1. Seleccionar una IP pública existente o "Ninguno"si no se requiere la IP pública. También se puede crear una nueva dirección IP pública seleccionando la opción Çrear".
1. Seleccionar la opción .Administración".
1. En el campo "Diagnóstico de arranque"seleccionar "Deshabilitar".
1. Dar clic en el botón Revisar y crear".
1. Dar clic en el botón Çrear".
1. Dar clic en el botón Ïr al recurso".

27.5. Creación de una interfaz de red

1. En la página de inicio de Azure buscar Ïnterfaces de red".
1. Seleccionar -Crear".
1. Ingresar el grupo de recursos.
1. Ingresar el nombre de la interfaz de red.

1. Seleccionar la región donde se creará la interface de red. La región deberá ser la misma donde se creó la red virtual a la que se conectará la interface.

1. Seleccionar la red virtual a la que se conectará la interface de red.

1. Seleccionar la subred a la que se conectará la interface de red.

1. En el campo .Asignación de la dirección IP privada"se puede seleccionar una IP dinámica o estática, si se selecciona IP estática se deberá ingresar una dirección IP estática.

1. Seleccionar el grupo de seguridad de red.

1. Dar clic en el botón Revisar y crear".

1. Dar clic en el botón Çrear".

1. Dar clic en el botón Ïr al recurso".

1. Si se quiere asociar un NSG a la interface seleccionar la opción "Grupo de seguridad de red", seleccionar el NSG y dar clic en la opción "Guardar".

27.6. Añadir una interface de red a una máquina virtual

Para añadir (asociar) una interface de red a una máquina virtual:

1. En la página de inicio de Azure buscar "Máquinas virtuales".

1. Seleccionar la máquina virtual.

1. Detener la máquina virtual.

1. Seleccionar la opción Redes.en el menú Çonfiguración"que aparece a la izquierda de la pantalla.

1. Seleccionar la opción .Adjuntar interfaz de red".

1. Seleccionar una interfaz de red previamente creada.

1. Dar clic en el botón .Aceptar".

1. En la pantalla Redes"se puede seleccionar la interface de red conectada.

27.7. Quitar una interface de red de una máquina virtual

Para quitar (desasociar) una interface de red de una máquina virtual:

1. En la página de inicio de Azure buscar "Máquinas virtuales".

1. Seleccionar la máquina virtual.

1. Detener la máquina virtual.

1. Seleccionar la opción Redes.[en] el menú Çonfiguración"que aparece a la izquierda de la pantalla.

1. Seleccionar la opción "Desasociar interfaz de red".

1. Seleccionar la interface de red a desconectar (desasociar).

27.8. Agregar una configuración IP a una interface de red

1. Buscar Ïnterfaces de red.[en] la barra e búsqueda en la pantalla de inicio de Azure.

1. Seleccionar la interfaz de red.

1. Seleccionar Çonfiguraciones de IP".

1. Dar clic en +.[A]gregar"

1. Ingresar el nombre de la configuración IP.

1. Seleccionar el tipo de asignación de la IP privada: dinámica o estática.

1. Seleccionar .[A]sociar"si se quiere asociar una IP pública a la configuración. Notar que solo la IP pública de la configuración "Principal.[es] visible a Internet.

Dando clic a los tres puntos que se encuentran a la derecha de una configuración se puede eliminar la configuración. La configuración "Principal"no se puede eliminar.

27.9. Agregar una IP pública a una máquina virtual

Para agregar una IP pública a una máquina virtual, es necesario asignar (asociar) la IP pública a la configuración IP principal de la interface de red de la máquina virtual.

1. En la página de inicio de Azure buscar "Máquinas virtuales".

1. Seleccionar la máquina virtual.

1. Seleccionar la opción Redes.[en] el menú Çonfiguración"que aparece a la izquierda de la pantalla.

1. Seleccionar la interfaz de red.

1. Seleccionar Çonfiguraciones de IP".

1. Seleccionar la configuración IP a la que se le asignará la IP pública.

1. Checar el campo .[A]sociar dirección IP pública"

1. En el campo "Dirección IP pública"seleccionar la IP pública o crear una nueva.

1. Dar clic al botón "Guardar".

1. Cerrar la ventana.

27.10. Quitar una IP pública a una máquina virtual

Para quitar la IP pública a una máquina virtual, es necesario desasociar la IP pública de la configuración IP principal de la interface de red de la máquina virtual.

1. En la página de inicio de Azure buscar ”Máquinas virtuales”.
1. Seleccionar la máquina virtual.
1. Seleccionar la opción Redes.[en] el menú Çonfiguración”que aparece a la izquierda de la pantalla.
1. Seleccionar la interfaz de red.
1. Seleccionar Çonfiguraciones de IP”.
1. Seleccionar la configuración IP a la que se le asignará la IP pública.
1. En el campo .[A]sociar dirección IP pública”quitar el checado.
1. Dar clic al botón ”Guardar”.
1. Cerrar la ventana.

27.11. Creación de una Puerta de enlace de red virtual (VPN Gateway)

1. Acceder al portal de Azure.
1. Seleccionar Çrear un recurso”.
1. Seleccionar ”Puerta de enlace de red virtual”.
1. Dar clic en el botón Çrear”.
1. Ingresar el nombre del gateway. Notar que el grupo de recursos es el mismo que el grupo de recursos donde está la red virtual.
1. Seleccionar la región.
1. En el campo ”Tipo de puerta de enlace”seleccionar ”VPN”.
1. Seleccionar el SKU ”VpnGw2AZ”.
1. EN el campo ”Generación”seleccionar ”Generation2”.
1. Seleccionar la red virtual a la que se conectará el gateway (notar que solo aparecen en la lista las redes virtuales creadas en la misma región).
1. En el campo Ïntervalo de direcciones de subred de puerta de enlace”(o el campo ”subred”) se muestra automáticamente un intervalo correspondiente a la ”GatewaySubnet”

1. Seleccionar Çrearüna dirección IP o Üsar existente". Si se va a crear una IP pública ingresar el nombre. Notar que el gateway requiere una dirección IP pública para recibir la conexión de otro gateway.

1. En el campo "Habilitar el modo activo/activo"seleccionar "Deshabilitado".

1. Dar clic en el botón Revisar y crear".

1. Dar clic en el botón Çrear".

Nota. "Setting up a virtual network is free of charge. However, we do charge for the VPN gateway that connects to on-premises and other virtual networks in Azure. This charge is based on the amount of time that gateway is provisioned and available".

Fuente: https://azure.microsoft.com/en-gb/pricing/details/vpn-gateway

27.12. Creación de una VPN VNet-to-VNet

1. Crear dos redes virtuales en dos regiones diferentes (p.e. East US y West US):

- Se deberá crear en cada red virtual una subred llamada "GatewaySubnet". Esta subred será utilizada por las máquinas virtuales que aprovisiona Azure para ejecutar el Gateway. Para crear la subred "GatewaySubnet.ªl momento de crear la red virtual:

- Seleccionar la opción .Agregar una subred.en la ventana "Direcciones IP".
- En el campo "Propósito de la subred"seleccionar "Virtual Network Gateway".
- En el campo "Dirección inicialïngresar la dirección IP inicial del intervalo de direcciones IP que define la subred.
- En el campo "Tamaño"seleccionar el tamaño del intervalo de direcciones IP de la subred.
- Dar clic en el botón .Agregar".

- Cada red virtual en un grupo de recursos diferente.
- Las dos redes virtuales no deberán tener direcciones IP en común (e.d. cada red virtual deberá tener sus espacios de direcciones diferentes).

1. Crear dos puertas de enlace de red virtual (Virtual Network Gateway), una para cada red virtual.

1. En el cuadro de búsqueda del portal de Azure buscar Redes virtuales".

1. Seleccionar la primera red virtual a conectar.

1. En el menú Çonfiguración"que aparece en la parte izquierda de la pantalla seleccionar "Dispositivos conectados"(en la lista aparecen los dispositivos conectados a la red virtual, p.e. interfaces de red virtual, Gateways, etc.).

1. Seleccionar el Gateway conectado a la red virtual.

1. Ahora vamos a crear una conexión de la primera red a la segunda red y una conexión de la segunda red a la primera red. En el menú Çonfiguración"que aparece en la parte izquierda de la pantalla seleccionar Çonexiones".

1. Seleccionar -Agregar"para agregar una nueva conexión.

1. Seleccionar el tipo de conexión: "De VNet a VNet".

1. Checar el campo .Establecer conectividad bidireccional"(debido a que las conexiones son unidireccionales, al indicar conectividad bidireccional se van a crear dos conexiones de manera que el tráfico pueda ir del primer Gateway al segundo Gateway y viceversa.

1. Ingresar el nombre de la primera conexión.

1. Ingresar el nombre de la segunda conexión.

1. Seleccionar la región donde se encuentra la primera red virtual a conectar (seleccionada en el paso 3).

1. Seleccionar la pestaña .Opciones".

1. Seleccionar el primer Gateway (la puerta de enlace virtual seleccionada en el paso 5).

1. Seleccionar el segundo Gateway (la puerta de enlace virtual que estará en el otro extremo de la VPN).

1. Ingresar una clave compartida para el encriptado de la conexión (letras y números).

1. Dar clic en el botón Revisar y crear".

1. Dar clic en el botón Çrear".

1. Para ver el estado de las conexiones, seleccionar ingresar "Todos los recursos.en el cuadro de búsqueda de recursos. Seleccionar la primera conexión (el nombre que se ingresó en el paso 11). El estado deberá ser Çonectado". De igual manera se puede verificar el estado de la segunda conexión.

En general una VPN no se encuentra en estado çonectado"mientras no inicie la transmisión de tráfico.

1. Ahora se puede crear una maquina virtual en la primera VNet y otra máquina virtual en la segunda VNet. Entonces se puede hacer "ping"de la primera máquina virtual a la segunda y viceversa utilizando las direcciones IP privadas de las máquinas virtuales.

Las máquinas virtuales no responden al "pingçuando se realiza a través de IP públicas.

Nota. Para eliminar los Gateways es necesario eliminar primero las conexiones.

Capítulo 28

OLAP - Online Analytical Processing

En el área de Inteligencia de Negocios o Business Intelligence (BI), un sistema de procesamiento analítico en línea (OLAP), es una tecnología que permite realizar análisis multidimensional de grandes volúmenes de datos de manera rápida y eficiente.

Los datos utilizados para el análisis mutidimensional, se almacenan en un almacén de datos empresarial (data warehouse), o un almacén de datos departamental (data mart).

28.1. Data warehouse

Un data warehouse es un almacén de datos con una estructura multidimensional, ya que las métricas (indicadores) de negocio, como serían las ventas, se guardan en tablas llamadas tablas de hechos (fact tables) relacionadas con diversas dimensiones, como serían la geografía, el tiempo, los productos, los clientes, los empleados, entre otras.

Las dimensiones pueden dividirse en diferentes niveles jerárquicos, por ejemplo, una dimensión geográfica puede tener los siguientes niveles jerárquicos: colonia, municipio, estado, país y región. Una dimensión temporal podría tener los siguientes niveles jerárquicos: fecha, día de la semana, mes, trimestre, semestre y año.

Una dimensión de producto podría tener los niveles jerárquicos: producto y categoría.

Los datos en un data warehouse se pueden estructurar en dos tipos de modelos E-R: estrella (star) y copo de nieve (snowflake).

28.1.1. Modelo estrella

En el esquema estrella la fact table se enlaza con cada tabla de dimensión.

La fact table contiene las métricas (p.e. las ventas), y las llaves foráneas que enlazan la fact table con las tablas de dimensiones.

Cada tabla de dimensión contiene una llave primaria y una descripción.

Por ejemplo, la tabla correspondiente a la dimensión ”producto”, incluye la llave primaria del producto y el nombre del producto.

La tabla correspondiente a la dimensión çategoría”(categoría de producto), incluye la llave primaria de la categoría y el nombre de la categoría.

En este caso, "productoτ çategoría"son dos niveles jerárquicos de una misma dimensión.

En un esquema estrella todas las tablas de dimensiones se enlazan con la fact table.

Consideremos el siguiente esquema estrella:

En este caso, la fact table contiene los registros correspondientes a las ventas.

Las tablas employee, category, product, order_date, customer y country se enlazan directamente a la tabla fact table.

28.2. Modelo copo de nieve

En el esquema copo de nieve (snowflake), la fact table se enlaza con las tablas correspondientes al primer nivel jerárquico de cada dimensión.

Al igual que en el modelo estrella, en el modelo copo de nieve la fact table contiene las métricas (p.e. las ventas).

Las llaves foráneas en la fact table, enlazan la fact table con las tablas correspondientes al primer nivel jerárquico de cada dimensión.

Si una dimensión tiene varios niveles jerárquicos, cada nivel jerárquico será una tabla de dimensión incluyendo una llave primaria y descripción, y una llave foránea que enlaza la tabla con la tabla correspondiente al nivel jerárquico superior, si lo hay.

Consideremos el siguiente esquema copo de nieve:

En este caso, la fact table contiene los registros correspondientes a las ventas.

Las tablas employee, product, order_date, customer y country se enlazan directamente a la tabla fact table.

La tabla product se enlaza con la tabla category, debido a que product y category son dos niveles jerárquicos de la dimensión "producto".

Una ventaja del modelo copo de nieve comparado con el modelo estrella, es que en el modelo copo de nieve los datos ocupan menos espacio en disco, sin embargo, las consultas en éste modelo requieren enlazar más tablas, lo que hace las consultas mucho más lentas.

En realidad, el espacio en disco que ahorra el modelo copo de nieve es poco, debido a que solo se ahorra el espacio ocupado por llaves foráneas en la fact table; por otra parte, las consultas son más rápidas en el modelo estrella.

Desde luego, un modelo normalizado, como es el caso del modelo copo de nieve, garantiza una mayor calidad en los datos. Considerando que los datos de la base de dato multidimensional son extraídos de una base de datos transaccional, la calidad de los datos la debería garantizar la base de datos transaccional, no la base de datos multidimensional.

28.3. ROLAP

Los modelos estrella y copo de nieve anteriores son modelos Entidad-Relación para una base de datos relacional.

A un sistema de procesamiento analítico en línea sobre una base de datos relacional, se conoce como ROLAP o Relational Online Analytical Processing.

Los sistemas ROLAP tiene las siguientes ventajas:

Permiten a las empresas aprovechar su DBMS relacional, sin la necesidad de una inversión adicional en otro tipo de tecnología OLAP. Escalan a grandes volúmenes de datos. El modelado de datos es flexible, a medida que se requieren más métricas, dimensiones, o niveles jerárquicos en las dimensiones, se pueden agregar sin un impacto mayor en el modelo existente. El desempeño se puede aumentar creando tablas agregadas. SQL permite realizar consultas complejas de acuerdo a las necesidades de la empresa.

28.4. Cubo OLAP

Un cubo OLAP es el resultado de una consulta a la base de datos mutidimensional. En un sistema ROLAP, un cubo es el resultado de ejecutar una instrucción SELECT.

Por ejemplo, si consideramos el modelo copo de nieve que vimos anteriormente, podemos obtener un cubo OLAP que incluya las ventas acumuladas para las dimensiones country, category y customer:

Entonces, el cubo OLAP se puede obtener ejecutando la siguiente instrucción SELECT:

```
SELECT a.country,b.category,c.customer,sum(e.sales)
FROM country a,category b,customer c,product d,fact_table e
WHERE e.id_country=a.id_country and e.id_product=d.id_product
and d.id_category=b.id_category and e.id_customer=c.id_customer
GROUP BY 1,2,3
ORDER BY 1,2,3;
```

Nota. En las versiones recientes de Excel, a los cubos de datos se les llama tablas dinámicas.

Capítulo 29

Azure Files

Azure Files es un servicio administrado de archivos compartidos en la nube, los cuales se pueden acceder mediante el protocolo SMB (Server Message Block).

Para utilizar Azure Files, se debe crear una cuenta de almacenamiento, y dentro de ésta, se podrán crear recursos compartidos de archivos; para cada recurso compartido de archivos se podrá configurar una capacidad entre 100 GB y 100 TB.

Los recursos compartidos de archivos funcionan como filesystems sin servidor (serverless), y pueden montarse simultáneamente en Linux, Windows y MacOS.

Debido a que Azure Files no soporta el encriptado sobre el protocolo NFS, la seguridad en la transferencia se basa en la seguridad de la red virtual, por tanto, NFS solo puede utilizarse dentro de una red virtual de Azure.

SMB puede utilizarse fuera de la red virtual.

Para acceder los recursos compartidos de archivos, además de los protocolos SMB y NFS, también es posible utilizar APIs para .NET, Java, Python, JavaScript, C++, y Go, o bien, la API REST de Azure Files.

Reproducir Vídeo

Recurso compartido de archivos mediante SMB

Crear una cuenta de almacenamiento

1. Ingresar al portal de Azure.
2. En el campo de búsqueda escribir Çuentas de almacenamiento".
3. Seleccionar -Crear".
4. Seleccionar un grupo de recursos o crear uno nuevo.
5. Ingresar en nombre de la cuenta de almacenamiento. El nombre debe ser único en Azure, de 3 a 24 caracteres (letras minúsculas y dígitos).
6. Seleccionar la región.
7. En el campo "Servicio principal"seleccionar .ªzure Files".
8. En el campo Rendimiento"seleccionar "Prémium". Esta selección no se puede cambiar después de crear la cuenta de almacenamiento.
9. En el campo Redundancia"seleccionar .ªlmacenamiento con redundancia local (LRS)".
10. Dar clic en el botón Revisar y crear".

11. Dar clic en el botón Çrear".

Agregar un recurso compartido de archivos

1. Seleccionar la cuenta de almacenamiento.
2. Seleccionar la opción Recursos compartidos de archivos".
3. Seleccionar -Recurso compartido de archivos".
4. Ingresar el nombre del recurso compartido de archivos.
5. Ingresar la capacidad aprovisionada (entre 100 y 102400 GB).
6. En la sección "Backup"se puede deshabilitar la opción "Habilitar copia de respaldo".
7. Dar clic en el botón Revisar+crear".
8. Dar clic en el botón Çrear".

Crear una conexión al recurso compartido de archivos

1. Seleccionar la cuenta de almacenamiento.
2. Seleccionar la opción Recursos compartidos de archivos".
3. Seleccionar el recurso compartido de archivos.
4. Dar clic en la opción Çonectar".
5. Seleccionar "Linux".
6. Dar clic en el botón "Mostrar script".
7. Copiar el script.

Montar el recurso compartido de archivos en una máquina virtual con Ubuntu

1. Crear una máquina virtual con Ubuntu.
2. Ejecutar el script obtenido anteriormente.

Notar que el script crea un directorio el cual será el punto de montaje, p.e. /media/<recurso-compartido-de-archivos>

1. Para ver el archivo en Azure:

3.1 Seleccionar la cuenta de almacenamiento. 3.2 Seleccionar la opción Recursos compartidos de archivos". 3.3 Seleccionar el recurso compartido de archivos. 3.4 Seleccionar la opción .Examinar".

1. Puede cargar un archivo directamente en Azure:

4.1 Seleccionar la cuenta de almacenamiento. 4.2 Seleccionar la opción Recursos compartidos de archivos". 4.3 Seleccionar el recurso compartido de archivos. 4.4 Seleccionar la opción .Examinar". 4.5 Seleccionar Çargar". 4.6 Seleccionar "Buscar archivos". 4.7 Seleccionar el archivo en la ventana. 4.8 Dar clic en el botón Çargar". 4.9 En la máquina virtual se puede ver que el archivo se encuentra en el directorio punto de montaje /media/<recurso-compartido-de-archivos>

Donde <recurso-compartido-de-archivos>es el nombre del recurso compartido de archivos.

Notas.

- Antes de ejecutar el script es recomendable borrar las credenciales anteriores (si ya se había

montado un recurso compartido de archivos previamente): /etc/smbcredentials/<cuenta-de-almacenamiento>.cred

- Para desmontar el recurso compartido de archivos se deberá ejecutar: sudo umount /media/<recurso-compartido-de-archivos>
- Para volver a montar el recurso compartido de archivos es necesario ejecutar nuevamente el script (comentando la línea donde se crea el directorio).

Capítulo 30

El sistema de archivos distribuidos NFS

El sistema de archivos NFS (Network File System) permite que una computadora (cliente) tenga acceso de manera transparente a los archivos contenidos en un servidor remoto.

Utilizando NFS, la computadora cliente puede montar un sistema de archivos remoto, por tanto, es posible desde el cliente, crear, borrar, leer y escribir archivos que se encuentran "físicamente.[en] el servidor.

NFS se implementa mediante una arquitectura cliente-servidor.

Cuando la computadora cliente monta el sistema de archivos remoto, el cliente NFS se conecta al servidor NFS, entonces el cliente podrá acceder a los archivos en el sistema de archivos remoto, utilizando el protocolo NFS a través del puerto default 2049.

30.1. Instalación de NFS en el servidor

1. Para instalar NFS en el servidor se ejecutan los siguientes comandos:

```
sudo apt update
sudo apt install nfs-kernel-server
```

1. Crear el directorio (punto de montaje) que va a compartir el servidor con los clientes:

```
sudo mkdir /var/nfs -p
```

La opción -p indica que se deberán crear todos los directorios en la ruta si no existen.

1. El propietario del directorio /var/nfs es root debido a que este directorio se creó con sudo. Podemos ver el propietario del directorio /var/nfs ejecutando el comando:

```
ls -l /var
```

1. Debido a que NFS convierte el acceso del usuario root en el cliente en un acceso con el usuario "nobody:nogroup.[en] el servidor, es necesario cambiar el propietario y permisos del directorio creado anteriormente:

```
sudo chown nobody:nogroup /var/nfs
sudo chmod 777 /var/nfs
```

1. Podemos verificar el nuevo propietario:

```
ls -l /var
```

1. Ahora se debe registrar el directorio creado en el archivo de configuración de NFS.

6.1 Editar el archivo /etc/exports:

```
sudo vi /etc/exports
```

6.2 Agregar la siguiente línea, guardar y salir del editor (en este caso 40.76.45.28 es la IP del cliente):

```
/var/nfs 40.76.45.28(insecure,rw,sync,no_subtree_check)
```

La opción ïnsecure"permite que NFS use un puerto ïnseguro", es decir, un puerto mayor o igual a 1024, ya que los puertos por debajo de 1024 solo pueden ser abiertos con permisos de administrador. En este caso usaremos el puerto 2049 el cual es el puerto estándar para NFS.

30.1.1. Notas de seguridad.

- Debido a que los atacantes saben que el puerto 2049 es el puerto utilizado para NFS, es recomendable utilizar otro puerto.
- Se recomienda utilizar la versión 4 de NFS la cual implementa el protocolo de autenticación Kerberos (autenticación mutua entre el cliente y el servidor) así como el encriptado mediante TLS o IPsec, sin embargo, la versión 4 puede resultar más lenta que la versión 3.
- Es importante configurar el NSG del servidor para que solo acepte conexiones desde las direcciones IP de clientes de confianza.

La opción rw"permite la lectura y escritura.

La opción "sync"hace que las modificaciones realizadas por el cliente sobre los archivos se escriban en el disco del servidor. Por otra parte, la opción .ªsync"no escribe los cambios en el disco, lo que puede ocasionar que los cambios se pierdan en caso de una caída del servidor.

```
La opción "no_subtree_check" deshabilita la verificación de que el archivo que accede el cliente esté
↪  en el árbol de directorios exportado. Esta verificación causa problemas si el archivo es
↪  renombrado cuando el cliente lo tiene abierto. En general la opción "subtree_check" se utiliza
↪  para sistemas de archivos donde no se renombran los archivos y la opción "no_subtree_check" se
↪  utiliza para sistemas de archivos donde se realizará renombramiento de archivos.
```

Para una descripción de los permisos se puede consultar Understanding the /etc/exports File y exports linux page.

6.3 Actualizar la tabla de file systems exportados por NFS:

```
sudo exportfs -ra
```

6.4 Para ver los file systems exportados por NFS:

```
sudo exportfs
```

6.5 Para activar la nueva configuración, se requiere reiniciar el servidor NFS:

```
sudo systemctl restart nfs-kernel-server
```

1. Ahora debemos abrir el puerto de entrada 2049 utilizando el portal de Azure:

```
Intervalos de puertos de destino: 2049
Protocolo: TCP
Nombre: puerto_nfs
```

30.2. Instalación de NFS en el cliente

1. Para instalar NFS en el cliente se ejecutan los siguientes comandos:

```
sudo apt update
sudo apt install nfs-common
```

1. Crear el directorio (punto de montaje) en el cliente. el cual será .ªpejo"del punto de montaje del servidor:

```
sudo mkdir -p /nfs
```

1. Montar el directorio remoto /var/nfs (en este caso 40.87.94.140 es la IP del servidor):

```
sudo mount -v -t nfs 40.87.94.140:/var/nfs /nfs
```

1. Para desmontar el directorio /nfs:

```
sudo umount /nfs
```

30.3. Probar el acceso a archivos remotos

1. En el servidor crear un archivo, en este caso utilizamos el editor vi:

```
vi /var/nfs/texto.txt
```

1. Escribir un texto (como el siguiente) guardar y salir de la edición:

```
Esta es una prueba
```

1. En el cliente editar el archivo /nfs/texto.txt:

```
vi /nfs/texto.txt
```

1. Agregar la siguiente línea, guardar y salir de la edición:

```
Esta es otra prueba
```

1. En el servidor desplegar el contenido del archivo /var/nfs/texto.txt:

```
more /var/nfs/texto.txt
```

1. En el cliente desplegar el contenido del directorio /nfs:

```
ls -l /nfs
```

30.4. Referencia

Sistemas de archivos de red

Capítulo 31

NFS sobre un túnel SSH

Las versiones de NFS anteriores a la versión 4 no utilizan un canal seguro para autenticar y encriptar la comunicación, por esta razón cuando se utiliza NFS fuera de una red segura es necesario implementarlo sobre un túnel SSH.

Cuando habilitamos el túnel SSH para comunicar el cliente NFS con el servidor NFS, el cliente NFS se conecta localmente al cliente SSH (el cliente SSH hace las veces de servidor de NFS); entonces el cliente SSH envía los datos al servidor SSH, el cual a su vez redirige los datos al servidor de NFS (el servidor SSH hace las veces de cliente del servidor de NFS).

Debido a que el túnel SSH permite mapear los puertos, el cliente NFS se puede conectar al cliente SSH a través de un puerto diferente, al puerto que utiliza el servidor SSH para conectarse al servidor NFS. El mapeo de puertos es de utilidad cuando el puerto estándar 2049 está ocupado.

En la siguiente arquitectura podemos ver que el cliente NFS se conecta al cliente SSH, a través del puerto 3049, mientras que el servidor SSH se conecta al servidor NFS en localhost, a través del puerto 2049:

31.1. Instalación de NFS seguro en el servidor

1. Para instalar NFS en el servidor se ejecutan los siguientes comandos:

```
sudo apt update
sudo apt install nfs-kernel-server
sudo apt install portmap
```

1. Crear el directorio compartido en el servidor;

```
sudo mkdir /var/nfs -p
```

1. El propietario del directorio /var/nfs es root debido a que este directorio se creó con sudo. Podemos ver el propietario del directorio /var/nfs ejecutando el comando:

```
ls -l /var
```

1. Debido a que NFS convierte el acceso del usuario root en el cliente en un acceso con el usuario "nobody:nogroup.[en] el servidor, es necesario cambiar el propietario y permisos del directorio creado anteriormente:

```
sudo chown nobody:nogroup /var/nfs
sudo chmod 777 /var/nfs
```

1. Podemos verificar el nuevo propietario:

```
ls -l /var
```

1. Ahora se debe registrar el directorio creado en el archivo de configuración de NFS.

6.1 Editar el archivo /etc/exports:

```
sudo vi /etc/exports
```

6.2 Agregar la siguiente línea, guardar y salir del editor:

```
/var/nfs localhost(insecure,rw,sync,no_subtree_check)
```

Notar que el servidor aceptará conexiones del servidor SSH el cual actúa como cliente de NFS.

6.3 Actualizar la tabla de file systems exportados por NFS:

```
sudo exportfs -ra
```

6.4 Para ver los file systems exportados por NFS:

```
sudo exportfs
```

6.5 Para activar la nueva configuración, se requiere reiniciar el servidor NFS:

```
sudo systemctl restart nfs-kernel-server
```

Notar que en este caso no se abrirá el puerto de entrada 2049 en el servidor.

Nota de seguridad. Es recomendable configurar el NSG del servidor para que solo acepte conexiones SSH desde las direcciones IP de clientes de confianza.

31.2. Instalación de NFS seguro en el cliente

1. Para instalar NFS en el cliente se ejecutan los siguientes comandos:

```
sudo apt update
sudo apt install nfs-common
sudo apt install portmap
```

1. Crear el directorio de montaje en el cliente (punto de montaje):

```
sudo mkdir -p /nfs
```

1. Para crear un túnel SSH entre cliente y el servidor se ejecuta el siguiente comando:

```
ssh -fNv -L 3049:localhost:2049 ubuntu@40.84.237.35
```

En este caso, a manera de ejemplo, estamos mapeando el puerto 3049 al puerto 2049, es decir, el cliente NFS se conectará al cliente SSH (el cual hace las veces de servidor NFS) a través del puerto 3049, y el servidor SSH (el cual hace las veces de cliente NFS) se conectará al servidor NFS a través del puerto 2049.

```
Aquí "ubuntu" es un usuario en el servidor y "40.84.237.35" es la dirección IP del servidor.
```

1. Montar el directorio remoto:

```
sudo mount -t nfs -o port=3049 localhost:/var/nfs /nfs
```

Debido a que el puente SSH mapeará el puerto 3049 al puerto 2049, es necesario indicar el puerto al que se conectará el cliente NFS, en este caso el puerto 3049.

1. Para desmontar el directorio /nfs:

```
sudo umount /nfs
```

Nota. El uso de NFS sobre túnel SSH está indicado cuando la comunicación entre el cliente NFS y el servidor NFS cruza Internet, sin embargo, cuando el cliente NFS y el servidor NFS se encuentran ambos en una red segura (VNet), no es necesario utilizar el túnel SSH, toda vez que el encriptado incrementa la latencia en la comunicación.

Capítulo 32

Clases de direcciones IP v4

Las direcciones IP versión 4 se dividen en cinco clases o rangos, a saber:

- Clase A
- Clase B
- Clase C
- Clase D
- Clase E

Cada clase se define por un rango de valores que puede tomar el primer byte de la dirección IP, así las clases A, B y C son utilizadas para la comunicación unicast, mientras que la clase D es utilizada exclusivamente para la comunicación multicast.

La clase E está reservada para propósitos experimentales.

La siguiente tabla muestra los bytes que identifican a las redes y a los hosts en cada clase (Rango del primer byte), así como la máscara de subred, número de redes y número de hosts por red en cada clase.

Clase	Rango del primer byte	Identificador de red y host	Máscara de subred	Número de subredes	Número de hosts por subred
A	1 - 126	R.H.H.H	255.0.0.0	126	16,777,214
B	128 - 191	R.R.H.H	255.255.0.0	16,382	65,534
C	192 - 223	R.R.R.H	255.255.255.0	2,097,150	254
D	224 – 239	—	—	—	—
E	240 – 255	—	—	—	—

Referencia: Explicación de las cantidades de hosts y subredes (Cisco)

Las direcciones 127.X.X.X (loopback address) son utilizadas para identificar a la computadora local (localhost).

La dirección 255.255.255.255 es usada para broadcast a todos los hosts en la LAN. Las direcciones 224.0.0.1 y 224.0.0.255 están reservadas.

Las direcciones de clase A son utilizadas por algunos gobiernos y por proveedores de nube (p.e. Microsoft Azure) ¿Qué dirección IP está asociada al dominio m4gm.com?

Las direcciones de clase B son utilizadas por empresas e instituciones públicas de tamaño medio (como sería el caso del IPN) las cuales podrían requerir más de 254 direcciones públicas.

Las direcciones de clase C son usadas por empresas e instituciones públicas que no requieren muchas direcciones IP públicas.

La clase D a su vez, se divide en los siguientes intervalos de direcciones IP de acuerdo a su uso:

Dirección inicial	Dirección final	Uso
224.0.0.0	224.0.0.255	Direcciones reservadas para control de tráfico en una red local (p.e. enrutamiento)
224.0.1.0	224.0.1.255	Direcciones para multicast global, se enrutan a través de Internet
224.0.2.0	224.0.255.255	Direcciones reservadas para uso futuro
224.1.0.0	238.255.255.255	Direcciones disponibles para aplicaciones multicast
239.0.0.0	239.255.255.255	Direcciones para multicast local; no se enrutan en Internet

Referencia: RFC5771: IANA Guidelines for IPv4 Multicast Address Assignments

32.1. Classless Inter-Domain Routing (CIDR)

Como vimos anteriormente, las direcciones IP se agrupan en cinco clases, sin embargo, esta clasificación tiene un inconveniente, que muchas direcciones IP se desperdician debido a que solo se pueden asignar redes con tres tamaños fijos (clases A, B y C).

Así mismo, la clasificación de IPs por clase requiere que los ruteadores almacenen tablas de enrutamiento muy grandes, ya que se debe almacenar direcciones IP individuales y direcciones IP de subredes, lo que necesita gran cantidad de espacio de almacenamiento y por tanto el incremento en el costo de éstos dispositivos.

Como una alternativa a la clasificación de direcciones IP por clases, se desarrolló el concepto de Enrutamiento de dominios sin clases o CIDR (Classless Inter-Domain Routing).

La notación CIDR es una notación que agrega un número llamado longitud del prefijo a la dirección IPv4, el cual indica el número de unos que forman la máscara de subred a la que pertenece la IP.

Por ejemplo: 192.54.20.10/24 indica que la IP es 192.54.20.10 y la máscara de subred es 255.255.255.0, ya que 25510=111111112 es decir ocho unos binarios, por tanto, una longitud del prefijo igual a 24 indica que la máscara de subred contiene 24 unos (contados de izquierda a derecha).

Otro ejemplo es el siguiente: 192.54.20.10/20, en este caso la máscara de subred correspondiente es: 255.255.240.0 ya que 25510=111111112 y 24010 = 111100002, es decir, 8+8+4= 20 unos.

De esta manera, es posible saber si dos o más IPs en formato CIDR pertenecen a la misma subred, solo es necesario verificar que las IPs coinciden en los N primeros bits, donde N es la longitud del prefijo que sucede a la diagonal /".

La notación CIDR permite implementar redes con una variedad en el número de hosts, a esta propiedad se le llama máscara de subred de longitud variable (VLSM: Variable Length Subnet Mask).

Por ejemplo, supongamos que tenemos dos computadoras con las siguientes IP:

192.30.50.20/22 192.30.48.10/22

¿Estas computadoras pertenecen a la misma subred?

Primero convertimos cada IP a binario. Entonces verificamos que los primeros 22 dígitos (de izquierda a derecha) coincidan.

192.030.050.020 = 11000000 00011110 00110010 00010100 192.030.048.010 = 11000000 00011110 00110000 00001010 255.255.252.000 = 11111111 11111111 11111100 00000000 (22 unos)

En efecto, los primeros 22 dígitos binarios coinciden en ambas IPs, por tanto, las dos computadoras pertenecen a la misma subred.

32.2. Espacios de direcciones IP

La notación CIDR permite definir espacios de direcciones IP en una red.

Un espacio de direcciones es un intervalo de direcciones IP definido en notación CIDR.

Por ejemplo, 10.2.0.0/16 indica el intervalo de direcciones IP entre 10.2.0.0 y 10.2.255.255, un total de 65,536 direcciones IP.

El número total de direcciones IP que contiene un espacio de direcciones se puede calcular utilizando la longitud del prefijo CIDR, en el caso anterior el número de direcciones se calcula de la siguiente manera: 2(32-16)=216=65536.

Por otra parte la longitud del prefijo /32 define un espacio de direcciones IP con un número de direcciones IP igual a: 2(32-32)=20=1

32.3. Direcciones IP utilizables y no utilizables

La siguiente tabla muestra el número total de IPs para cada longitud del prefijo, el número de IPs utilizables, las IPs no utilizables y la máscara de subred.

Las direcciones IPs no utilizables son la dirección IP de la subred (la primera dirección IP del intervalo de direcciones IP) y la dirección IP de broadcast (la última dirección IP del intervalo de direcciones IP); la dirección de broadcast se utiliza para enviar tráfico a todos los dispositivos dentro de la subred.

Longitud del prefijo	Número total de IPs	IPs utilizables	IPs no utilizables	Máscara de subred
/32	1	1	Ninguna	255.255.255.255
/31	2	0	Todas (Solo para punto a punto)	255.255.255.254
/30	4	2	Red, Broadcast	255.255.255.252
/29	8	6	Red, Broadcast	255.255.255.248
/28	16	14	Red, Broadcast	255.255.255.240
/27	32	30	Red, Broadcast	255.255.255.224
/26	64	62	Red, Broadcast	255.255.255.192
/25	128	126	Red, Broadcast	255.255.255.128
/24	256	254	Red, Broadcast	255.255.255.0
/23	512	510	Red, Broadcast	255.255.254.0
/22	1024	1022	Red, Broadcast	255.255.252.0
/21	2048	2046	Red, Broadcast	255.255.248.0
/20	4096	4094	Red, Broadcast	255.255.240.0
/19	8192	8190	Red, Broadcast	255.255.224.0
/18	16384	16382	Red, Broadcast	255.255.192.0
/17	32768	32766	Red, Broadcast	255.255.128.0
/16	65536	65534	Red, Broadcast	255.255.0.0
/15	131072	131070	Red, Broadcast	255.254.0.0
/14	262144	262142	Red, Broadcast	255.252.0.0
/13	524288	524286	Red, Broadcast	255.248.0.0
/12	1048576	1048574	Red, Broadcast	255.240.0.0
/11	2097152	2097150	Red, Broadcast	255.224.0.0
/10	4194304	4194302	Red, Broadcast	255.192.0.0
/9	8388608	8388606	Red, Broadcast	255.128.0.0
/8	16777216	16777214	Red, Broadcast	255.0.0.0

Referencia: Classless Inter-domain Routing (CIDR): The Internet Address Assignment and Aggregation Plan

Capítulo 33

Redes virtuales en Azure

Cuando se crea una máquina virtual en Azure se crea por default:

- Una red virtual (VNet).
- Una subred.
- Una interfaz de red (NIC, Network Interface Card) incluyendo una IP privada.
- Un grupo de seguridad de red (NSG, Network Security Group).
- Una IP pública.
- Un disco de sistema operativo.

33.1. Grupo de recursos

Un grupo de recursos es similar a un directorio de sistema operativo.

Un grupo de recursos puede contener diferentes recursos de Azure.

Cuando se elimina un grupo de recursos también se eliminan los recursos que contiene el grupo de recursos.

33.2. Red virtual (VNet)

Una red virtual es una red privada en Azure.

Las computadoras conectadas a una red virtual pueden comunicarse en forma segura utilizando sus direcciones IP privadas debido a que las direcciones IP privadas no son visibles desde Internet.

Una red virtual está compuesta por uno o más espacios de direcciones IPv4.

Un espacio de direcciones es un intervalo de direcciones IP definido en notación CIDR.

Los intervalos de direcciones IP en una red virtual no se superponen, es decir, los intervalos no tienen direcciones IP en común.

Por ejemplo, 10.2.0.0/16 indica el intervalo 10.2.0.0 a 10.2.255.255, un total de 65,536 direcciones.

El tamaño máximo del espacio de direcciones de una red virtual es /8 (en notación CIDR, 224=16,777,216 direcciones IP).

Ver: Agregar o quitar un rango de direcciones.

33.3. Subred

Una red virtual puede tener una o varias subredes.

Cada subred se define como un intervalo de direcciones IP dentro de la red virtual.

Cuando se crea una red virtual, se crea por default una subred llamada “default”. Esta subred puede modificarse o eliminarse.

El tamaño mínimo de una subred es /29 (23=8 direcciones IP) y el máximo /8 (224=16,777,216 direcciones IP).

Dos computadoras conectadas a diferentes subredes dentro de una misma red virtual se pueden comunicar.

33.4. Grupo de seguridad de red

Un grupo de seguridad de red (NSG, Network Security Group) es un conjunto de reglas de entrada y reglas de salida para los puertos.

Las reglas de puerto de entrada controlan las conexiones TCP y paquetes UDP que vienen del exterior de la máquina virtual, mientras que las reglas de puerto de salida controlan las conexiones TCP y paquetes UDO que salen de la máquina virtual.

Notar que una vez que se establece la conexión TCP, la comunicación es bidireccional, es decir, la computadora que se conecta (cliente) puede enviar y recibir datos de la computadora a la que se conecta (servidor),

Cada regla de entrada permite/deniega conexiones TCP o paquetes UDP desde:

- Una dirección IP de origen específica.
- Un intervalo de direcciones IP de origen.
- Cualquier dirección IP de origen.
- Un intervalo de puertos de entrada.
- Un puerto de entrada específico.
- Cualquier puerto de entrada.

Cada regla de salida permite/deniega conexiones TCP o paquetes UDP hacia:

- Una dirección IP de destino específica.
- Un intervalo de direcciones IP de destino.
- Cualquier dirección IP de destino.
- Un intervalo de puertos de destino,.
- Un puerto de destino específico.
- Cualquier puerto de destino.

También se puede configurar si se permite o deniega un protocolo específico (TCP, UDP, ICMP o cualquiera de los anteriores) o si se permite o deniega cualquier protocolo.

A cada regla del grupo de seguridad de red se le asigna una prioridad, de tal manera que las reglas se aplican de menor a mayor prioridad, es decir, las reglas con menor prioridad tienen mayor precedencia.

Por ejemplo, si una regla con menor prioridad permite el tráfico y otra regla con mayor prioridad deniega el mismo tráfico, entonces se aplicará la primera regla y se ignorará la segunda.

Un grupo de seguridad de red puede asociarse a nivel subred y a nivel interfaz de red.

Un mismo grupo de seguridad de red puede asociarse a varias subredes y varias interfaces de red.

Por default todo el tráfico que viene de un balanceador de carga o de la red virtual es permitido, y todo el tráfico que viene de Internet es denegado, por tanto para cambiar esto es necesario crear las reglas necesarias para controlar el tráfico de entrada y de salida.

En la siguiente figura podemos ver cómo dos computadoras en la misma red virtual pueden comunicarse sin restricciones.

Por otra parte, para conectar las computadoras a Internet es conveniente configurar las reglas de entrada y de salida.

Debido a que las direcciones IP en una red virtual son privadas, para comunicar dos computadoras conectadas a diferentes redes virtuales se deberá utilizar las direcciones IP públicas de las computadoras:

Capítulo 34

Replicación de un sistema completo en la nube

En la actualidad el cómputo en la nube nos permite realizar el aprovisionamiento dinámico de recursos de forma fácil y rápida. Entonces ¿por qué no replicar el sistema completo?

34.1. Arquitectura de un sistema replicado

La siguiente figura muestra la arquitectura general de un sistema replicado:

En este caso se supone que el sistema consta de dos o más servidores de aplicaciones conectados a un servidor de bases de datos, el cual puede implementar la replicación de datos mediante discos espejo.

El administrador de tráfico funciona como un proxy inverso, el cual recibe las conexiones y peticiones de los clientes y las reenvía al sistema principal con copia a la réplica.

El sistema principal procesa las peticiones del cliente, y envía las respuestas al administrador de tráfico, el cual las reenvía al cliente. Notar que el administrador de tráfico deberá ignorar las respuestas de la réplica.

Para garantizar la recuperación en caso de desastre catastrófico en el site donde ejecuta el sistema principal, la réplica deberá ejecutar en una locación diferente, por lo tanto, el administrador de tráfico y la réplica deberán tener cada uno una IP pública.

Mientras el sistema principal puede estar ejecutando en la nube o en una instalación propia de la empresa (on-premise), la réplica estaría funcionando en la nube, en alguna locación geográfica diferente a la locación donde ejecuta el sistema principal.

El administrador de tráfico puede ser un programa o un appliance localizado en mismo site donde ejecuta el sistema principal, de manera que el administrador de tráfico se pueda conectar al sistema principal mediante una red privada.

Como puede observarse, tanto el sistema principal como la réplica realizarán las mismas transacciones sobre la base de datos (y/o el sistema de archivos), por lo que la consistencia de los datos está garantizada.

Si se produce un error en la comunicación entre el administrador de tráfico y la réplica, el cliente deberá recibir un de error de comunicación.

En caso de falla del sistema principal y/o falla del administrador de tráfico, solo habrá que redefinir el dominio del sistema a la IP pública de la réplica, entonces los clientes estarán conectados

directamente al sistema de respaldo.

Desde luego, posteriormente será necesario: 1) sacar de producción la réplica, 2) realizar una copia espejo de la réplica, y 3) restaurar el sistema principal y el administrador de tráfico.

34.2. Replicación de una arquitectura de tres capas en la nube

Como caso de estudio, veremos la replicación de la arquitectura de tres capas en la nube que implementamos en la tarea 1, consistente en una aplicación web (front-end), servicios web para Tomcat (back-end) y MySQL.

Como se explicó anteriormente, para replicar un sistema, podemos crear una máquina virtual en la nube (réplica) que procese todas las peticiones que realizan los clientes, simultáneamente al proceso de las mismas peticiones que realiza el sistema principal.

La arquitectura del sistema replicado que estudiaremos es la siguiente:

En el NSG (Network Security Group) del sistema principal se abrirá el puerto de entrada 80, y en el NSG de la réplica se abrirá el puerto de entrada 8080.

AdministradorTrafico.java

Vamos a utilizar el programa Proxy.java el cual es un proxy transparente escrito en Java, modificado para que funcione como un administrador de tráfico.

La arquitectura del programa Proxy.java se muestra a continuación:

Como podemos ver, el programa Proxy.java consta de un servidor y un cliente.

El servidor del proxy recibe la conexión del cliente del usuario, entonces el cliente del proxy realiza una conexión al servidor del usuario.

Cada petición que envía el cliente del usuario es recibida por el servidor del proxy, entonces es reenviada por el cliente del proxy al servidor del usuario.

La respuesta del servidor del usuario es recibida por el cliente del proxy, el cual la reenvía al servidor del proxy, y este a su vez, la reenvía al cliente del usuario.

Para implementar el administrador de tráfico, se agregó al programa Proxy.java, un cliente al programa el cual se conectará a una réplica del servidor del usuario, de manera que ahora las peticiones que recibe el servidor del proxy serán reenviadas al servidor principal y al servidor réplica.

Podemos ver que sólo la respuesta del servidor principal será enviada al usuario.

Para probar la replicación de una arquitectura de tres capas basada en Tomcat, haremos lo siguiente:

1. Crear dos máquinas virtuales utilizando la imagen que se creó en la tarea

1. Abrir el puerto de entrada 80 protocolo TCP en el NSG (Network Security Group) de la máquina virtual 1 (sistema principal).

1. Abrir el puerto de entrada 8080 protocolo TCP en el NSG (Network Security Group) de la máquina virtual 2 (réplica), ingresar en el campo .ºrigen"("Source"si la pantalla está en inglés) la IP de la máquina virtual 1 (por seguridad, la máquina virtual 1 será la única computadora que podrá acceder la máquina virtual 2).
2. Conectar a la máquina virtual 1 utilizando el programa ssh.

1. Utilizando el programa sftp enviar a la máquina virtual 1 el archivo: AdministradorTrafico.java

1. Compilar en la máquina virtual 1 el programa AdministradorTrafico.java.

1. Iniciar Tomcat en las máquinas virtuales 1 y 2.

1. Ejecutar en la máquina virtual 1 el administrador de tráfico:

```
sudo java AdministradorTrafico localhost 8080 ip-máquina-virtual-2 8080 80 &
```

Donde:

- localhost es la IP de la máquina virtual principal, donde ejecuta el administrador de tráfico.
- 8080 es el puerto que abre Tomcat en la máquina virtual principal.
- IP-máquina-virtual-2 es la IP de la réplica,
- 8080 es el puerto abierto por Tomcat en la réplica,
- 80 es el puerto abierto por el programa AdministradorTrafico.java en el sistema principal

Notar que no es necesario abrir el puerto 8080 en la primera máquina virtual, ya que el programa AdministradorTrafico.java y Tomcat se comunican localmente mediante loopback.

En este caso ejecutamos el administrador de tráfico con ”sudo”para que éste pueda abrir el puerto 80 en la máquina virtual 1.

1. En la computadora local (Windows, Linux o MacOS):

9.1 En un navegador se deberá ingresar la URL para iniciar la aplicación web, notar que no es necesario ingresar el número de puerto, ya que se utiliza el puerto default 80:

`http://ip-máquina-virtual-1/prueba.html`

9.2 Ahora se deberá ejecutar diversas funciones en la aplicación web de manera que se realicen las transacciones correspondientes en la base de datos.

9.3 Podemos verificar que las bases de datos en la máquina virtual y la máquina virtual réplica son idénticas.

34.3. Administrador de tráfico SSL

En el caso anterior utilizamos el programa AdministradorTrafico.java como un administrador de tráfico para HTTP, lo cual no es de utilidad en la práctica debido a que actualmente se utiliza el protocolo HTTPS en lugar del protocolo HTTP por razones de seguridad.

En realidad, el programa AdministradorTrafico.java permite derivar cualquier tráfico TCP no SSL, por ejemplo: HTTP, FTP, Telnet, SMTP, etc.

Para que el programa AdministradorTraficor.java pueda administrar tráfico HTTPS, solo es necesario implementar la conexión con sockets seguros en el servidor del proxy; el programa modificado se llama AdministradorTraficoSSL.java

La arquitectura del sistema replicado es la siguiente:

Ahora, se abrirá el puerto de entrada 443 en el NSG (Network Security Group) del sistema principal, y se abrirá el puerto de entrada 22 en el NSG de la réplica. El programa AdministradorTraficoSSL.java requiere un keystore que contenga el certificado del servidor, este certificado puede ser un certificado auto-firmado o un certificado emitido por una CA. Por tanto, es necesario definir el nombre del keystore y la contraseña en el programa AdministradorTraficoSSL.java.

Notar que en esta arquitectura no es necesario instalar un certificado digital en Tomcat, ya que Tomcat seguirá utilizando el protocolo HTTP.

Debido a que el administrador de tráfico (AdministradorTraficoSSL.java) se comunica en forma insegura con el Tomcat que ejecuta en la réplica, será necesario crear un túnel SSH entre el administrador de tráfico y Tomcat.

Para ejecutar el administrador de tráfico HTTPS:

```
sudo java AdministradorTraficoSSL localhost 8080 localhost 8081 443 &
```

Notar que la IP de la máquina virtual 2 se substituye por ”localhost”, debido a que el segundo cliente del administrador de tráfico se conecta al cliente de SSH, el cual ejecuta en la computadora principal.

También es necesario que el cliente SSH abra un puerto diferente (p.e. 8081) al puerto que abre Tomcat (8080), debido a que el cliente SSH y Tomcat ejecutan en la misma computadora.

Entonces para ejecutar la aplicación web, ingresamos en el navegador la URL de la aplicación web:

`https://ip-máquina-virtual-1/prueba.html`

Si el certificado del servidor es un certificado auto-firmado, entonces el navegador mostrará una advertencia que indica que el certificado es inseguro. En este caso podemos continuar debido a que sabemos que el certificado es seguro porque lo hemos creado nosotros.

Capítulo 35

Tablas agregadas

Ahora veremos cómo utilizar tablas agregadas para ejecutar eficientemente consultas multidimensionales, así mismo, veremos cómo podemos generar las tablas agregadas para un sistema de BI, utilizando técnicas de MapReduce. Consideremos la base de datos .ejemplo_olap"(modelo estrella) que creamos anteriormente.

Ejecutando la siguiente instrucción SELECT, podemos obtener un cubo OLAP que incluya las ventas acumuladas con respecto a las dimensiones country, category y product:

```
SELECT c.country, cat.category, p.product, SUM(f.sales) AS total_sales
FROM fact_table f
JOIN country c ON f.id_country = c.id_country
JOIN product p ON f.id_product = p.id_product
JOIN category cat ON f.id_category = cat.id_category
GROUP BY c.country, cat.category, p.product
ORDER BY c.country, cat.category, p.product;
```

Sin embargo, si la fact table es muy grande, la ejecución de la instrucción SELECT puede tardar mucho tiempo.

Para que las consultas multidimensionales ejecuten más rápido, se utilizan tablas agregadas.

Supongamos que creamos una tabla llamada çountry_category_product", incluyendo los siguientes campos: id_country, id_category, id_product, sales. Esta tabla va a conteneer las ventas acumuladas por país, categoría y producto.

Entonces se dice que esta tabla es una tabla agregada, debido a que contiene una métrica acumulada.

Para cargar la tabla agregada çountry_category_product.ejecutamos la siguiente instrucción:

```
INSERT INTO country_category_product (id_country, id_category, id_product, sales)
SELECT c.id_country, p.id_category, f.id_product, SUM(f.sales) as total_sales
FROM fact_table f
INNER JOIN country c ON f.id_country = c.id_country
INNER JOIN product p ON f.id_product = p.id_product
GROUP BY c.id_country, p.id_category, f.id_product;
```

Entonces podemos utilizar la tabla agregada çountry_category_product"(en lugar de la

fact_table) para obtener el cubo OLAP con las ventas acumuladas con respecto a las dimensiones country, category y product:

```
SELECT c.country, cat.category, p.product, SUM(ccp.sales) AS total_sales
FROM country_category_product ccp
JOIN country c ON ccp.id_country = c.id_country
JOIN category cat ON ccp.id_category = cat.id_category
JOIN product p ON ccp.id_product = p.id_product
GROUP BY c.country, cat.category, p.product
ORDER BY c.country, cat.category, p.product;
```

Podemos ver que el cubo OLAP se genera más rápido, si utilizamos la tabla agregada en lugar de la fact table, ya que la tabla agregada tiene menos registros que la fact table.

De la misma forma, si queremos obtener un cubo OLAP con las dimensiones country y category, acumulando sales, también podemos utilizar la tabla agregada çountry_category_product”:

```
SELECT c.country, cat.category, SUM(ccp.sales) AS total_sales
FROM country_category_product ccp
JOIN country c ON ccp.id_country = c.id_country
JOIN category cat ON ccp.id_category = cat.id_category
GROUP BY c.country, cat.category
ORDER BY c.country, cat.category;
```

También es posible utilizar la tabla agregada çountry_category_product”, para obtener un cubo OLAP que incluya solo la dimensión country, acumulando sales:

```
SELECT c.country, SUM(ccp.sales) AS total_sales
FROM country_category_product ccp
JOIN country c ON ccp.id_country = c.id_country
GROUP BY c.country, cat.category
ORDER BY c.country, cat.category;
```

Y para obtener un cubo con cero dimensiones, también podemos utilizar la tabla agregada çountry_category_product”:

```
SELECT SUM(ccp.sales) AS total_sales
FROM country_category_product ccp;
```

Entonces, para que las consultas OLAP ejecuten rápidamente, los analistas de BI deben saber previamente qué tipo de cubos van a generar los usuarios, de manera que puedan crear y cargar las tablas agregadas necesarias.

El proceso de carga de la fact table, actualización de las tablas de dimensiones, y carga de las tablas agregadas, generalmente se realiza por la noche, de manera que los usuarios del sistema de BI puedan al otro día generar los cubos OLAP que requieran.

Si las fact tables son muy grandes, el proceso de carga de las tablas agregadas puede tardar demasiado.

Para acelerar la carga de las tablas agregadas, los analistas de BI pueden distribuir el proceso en varias computadoras, sin embargo, esto implicaría instalar el DMBS en diferentes computadoras, lo cual requiere más licencias para el DBMS lo cual incrementa considerablemente los costos.

Ahora veremos cómo podemos generar las tablas agregadas para un sistema de BI, utilizando técnicas de MapReduce.

35.1. MapReduce

MapReduce es un modelo de programación utilizado para procesar grandes cantidades de datos en el área de Big Data.

Este modelo de programación fue introducido originalmente por Google para indexar la WWW, no obstante, la implementación más popular de MapReduce es actualmente Apache Hadoop.

MapReduce consta de dos operaciones, la operación ”Map”(mapear) y la operación Reduce”(reducir).

- Map. En la fase de Map, se lee un conjunto de datos y se genera una salida consistente en parejas çlave-valor”(parejas ”key-value”).
- Reduce. En la fase Reduce, se lee las parejas de clave-valor creadas en la fase Map, entonces se genera un conjunto más pequeño de parejas clave-valor. La reducción”puede implicar sumar los valores para una misma clave, obtener el mayor o menor valor para una misma clave, obtener el promedio de los valores para una misma llave, etc.

Por ejemplo, supongamos que tenemos un archivo CSV con los siguientes datos de entrada:

1,2,1,200.0 2,3,1,100.0 1,2,1,50.0 2,3,1,200.0 1,2,3,100.0

Supongamos que el proceso Map obtiene la clave concatenando los tres primeros números separándolos con una coma.

Entonces las parejas clave-valor que produce el proceso Map son las siguientes:

”1,2,1”200.0 ”1,2,1”50.0 ”2,3,1”100.0 ”2,3,1”200.0 ”1,2,3”100.0

Ahora bien, la salida del proceso Map no entra directamente al proceso Reduce, primero es necesario crear una colección con los valores asociados a cada clave:

”1,2,1”200.0 50.0 ”2,3,1”100.0 200.0 ”1,2,3”100.0

El proceso Reduce recibe una colección de valores para cada clave. En este caso, el proceso Reduce va a acumular los valores para cada clave, entoncesd el proceso Reduce finalmente produce las siguientes parejas clave-valor:

”1,2,1”250.0 ”2,3,1”300.0 ”1,2,3”100.0

35.1.1. Job MapReduce

Un job MapReduce es la ejecución de un programa que realiza los procesos Map y Reduce sobre una arquitectura master-slave.

Los datos de entrada se colocan inicialmente en un sistema de archivos distribuido llamado HDFS (Hadoop Distributed File System).

En un job MapReduce se realizan las siguientes tareas:

1. Split (dividir). El HDFS distribuye los datos de entrada en los nodos slave que forman un cluster.
2. Map (mapear). Cada nodo slave ejecuta el proceso Map, el resultado intermedio consistente en parejas clave-valor.
3. Shuffling (repartir). Los resultados intermedios son enviados a diferentes nodos slave, de acuerdo a la clave, de manera que los valores asociados a una misma clave sean puestos en el mismo nodo.
4. Merge (unir). Los resultados intermedios son agrupados por la clave.
5. Reduce (reducir). Cada nodo slave ejecuta el procedo Reduce, el resultado se escribe en el HDFS.

35.2. Cómo programar un proceso MapReduce en Java

Ahora veremos cómo programar en Java un proceso MapReduce utilizando Apache Hadoop.

Hadoop es una herramienta utilizada en al área de Big Data para almacenar y procesar grandes volúmenes de datos. Los datos se distribuyen en clusters de nodos utilizando el sistema de archivos HDFS (Hadoop Distributed File System).

Supongamos que tenemos datos de entrada en un archivo CSV.

Cada registro en el archivo contiene tres números enteros y un valor punto flotante.

Entonces queremos acumular los valores punto flotante utilizando como clave la combinación de los tres números enteros.

Primeramente, necesitamos crear una clase ”Mapper”:

```
// Generado por ChatGPT, 2024
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.Mapper;
import java.io.IOException;
public class AggregationMapper extends Mapper<LongWritable, Text, Text, DoubleWritable> {
private Text dimensionKey = new Text();
private DoubleWritable metricValue = new DoubleWritable();
@Override
public void map(LongWritable key, Text value, Context context) throws IOException,
↪  InterruptedException {
String[] parts = value.toString().split(",");
if (parts.length == 4) {
    String d1 = parts[0].trim();
    String d2 = parts[1].trim();
    String d3 = parts[2].trim();
    double m1 = Double.parseDouble(parts[3].trim());
    dimensionKey.set(d1 + "," + d2 + "," + d3);
    metricValue.set(m1);
    context.write(dimensionKey, metricValue);
}
}
}
```

El método ”map.[es] invocado por cada registro (row) en el archivo de datos de entrada.

El método ”maprecibe como parámetros:

- key. La posición del registro (línea de texto) en el archivo de entrada. El primer registro es 0, el segundo registro es 1, y así sucesivamente.
- value. El contenido del registro de entrada (línea de texto), de tipo Text.
- context. El contexto donde se escribirá la salida (parejas clave-valor).

En este caso, el método ”map”divide el registro de entrada utilizando la coma como separador.

Entonces obtiene la clave, de tipo Text, formada por los tres primeros números concatenados mediante una coma.

Finalmente escribe a la salida la clave y el valor de tipo DoubleWritable.

Ahora necesitamos crear una clase Reducer”:

```
// Generado por ChatGPT, 2024
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.Reducer;
import java.io.IOException;
public class AggregationReducer extends Reducer<Text, DoubleWritable, Text, DoubleWritable>{
  private DoubleWritable result = new DoubleWritable();
  @Override
  public void reduce(Text key, Iterable<DoubleWritable> values, Context context) throws IOException,
  ↪  InterruptedException {
double sum = 0.0;
for (DoubleWritable val : values) {
  sum += val.get();
}
result.set(sum);
context.write(key, result);
  }
}
```

El método reduce.[es] invocado para cada clave diferente.

El método reducerecibe como parámetros:

- key. La clave creada por el proceso Map, de tipo Text.
- values. Una colección de valores asociados con la clave, en este caso los valores son de tipo DoubleWritable.
- context. El contexto donde se escribirá la salida (parejas clave-valor).

En este caso, el método reduce.[a]cumula los valores asociados con la clave, entonces escribe a la salida la clave y la suma de los valores.

El programa principal que ejecuta el proceso MapReduce es el siguiente:

```
// Generado por ChatGPT, 2024.
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.*;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
public class AggregationDriver {
```

```
public static void main(String[] args) throws Exception {
if (args.length != 2) {
    System.err.println("Usage: AggregationDriver <input path> <output path>");
    System.exit(-1);
}
Configuration conf = new Configuration();
Job job = Job.getInstance(conf, "Data Aggregation");
job.setJarByClass(AggregationDriver.class);
job.setMapperClass(AggregationMapper.class);
job.setReducerClass(AggregationReducer.class);
job.setOutputKeyClass(Text.class);
job.setOutputValueClass(DoubleWritable.class);
FileInputFormat.addInputPath(job, new Path(args[0]));
FileOutputFormat.setOutputPath(job, new Path(args[1]));
System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}
```

El programa principal realiza lo siguiente:

- Crea un job MapReduce, en este caso el nombre del job es ”Data Aggregation”.
- Define la clase correspondiente al programa principal AggregationDriver.class).
- Define la clase Mapper (AggregationMapper.class).
- Define la clase Reducer (AggregationReducer.class).
- Define el tipo de datos de la clave de salida (Text.class).
- Define el tipo de dato del valor de salida (DoubleWriteble.class).
- Define el nombre del directorio donde se encuentran los archivos con los datos de entrada (InputPath).
- Define el nombre del directorio donde se colocarán los archivos de salida (OutputPath).
- Espera que termine el job MapReduce.

35.3. Azure HDInsight

Azure HDInsight es un servicio para análisis de datos empresariales, open-source y completamente administrado.

Está diseñado para manejar grandes volúmenes de datos rápida y eficientemente.

En HDInsight se puede utilizar diferentes fraeworks open-source como Apache Hadoop, entre otros.

HDInsight permite reducir costos, así mismo, es un servicio elástico ya que permite escalar en forma automática la carga de trabajo hacia arriba o hacia abajo.

Algunos ejemplos de uso de HDInsight:

- Procesamiento ETL (Extract-Transform-Load), el cual se utiliza para extraer datos (estructurados o no estructurados) de fuentes heterogeneas, realizar alguna transformación a los datos y cargar los datos en un almacén de datos. Los datos no estructurados pueden provenir de las redes sociales, Internet, o en general, de archivos con información que se desee procesar.

- Procesamiento de datos para Data warehouse, donde es necesario crear tablas agregadas para optimizar las consultas que se realizan sobre los modelos multidimensionales. En este caso, se utiliza un cluster Hadopp.
- Internet de las cosas (IoT), donde es necesario procesar en tiempo real flujos de datos (stream processing) de diferentes dispositivos conectados a Internet, entonces almacenar los datos procesados, enviarlos a aplicaciones en tiempo real, o enviarlos a paneles de control (dashboards) en tiempo real. En este caso, se utiliza un cluster Kafka. Apache Kafka es una plataforma distribuida de streaming de eventos en base a una arquitectura publisher-suscriber.

En HDInsight se puede programar en Java, Python, .NET y Go.

Parte IV

Unidad 4

Capítulo 36

La clase de hoy vamos a explicar cómo utilizar JSON para serializar y de-serializar objetos.

JSON (JavaScript Object Notation) es un formato texto para el intercambio de datos.

JSON define una sintaxis parecida a la sintaxis utilizada en Javascript para escribir objetos.

Las diferencias entre un objeto JSON y un objeto Javascript son las siguientes:

- Un objeto JSON lleva comillas en los nombres de los campos.
- Los valores en un objeto JSON solo pueden ser arreglos, objetos, números, strings, true, false y null
- Un objeto JSON no permite comentario.

Por ejemplo:

```
Objeto Javascript: {a:1, b:2, c:3}
Objeto JSON: {"a":1, "b":2, "c":3}
```

JSON es un formato independiente del lenguaje de programación, de manera que es posible escribir fácilmente programas en cualquier lenguaje que creen mensajes en formato JSON, así como programas que lean mensajes en formato JSON.

En JSON es posible crear dos estructuras: objetos y arreglos.

36.1. Sintaxis de JSON

```
Un whitespace es un separador de tokens, de acuerdo a la siguiente sintaxis:

Fuente: www.json.org
```

Un objeto es una colección no ordenada de parejas nombre:valor separadas por coma. Un objeto comienza con una llave que abre “{“ y termina con una llave que cierra “}”.

```
La sintaxis de un objeto es la siguiente:

Fuente: www.json.org
```

Un arreglo es una colección ordenada de valores separados por coma. Un arreglo comienza con un corchete que abre "[" y termina con un corchete que cierra "]".

```
La sintaxis de un arreglo es la siguiente:

Fuente: www.json.org
```

Un valor puede ser una cadena de caracteres entre comillas, o un número, o un objeto, o un arreglo, o las constantes true, false o null.

```
La sintaxis de un valor es la siguiente:

Fuente: www.json.org
```

Una cadena de caracteres (string) es una secuencia de cero o más caracteres Unicode encerrados entre comillas.

```
Fuente: www.json.org
Aquí, "codepoint" se refiere a cualquier carácter.
```

Notas.

- Un texto JSON se codifica por default utilizando UTF-8, por tanto, una string JSON puede incluir caracteres de cualquier idioma.
- Debido a que los nombres de los campos son strings, también los nombres de los campos pueden incluir caracteres de cualquier idioma.
- Se puede representar cualquier carácter mediante una secuencia de escape \uxxxx, donde xxxx es un número hexadecimal de cuatro dígitos. La secuencia de escape \uxxxx es necesaria para incluir caracteres de cualquier idioma, cuando se codifica un texto JSON utilizando codificación ANSI o codificación ASCII, sin embargo, la codificación estándar para JSON es UTF-8.
- Debido a que la codificación ASCII es un subconjunto de ANSI, y ANSI es un subconjunto de UTF-8, un texto JSON con codificación ASCII o ANSI puede ser des-serializado correctamente.

```
Un número tiene la siguiente sintaxis:

Fuente: www.json.org
```

36.2. Ahora veremos un ejemplo, utilizando GSON (implementación de JSON desarrollada por Google).

Se requiere descargar el archivo gson-2.8.6.jar de la siguiente URL:

https://repo1.maven.org/maven2/com/google/code/gson/gson/2.8.6/gson-2.8.6.jar

Capturar el siguiente programa EjemploGSON.java y colocarlo en el mismo directorio donde está el archivo gson-2.8.6.jar que se descargó anteriormente.

```
import com.google.gson.Gson;
import com.google.gson.GsonBuilder;
import java.sql.Timestamp;

class EjemploGSON
{
  static class Empleado
  {
String nombre;
int edad;
float sueldo;
Timestamp fecha_ingreso;
Empleado(String nombre,int edad,float sueldo,Timestamp fecha_ingreso)
{
  this.nombre = nombre;
  this.edad = edad;
  this.sueldo = sueldo;
  this.fecha_ingreso = fecha_ingreso;
}
  }

  public static void main(String[] args)
  {
Empleado[] e = new Empleado[3];
e[0] = new Empleado("Hugo",20,1000,Timestamp.valueOf("2020-01-01 20:10:00"));
e[1] = new Empleado("Paco",21,2000,Timestamp.valueOf("2019-10-01 10:15:00"));
e[2] = new Empleado("Luis",22,3000,Timestamp.valueOf("2018-11-01 00:00:00"));

Gson j = new GsonBuilder().setDateFormat("yyyy-MM-dd'T'HH:mm:ss.SSS").create();

String s = j.toJson(e);
System.out.println(s);

Empleado[] v = (Empleado[])j.fromJson(s,Empleado[].class);

for (int i = 0; i < v.length; i++)
  System.out.println(v[i].nombre + " " + v[i].edad + " " + v[i].sueldo + " " + v[i].fecha_ingreso);
  }
}
```

Para compilar el programa EjemploGSON.java se debe ejecutar el siguiente comando:

```
javac -cp gson-2.8.6.jar EjemploGSON.java
```

Para ejecutar el programa en Windows:

```
java -cp gson-2.8.6.jar;. EjemploGSON
```

Para ejecutar el programa en Linux:

```
java -cp gson-2.8.6.jar:. EjemploGSON
```

Ahora se explicará cómo funciona este programa.

Primeramente, se declaran los imports de las clases que se utilizarán: Gson, GsonBuilder y Timestamp.

Se define una clase Empleado que contiene los campos: nombre, edad, sueldo y fecha_ingreso. Por conveniencia se define un constructor para inicializar los campos al crear una instancia.

Notar que la fecha se maneja como fecha-hora debido a que en los sistemas globales (Internet) la fecha no indica qué sucede antes y que sucede después a nivel mundial; recordar lo que se vimos sobre tiempo UTC y tiempo local.

```
En la función "main" se crea un arreglo de 3 empleados, cada elemento se inicializa con una instancia
↪  de la clase "Empleado" con diferentes datos.

Entonces se crea una instancia de la clase Gson. Notar que se utiliza el método setDateFormat() para
↪  utilizar el formato de fecha ISO8601, el cual es el formato que utiliza Javascript para manejar
↪  fecha-hora.

Después se utiliza el método toJson de la clase Gson para serializar el arreglo de empleados. Esta
↪  clase produce la siguiente string:

[{"nombre":"Hugo","edad":20,"sueldo":1000.0,"fecha_ingreso":"2020-01-01T20:10:00.000"},{"nombre":"Pac⌋
↪  o","edad":21,"sueldo":2000.0,"fecha_ingreso":"2019-10-01T10:15:00.000"},{"nombre":"Luis","edad":2⌋
↪  2,"sueldo":3000.0,"fecha_ingreso":"2018-11-01T00:00:00.000"}]

Finalmente se utiliza el método fromJson de la clase Gson para deserializar la string anterior y
↪  producir un nuevo arreglo de empleados. Entonces se despliegan los datos de los empleados:

Hugo 20 1000.0 2020-01-01 20:10:00.0
```

Paco 21 2000.0 2019-10-01 10:15:00.0 Luis 22 3000.0 2018-11-01 00:00:00.0

Capítulo 37

Servicios web

La clase de hoy veremos los conceptos básicos de los servicios web (Web Services) así como los elementos de servicios web SOAP (Simple Object Access Protocol) y REST (REpresentational State Transfer).

37.1. Conceptos básicos de Servicios web

En el documento Web Services Architecture (2004) del World Wide Web Consortium (W3C) define un servicio web como:

"Un sistema de software diseñado para soportar la interacción interoperable de máquina a máquina en una red. Cuenta con una interfaz descrita en un formato procesable por máquina (específicamente Servicios Web de lenguaje de descripción, conocido por el acrónimo WSDL). Otros sistemas interactúan con el servicio Web de una manera prescrita por su descripción utilizando mensajes SOAP, típicamente transportados utilizando HTTP con una serialización XML en conjunción con otros estándares relacionados con la Web".

Un servicio web es un concepto abstracto que se implementa mediante un agente concreto.

Un agente es el software o hardware que envía y recibe mensajes.

El servicio es el recurso caracterizado por la funcionalidad que provee. Un servicio web no cambia aún cuando cambie el agente, es decir, la funcionalidad es independiente de la implementación de ésta.

El propósito de un servicio web es proveer cierta funcionalidad a nombre de su propietario (una persona o una organización).

La entidad proveedora es aquella persona u organización que provee un agente que implementa un determinado servicio.

Una entidad solicitante es una persona u organización que desea hacer uso del servicio mediante un agente solicitante (también llamado solicitante del servicio) que intercambia mensajes con el agente proveedor (también llamado proveedor del servicio).

En la mayoría de los casos el agente solicitante es el que inicia la comunicación con el agente proveedor, aunque no siempre es así, no obstante se sigue llamando agente solicitante aunque no sea el que inicia la comunicación.

La semántica de un servicio web es la expectativa compartida sobre el comportamiento del servicio, en particular el comportamiento en respuesta a los mensajes que recibe.

Se le llama contrato al acuerdo entre la entidad solicitante y la entidad proveedora. Un contrato puede ser explícito o implícito, escrito u oral, establecido entre las personas y/o las computadoras, legal o informal.

Hay dos tipos de contratos:

- Contrato de descripción. La descripción del servicio es el contrato que gobierna la mecánica de interacción con un servicio en particular, esto es, los nombres de los métodos web, los nombres de los parámetros, los tipos de dato de los parámetros y el tipo de dato del resultado. En los servicios SOAP el contrato de descripción es el WSDL, y en los servicios REST el contrato de descripción puede escribirse utilizando OpenAPI.
- Contrato semántico. La semántica del servicio es el contrato que gobierna el significado y propósito de la interacción, esto es, la funcionalidad del servicio, por tanto, el contrato semántico describe lo que hace cada método web. El contrato semántico se escribe en lenguaje natural.

También puede haber contratos "híbridos" que incluyan elementos de descripción y elementos de semántica.

A partir de una descripción detallada del servicio web en lenguaje natural, es posible utilizar una IAG para generar el contrato de descripción y el contrato semántico; así mismo, apartir de los contratos, la IAG podrá generar el código correspondiente al agente proveedor y al agente solicitante.

37.1.1. Participación en un servicio web

Una entidad solicitante puede participar de un servicio web de diferentes maneras. La siguiente figura muestra el proceso general de participación en un servicio web.

1. Las entidades solicitante y proveedora se conocen una a la otra, o por lo menos una conoce a la otra.
2. Las entidades acuerdan la descripción (WSD: Web Service Description) y semántica del servicio.
3. La descripción y la semántica son implementadas por el agente solicitante y el agente proveedor.
4. Los agentes solicitante y proveedor intercambian mensajes.

Fuente: Web Services Architecture, W3C

37.2. Servicios web SOAP

SOAP (Simple Object Access Protocol) define un protocolo de RPC (Remote Procedure Call) basado en XML, para la interacción cliente-servidor a través de la red utilizando:

1. HTTP como la base de transporte

1. Documentos XML para la codificación de requerimientos y respuestas.

SOAP permite la comunicación entre aplicaciones ejecutando en diferentes sistemas operativos, con diferentes tecnologías y lenguajes de programación.

37.2.1. Componentes de un mensaje SOAP

Un mensaje SOAP es un documento XML compuesto por los siguientes elementos:

- Envelope (sobre). Identifica el documento XML como un mensaje SOAP.
- Header (encabezado). Contiene información de encabezado.
- Body (cuerpo). Contiene información del requerimiento y la respuesta.
- Fault (falla). Contiene errores e información de estatus.

Para desarrollar servicios web SOAP en Java (p.e. Tomcat), se deberá utilizar una biblioteca que implemente la especificación JAX-WS.

37.2.2. Web Services Description Language (WSDL)

Un documento WSDL es un archivo XML que contiene la descripción de un servicio web SOAP. Este especifica la localización del servicio y los métodos del servicio.

Un cliente puede hacer un requerimiento HTTP a un servicio web SOAP para obtener el WSDL que describe el servicio web.

Los elementos de un documento WSDL son los siguientes:

Elemento	Descripción
<types>	Define los tipos de dato usados por el servicio web
<message>	Define los elementos de datos para cada operación
<portType>	Describe las operaciones que pueden ser ejecutadas y los mensajes involucrados
<binding>	Define el protocolo y el formato de los datos para cada portType

En la actividad individual de la clase de hoy, vamos a implementar un servicio web SOAP en Java.

Una vez publicado el servicio web, ingresamos la siguiente URL en un navegador:

```
http://localhost:8080/ServicioWeb?wsdl
```

Entonces obtenemos el WSDL del servicio web:

```
<definitions
↪  xmlns:wsu="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-utility-1.0.xsd"
     xmlns:wsp="http://www.w3.org/ns/ws-policy"
     xmlns:wsp1_2="http://schemas.xmlsoap.org/ws/2004/09/policy"
     xmlns:wsam="http://www.w3.org/2007/05/addressing/metadata"
     xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
     xmlns:tns="http://ws/"
     xmlns:xsd="http://www.w3.org/2001/XMLSchema"
     xmlns="http://schemas.xmlsoap.org/wsdl/"
     targetNamespace="http://ws/"
     name="ServicioWebService">
```

```
<types>
<xsd:schema>
    <xsd:import namespace="http://ws/"
                schemaLocation="http://localhost:8080/ServicioWeb?xsd=1"/>
</xsd:schema>
</types>

<message name="mayusculas">
<part name="parameters" element="tns:mayusculas"/>
</message>

<message name="mayusculasResponse">
<part name="parameters" element="tns:mayusculasResponse"/>
</message>

<message name="suma">
<part name="parameters" element="tns:suma"/>
</message>

<message name="sumaResponse">
<part name="parameters" element="tns:sumaResponse"/>
</message>

<message name="suma2">
<part name="parameters" element="tns:suma2"/>
</message>

<message name="suma2Response">
<part name="parameters" element="tns:suma2Response"/>
</message>

<portType name="ServicioWeb">
<operation name="mayusculas">
    <input wsam:Action="http://ws/ServicioWeb/mayusculasRequest"
           message="tns:mayusculas"/>
    <output wsam:Action="http://ws/ServicioWeb/mayusculasResponse"
            message="tns:mayusculasResponse"/>
</operation>

<operation name="suma">
    <input wsam:Action="http://ws/ServicioWeb/sumaRequest"
           message="tns:suma"/>
    <output wsam:Action="http://ws/ServicioWeb/sumaResponse"
            message="tns:sumaResponse"/>
</operation>

<operation name="suma2">
    <input wsam:Action="http://ws/ServicioWeb/suma2Request"
           message="tns:suma2"/>
    <output wsam:Action="http://ws/ServicioWeb/suma2Response"
            message="tns:suma2Response"/>
</operation>
</portType>

<binding name="ServicioWebPortBinding" type="tns:ServicioWeb">
<soap:binding transport="http://schemas.xmlsoap.org/soap/http"
             style="document"/>
```

```
<operation name="mayusculas">
    <soap:operation soapAction=""/>
    <input>
        <soap:body use="literal"/>
    </input>
    <output>
        <soap:body use="literal"/>
    </output>
</operation>

<operation name="suma">
    <soap:operation soapAction=""/>
    <input>
        <soap:body use="literal"/>
    </input>
    <output>
        <soap:body use="literal"/>
    </output>
</operation>

<operation name="suma2">
    <soap:operation soapAction=""/>
    <input>
        <soap:body use="literal"/>
    </input>
    <output>
        <soap:body use="literal"/>
    </output>
</operation>
</binding>

<service name="ServicioWebService">
<port name="ServicioWebPort" binding="tns:ServicioWebPortBinding">
    <soap:address location="http://localhost:8080/ServicioWeb"/>
</port>
</service>

</definitions>
```

37.2.3. A continuación, se explica las secciones del WSDL:

```
<definitions>

El nombre del servicio web: name="ServicioWebService"
Los namespaces con los prefijos: wsu, wsp, wsp1_2, wsam, soap, tns, xsd.
El espacio de nombres principal del servicio: targetNamespace="http://ws/"
```

```
<types>

Define los tipos de datos que utiliza el servicio.
```

Importa un esquema XSD externo "`http://localhost:8080/ServicioWeb?xsd=1`" donde se definen los tipos de datos de los parámetros.

```
<message>
```

Define los mensajes entreda y salida (request y reponse). Cada mensaje hace referencia a un elemento definido en el XSD.

```
<portType>

Define las operaciones del servicio, en este caso: mayusculas, suma y suma2.
```

```
<binding>

Define cómo se comunica el servicio, en este caso, usa SOAP sobre HTTP.
```

```
<service>

Define el endpoint del servicio "http://localhost:8080/ServicioWeb"
```

De la misma forma, para obtener el esquema XSD externo, ingresamos la URL "http://localhost:8080/ServicioWeb?xsd=1" en el navegador:

```
<xs:schema xmlns:tns="http://ws/"
   xmlns:xs="http://www.w3.org/2001/XMLSchema"
   version="1.0"
   targetNamespace="http://ws/">

<xs:element name="mayusculas" type="tns:mayusculas"/>
<xs:element name="mayusculasResponse" type="tns:mayusculasResponse"/>
<xs:element name="suma" type="tns:suma"/>
<xs:element name="suma2" type="tns:suma2"/>
<xs:element name="suma2Response" type="tns:suma2Response"/>
<xs:element name="sumaResponse" type="tns:sumaResponse"/>

<xs:complexType name="suma">
<xs:sequence>
    <xs:element name="a" type="xs:double"/>
    <xs:element name="b" type="xs:double"/>
</xs:sequence>
</xs:complexType>

<xs:complexType name="sumaResponse">
<xs:sequence>
    <xs:element name="return" type="xs:double"/>
</xs:sequence>
</xs:complexType>

<xs:complexType name="suma2">
<xs:sequence>
    <xs:element name="a"
```

```
                type="xs:int"
                minOccurs="0"
                maxOccurs="unbounded"/>
    <xs:element name="b"
                type="xs:int"
                minOccurs="0"
                maxOccurs="unbounded"/>
</xs:sequence>
</xs:complexType>

<xs:complexType name="suma2Response">
<xs:sequence>
    <xs:element name="return"
                type="xs:int"
                minOccurs="0"
                maxOccurs="unbounded"/>
</xs:sequence>
</xs:complexType>

<xs:complexType name="mayusculas">
<xs:sequence>
    <xs:element name="s"
                type="xs:string"
                minOccurs="0"/>
</xs:sequence>
</xs:complexType>

<xs:complexType name="mayusculasResponse">
<xs:sequence>
    <xs:element name="return"
                type="xs:string"
                minOccurs="0"/>
</xs:sequence>
</xs:complexType>

</xs:schema>
```

37.3. Servicios web REST

REST (REpresentational State Transfer) define un conjunto de principios arquitectónicos para la creación de servicios web.

REST fue presentado por Roy Fielding el año 2000 en su tesis doctoral .Architectural Styles and the Design of Network-based Software Architectures".

El diseño de servicios web estilo REST sigue cuatro principios:

- Utilizar métodos HTTP de forma explícita.
- Un servicio web utiliza los métodos de HTTP: POST para crear un recurso, GET para leer un recurso, PUT para cambiar el estado o actualizar un recurso, y DELETE para borrar un recurso.
- Los servicios son "sin estado"(stateless).
- Los clientes de servicios web estilo REST deben enviar peticiones completas e independientes, es decir, las peticiones deben incluir todos los datos que permitan completar el

servicio, sin la necesidad de guardar un estado entre peticiones.

- Los URIs representan una estructura de directorios.
- Los URIs (Uniform Resource Identifier) deben ser intuitivos y auto-explicados. Un URI es una jerarquía que corresponde a la estructura de los servicios web definidos en la empresa.
- Se transfiere XML, JSON o ambos.
- Los recursos que provee un servicio web pueden ser documentos, imágenes, videos y en general objetos. La representación de objetos mediante XML o JSON es fácil e independiente de la plataforma.

Para desarrollar servicios web REST en Java (p.e. Tomcat), se deberá utilizar una biblioteca que implemente la especificación JAX-RS (p.e. Jersey).

37.3.1. OpenAPI

Al igual que los servicios SOAP, para desarrollar servicios REST se requiere definir un contrato de descripción y un contrato semántico.

Para definir el contrato de descripción para un servicio REST, se utiliza generalmente OpenAPI.

La especificación de OpenAPI define un lenguaje estándar que permite describir los métodos web de un servicio web.

Un contrato de descripción definido mediante OpenAPI, puede ser utilizado para la generación de documentación, la generación de código para el servidor, la generación de código para el cliente, la generación de programas de prueba, entre otros usos.

También es posible pedir a una IAG (p.e. ChatGPT), que genere un contrato de descripción utilizando OpenAPI a partir de una descripción en lenguaje natural, y luego genere código para el cliente y el servidor a partir del contrato de descripción.

Reproducir Vídeo

Capítulo 38

Actividades individuales a realizar

En esta actividad vamos a crear un servicio web SOAP utilizando JDK8.

Nota. Debido a que SOAP se considera obsoleto, JAX-WS no está soportado a partir de la versión 11 del JDK, toda vez que se alienta a los desarrolladores a implementar servicios REST mediante JAX-RS. En cualquier caso, para implementar servicios SOAP a partir del JDK 11 será necesario utilizar alguna biblioteca de JAX-WS (p.e. Metro).

1. Primeramente creamos un directorio llamado "ws.[en] el cual vamos a colocar los programas que vamos a utilizar. Este directorio corresponde al paquete "ws"donde colocaremos las clases.

1. Vamos a crear el archivo "ServicioWeb.java.[en] el directorio "ws":

```
package ws;

import javax.jws.WebService;
import javax.jws.WebMethod;
import javax.jws.WebParam;
import java.util.ArrayList;
import java.util.List;

@WebService
public class ServicioWeb {

@WebMethod
public double suma(@WebParam(name = "a") double a, @WebParam(name = "b") double b) {
return a + b;
}

@WebMethod
public String mayusculas(@WebParam(name = "s") String s) {
return s.toUpperCase();
}

@WebMethod
public List<Integer> suma2(@WebParam(name = "a") List<Integer> a, @WebParam(name = "b") List<Integer>
↪  b) {
List<Integer> c = new ArrayList<Integer>();
for (int i = 0; i < a.size(); i++)
    c.add(a.get(i) + b.get(i));
return c;
```

```
}
}
```

En este caso, nuestro servicio web va a incluir las operaciones: suma (suma dos números punto flotante), mayúsculas (convierte a mayúsculas una cadena de caracteres) y suma2 (suma los elementos de dos listas de enteros)

1. Para compilar el servicio web ejecutamos el siguiente comando en el directorio padre del directorio ”ws”:

```
javac ws/ServicioWeb.java
```

1. Para publicar el servicio web utilizamos el programa ”Servidor.java”que funciona como servidor de aplicaciones:

```
package ws;

import javax.xml.ws.Endpoint;

public class Servidor {
public static void main(String[] args) {
Endpoint.publish("http://localhost:8080/ServicioWeb", new ServicioWeb());
}
}
```

1. Para compilar el servidor ejecutamos el siguiente comando en el directorio padre del directorio ”ws”:

```
javac ws/Servidor.java
```

1. Ahora debemos iniciar la ejecución del servidor en otra ventana de cmd en Windows o terminal de Linux o MacOS:

```
java ws/Servidor
```

1. Ahora vamos a crear el cliente que consumirá el servicio web. Vamos a utilizar el programa ”wsimport”disponible en el JDK8 el cual va a generar las clases que utilizará el cliente.

Al programa ”wsimport”se le pasa como parámetro la URL del WSDL del servicio web, en este caso la URL es ”http://localhost:8080/ServicioWeb?wsdl"

```
Para ver el WSDL de nuestro servicio web, se puede ingresar ésta URL en un navegador.
```

Nota: antes de ejecutar el siguiente comando hacer una copia del archivo ServicioWeb.java debido a que wsimport lo borra.

Ejecutar el siguiente comando en el directorio padre del directorio ”ws”:

```
wsimport "http://localhost:8080/ServicioWeb?wsdl"
```

1. Vamos a crear el archivo Cliente.java en el directorio ”ws”:

```
package ws;

import java.net.URL;
import javax.xml.namespace.QName;
import ws.ServicioWeb;
import ws.ServicioWebService;
import java.util.ArrayList;
import java.util.List;

public class Cliente {
public static void main(String[] args) throws Exception {
ServicioWebService s = new ServicioWebService(
    new URL("http://localhost:8080/ServicioWeb?wsdl"),
    new QName("http://ws/", "ServicioWebService")
);
ServicioWeb obj = s.getServicioWebPort();

System.out.println(obj.suma(100, 200));
System.out.println(obj.mayusculas("hola"));

List<Integer> a = new ArrayList<Integer>();
a.add(1);
a.add(2);
a.add(3);

List<Integer> b = new ArrayList<Integer>();
b.add(4);
b.add(5);
b.add(6);

List<Integer> c = obj.suma2(a, b);
for (int i = 0; i < c.size(); i++)
    System.out.println(c.get(i));
}
}
```

1. Para compilar el cliente ejecutamos el siguiente comando en el directorio padre del directorio ”ws”:

```
javac ws/Cliente.java
```

1. Para ejecutar el cliente:

```
java ws/Cliente
```

Nota. Debido a que JDK 8.0 tiene soporte para JAX-WS, no es necesario incluir en la compilación y la ejecución la opción -cp con el archivo .jar correspondiente a JAX-WS.